Online Resource 2: Methodology and Numerical Supplement

Supplementary information for *Pruning Deep Neural Networks via the Marchenko–Pastur Distribution*

Leonid Berlyand
Department of Mathematics
Pennsylvania State University
University Park, PA 16802, USA

Houman Owhadi
Department of Computing and Mathematical Sciences
California Institute of Technology
Pasadena, CA 91125, USA

Theo Bourdais
Department of Computing and Mathematical Sciences
California Institute of Technology
Pasadena, CA 91125, USA

Yitzchak Shmalo*
Department of Mathematics
Pennsylvania State University
University Park, PA 16802, USA

Abstract

This file is Online Resource 2 for the manuscript *Pruning Deep Neural Networks via the Marchenko–Pastur Distribution* (henceforth Main) submitted to *Neural Processing Letters*. Online Resource 1 contains the mathematical proofs and random-matrix supplement. This file contains the methodology and numerical supplement: repository architecture, projection and fine-tuning code paths, speedup-measurement protocols, migrated numerical appendices, and the full literature-context table. A main contribution of the numerical package is that the reported small accuracy drops use very limited post-pruning fine-tuning: one epoch per unstructured pruning cycle and three distillation epochs for the CAST/CAST-conv rows, rather than long retraining schedules. The main manuscript cites this file as "Online Resource 2."

Companion repository. https://github.com/yspennstate/RMT_based_pruning_in_deep_learning

Main manuscript. https://github.com/yspennstate/RMT_based_pruning_in_deep_learning/blob/main/paper/main.pdf

17-model checkpoint archive (Hybrid Magnitude–SER). https://drive.google.com/drive/folders/1mm990SHAHlYdISHxirvMRdVQEAjpIxDd

Cross-paper conventions. We refer to results in Main as Main-Theorem X.Y, Main-Table Z, etc. Norm conventions, theorem numbering, and notation are inherited from Main.

Contents

*Corresponding author: Yitzchak Shmalo. Email: yitzchak.shmalo@gmail.com.

1 Introduction and how to read this paper

1.1 Scope

MAIN develops a Marchenko–Pastur (MP)-driven theory of deterministic pruning certificates and reports results: the multi-architecture Hybrid Magnitude–SER table, the CAST 2:4 + ToMe accuracy/throughput row, and the deterministic-certificate audit. Online Resource 1 contains the proof details. This online resource contains the operational and numerical material:

1. the full RMT-pruning protocol stack (BEMA, Spectral Edge Budgeting, Spectral Edge Reallocation, Hybrid Magnitude–SER);
2. the CAST and CAST-2E pipelines (cert-aware k:n projection, Cin permutation alignment, free restoration, frozen-mask distillation);
3. the reported hardware speedup measurements and the checkpoint-preserving deployability audit that separates native sparse-backend rows from MAC-accounting rows;
4. the certification-audit numerics (three diagnostics from MAIN-§5: top-1 drop vs. audit budget, empirical CE exceedances, $\sigma_+ \to B_{\text{proxy}}$ within-cycle correlations);
5. the extended fully-connected MP-pruning experiments;
6. ledgered checkpoint validation tables.

1.2 When to read which file

Use MAIN for the theoretical statements, accuracy and speedup tables, theory-to-numerics mapping table, and high-level reproducibility narrative.

This online resource is intended to support: replicating the reported experiments, understanding the relationship between the relevant code paths in the GitHub repository and the numerical claims in MAIN, and reading the extended-numerics record.

1.3 Repository at a glance

The companion GitHub repository contains:

- Approximately 60 Python scripts at the repository root implementing the classical RMT, magnitude, Hybrid Magnitude–SER, four-regime SEB, and drop-threshold protocols (these underlie MAIN-§3 and the multi-architecture validation table);
- A self-contained `cast_2e/` subtree containing the new CAST-2E pipeline: certificate-aware k:n projection (with $k = 2, 4, 6, 8, 12$ and $n = 4, 8, 12, 16$ supported), Cin permutation alignment, in-place inline fine-tuning runners avoiding the perm-hook serialization issue, and the suite of speedup benchmarks;
- Eighteen sweep-result JSONs covering all three architecture families (ResNet50, ConvNeXtV2-Base, ViT-B/L), six k:n patterns, dense vs. SER source ablations, and α_{ser} sweeps;
- native-2:4 throughput JSONs and the checkpoint-preserving deployability audit ledger used to separate measured sparse-backend rows from MAC-accounting rows;
- All post-fine-tuning evaluation JSONs underlying the FLOP table in MAIN.

2 Repository tree and architecture

2.1 Top-level layout

Listing 1: Top-level repository layout.

```
RMT_based_pruning_in_deep_learning/
+- README.md # repo overview, quick start
+- LICENSE # MIT
+- requirements.txt # canonical deps
+- adaptive_rmt/ # RMT diagnostics package
+- configs/ # YAML/JSON sweep configs
+- model_queue_runs/ # cached SVD/ESD per model
+- scripts/ # shell launchers / watchers
+- src/ # shared utilities
+- prune.py # the pruning entrypoint
+- fine_tune.py # the fine-tuning entrypoint
+- magnitude_rmt_sweep.py # the full sweep driver
+- rmt_magnitude_sweep.py # alt sweep ordering
+- hybrid_mag20_then_v8*.py # Hybrid Magnitude--SER
+- run_finetune_*.py # the FT runners (4 variants)
+- haar_*.py # Haar-SV preprocessing
+- iterative_*.py # iterative growing-a + 5pct
+- pruning_method_comparison_sweep.py # cross-method comparison
+- run_removed_matrix_audit_v*.py # certificate audit drivers
+- analyze_sweep.py # JSON -> table aggregator
+- direct_run_watchdog.py # crash-resilient runner
+- queue_*.py # multi-run queue managers
+- remote_*.py # remote pod helpers
+- cast_2e/ # NEW (this paper's content)
   +- methodology.tex # this document
   +- code/ # cert-aware k:n + FT runners
   +- scripts/ # launch / watcher scripts
   +- benchmarks/ # A40/A100 throughput JSONs
   +- benchmarks_speedup/ # 6:12->2:4 speedup JSONs
   +- masks_archive/ # per-layer mask stats
   +- post_ft_eval/ # post-FT eval JSONs
   +- sweep_results/ # k:n sweep cell JSONs
   +- AUDIT.md # full data-flow audit
   +- THEORY.md # CAST-2E theory map
```

2.2 How to navigate by paper section

Table 1 maps each MAIN section or result to the code path that produces it.

2.3 Coding conventions in the repository

The classical RMT, magnitude, and Hybrid Magnitude–SER scripts share a common contract:

- Each entrypoint reads a YAML/JSON config from `configs/` and writes its results to a per-run subdirectory of `model_queue_runs/<arch>/<config_id>/`.
- Every fine-tune call emits both a final `finetune_results.json` and a per-epoch ledger that the analyzer can consume even if a run is killed mid-epoch.
- The SVD and BEMA edge fits are cached in `model_queue_runs/<arch>/svd_cache/` so that re-running a sweep with a different sparsity schedule does not pay the SVD cost.

Table 1: Mapping from MAIN sections and results to code paths in the GitHub repository. "[. . .]" refers to wildcards covered by the listed file or by closely related variants in the same directory.

MAIN location	Code	ONLINE RESOURCE 2 §
MAIN-Table 2 (multi-arch Hybrid Mag–SER)	`hybrid_mag20_then_v8*.py`, `run_finetune_magnitude_v3.py`, `prune.py`	§9
MAIN-Table CAST FLOPs (FLOP table)	`cast_2e/code/project_kn_sparsity.py`, `cast_2e/code/run_vitb_ft_inline.py`, `cast_2e/code/run_resnet_ft_inline.py`	§3
MAIN-§3 RMT-guided pruning	`magnitude_rmt_sweep.py`, `pruning_method_comparison_sweep.py`	§12
MAIN-§5.4 deterministic certificate (§5.4 audit)	`run_removed_matrix_audit_v8_model_exec.py`	§11
SEB-budget hyperparameter sweeps	`configs/seb_*.yaml`, `magnitude_rmt_sweep.py`, `rmt_magnitude_sweep.py`	§9
BEMA edge fitting	`adaptive_rmt/rmt_diagnostics.py` (BEMA fits)	§8
α power-law signal	`adaptive_rmt/signals.py` (σ_+, Hill α), `haar_optuna*.py`	§9, alpha-signal sweep
A100 throughput benchmark (2:4)	`cast_2e/code/benchmark_sparse_throughput.py`	§5
Deployability audit for the CAST FLOP table	`cast_2e/code/audit_checkpoint_deployability.py`, `cast_2e/code/build_deployable_speedup_ledger.py`	§5
k:n sweep driver (sweeps in this paper)	`cast_2e/code/cert_opt_eval_kn*.py`	§6
ViT-B inline FT (row 83.74)	`cast_2e/code/run_vitb_ft_inline.py`	§4
ResNet inline FT (rows 75.67, 78.00, 80.59, 81.33)	`cast_2e/code/run_resnet_ft_inline.py`	§4
ConvNeXtV2 D1216/D816 (rows 86.35, 85.85)	`cast_2e/code/project_convnextv2_d1216.py` + ToMe runner	§4

The CAST-2E scripts in `cast_2e/code/` share a different but equally explicit contract:

- All projection scripts (`project_*.py`) emit a `student_pre_ft.pt` payload containing both the projected state dict and a JSON-serializable `cert_config` dictionary.
- The fine-tuning runners are deliberately "inline": they receive the calibration loader and projection arguments directly, run projection in-process, and then immediately enter the distillation loop with the projected weights and frozen mask, without an intervening save/load round-trip. This avoids a perm-hook serialization issue described in §3.

3 Pre-fine-tuning projection: cert-aware k:n structured sparsity

This section documents the pre-FT projection family generalising CAST (the original 2:4 version is recorded in §10) from 2:4 to general k:n. The 2:4 design and Stage-1/Stage-2 split are the same as in CAST. The extension adds:

1. parametric k:n at the row-group level, with $n \in \{4, 8, 12, 16, 32\}$ and $k \in \{1, \ldots, n-1\}$;
2. Cin permutation alignment within each layer's input groups, so that the cert-cost minimisation can choose patterns from a larger set of feasible k:n blocks;
3. an SER-prior term (the α_{ser} term of §3.3) that anchors the chosen pattern to the Hybrid Magnitude–SER mask;
4. native support for both Linear and Conv2d layers, with the convolutional case implemented by unfolding the kernel.

3.1 Generalised cost: $k{:}n$ row-group search with free restoration

Fix a Linear layer $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with $d_{\text{in}} = nG_\ell$. Reshape rows into contiguous n-tuples: $W_{i,g,k}$ is the k-th weight of group g in output row i, $i \in [d_{\text{out}}]$, $g \in [G_\ell]$, $k \in [n]$.

For each pair (i, g) the $k{:}n$ search enumerates the $\binom{n}{k}$ keep patterns

$$\mathcal{M}_{k:n} = \big\{m \in \{0,1\}^n : |m|_0 = k\big\}.$$

For $n = 4, k = 2$ this is the 6 patterns of CAST (Main-§G); for $n = 16, k = 8$ it is $\binom{16}{8} = 12870$ patterns. When the enumeration becomes prohibitive, `project_kn_sampled.py` provides a sampled variant.

The materialised slot value retains CAST's three-case rule (Main-Eq. G.1):

$$v_{i,g,k}(m) = m_k \cdot \begin{cases} W^{\text{SER}}_{i,g,k}, & M^{\text{SER}}_{i,g,k} = 1, \\ W^{\text{dense}}_{i,g,k}, & M^{\text{SER}}_{i,g,k} = 0 \text{ and free-restoration enabled}, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The "dense" source corresponds to setting $W^{\text{SER}} \equiv W^{\text{dense}}$ and $M^{\text{SER}} \equiv 0$, in which case Eq. (1) reduces to $v_{i,g,k}(m) = m_k W^{\text{dense}}_{i,g,k}$.

The within-group calibration covariance and cert-inspired cost are inherited from Main-Eqs. G.2–G.3:

$$C_g = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x) h_g(x)^\top, \qquad c_{i,g}(m) = r_{i,g}(m)^\top C_g \, r_{i,g}(m), \qquad r_{i,g}(m) = (1 - m) \odot v_{i,g,:}(m). \tag{2}$$

The argmin is solved by enumeration; tensor-shape $(\binom{n}{k}, d_{\text{out}}, G_\ell)$. For $\binom{n}{k} \geq 5{,}000$ the sampled variant draws $\binom{n}{k}_{\text{eff}} = 1024$ candidates uniformly without replacement.

3.2 Cin permutation alignment ("flatten the layer")

The native $k{:}n$ search treats the n input slots within each group as fixed by the natural channel order. Channel permutations are a known way to improve $N{:}M$ sparse-network accuracy without changing inference runtime [41]; our Cin-permutation variant uses an activation-weighted quartile interleave rather than their search objective. It aligns each layer's input columns by the importance score

$$s_j = \sum_i \big|W^{\text{dense}}_{i,j}\big| \cdot |\bar{h_j}|, \qquad |\bar{h_j}| = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} |h_j(x)|,$$

and then quartile-interleaves the columns: column j in the permuted layer is the $\pi(j)$-th most important one, where π is the importance-balanced permutation that places the g-th group's k columns into quartile positions $\big(g,\ g + G_\ell/4,\ g + 2G_\ell/4,\ g + 3G_\ell/4\big)$ in the original ordering. This "flattening the layer" step is designed to make each group sample a broader importance range, giving the cert-cost minimiser an informative candidate set within every group.

The permutation is implemented as a forward pre-hook, with the inverse permutation applied at the layer output to keep downstream activations aligned. The hook is registered at projection time and removed at the post-FT save step.

3.3 The α_{ser} Hamming-distance prior

To bias the per-row cert-cost minimiser toward the Hybrid Magnitude–SER mask without overriding it, we add a Hamming penalty (Main-Eq. G.4 generalised):

$$c^{\text{prior}}_{i,g}(m) = \alpha_{\text{ser}} \cdot \bar{c}_g \cdot \big|m - M^{\text{SER}}_{i,g,:}\big|_1, \qquad \bar{c}_g = \text{mean}_{i,k}\, v^2_{i,g,k} \cdot \mathbb{E}_x\big[h_{g,k}(x)^2\big]. \tag{3}$$

Setting $\alpha_{\text{ser}} = 0$ recovers pure cert-aware projection; $\alpha_{\text{ser}} \to \infty$ forces the SER mask. We sweep $\alpha_{\text{ser}} \in \{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$ in the cell sweeps reported in §6; the empirical sweet spot is $\alpha_{\text{ser}} = 0.5$ for ViT-B with the SER-source variant.

3.4 Conv2d generalisation

For a Conv2d kernel of shape $(C_{\text{out}}, C_{\text{in}}, k_h, k_w)$ we unfold the kernel into a $(C_{\text{out}},\ C_{\text{in}} \cdot k_h \cdot k_w)$ matrix and apply the same row-group search. The projection runs on 1×1 and 3×3 convolutions; depthwise convolutions and the first input-projection conv are excluded by an `is_eligible_conv` predicate exposed via `cast_2e/code/project_kn_sparsity.py`.

In ConvNeXtV2 the projection is restricted to Linear layers (the per-block depthwise 7×7 convolutions are excluded) because the depthwise structure interacts pathologically with the row-group k:n pattern; this restriction is recorded in `cast_2e/AUDIT.md` as a known limitation.

3.5 Inline projection + FT runner for permutation-hook consistency

Projection registers Cin-permutation forward pre-hooks on the model, but those hooks are not part of the saved state dict. Reloading at FT time therefore produces a model whose weights are in the permuted ordering but whose forward pass is unpermuted; on ImageNet validation the reloaded model scores top-1 ≈ 0.0006 (random-class chance for a 1000-class problem). `cast_2e/code/run_vitb_ft_inline.py` and `cast_2e/code/run_resnet_ft_inline.py` avoid this failure mode by performing projection in-process, immediately followed by FT, without any intervening save/load round-trip. Saving the post-FT model is safe because it is consumed only at evaluation time, where the same permutation hooks are re-registered before the validation pass.

3.6 Related structured / hardware-friendly pruning literature

Beyond the foundational 2:4 sparse Tensor Core work [37] and the ToMe token merging [4] cited in the main paper, several recent works are directly relevant to CAST-2E:

- **GPUSQ-ViT** [61] couples GPU-friendly N:M sparsity with quantization for ViT, providing the closest deployment-time baseline to our cert-aware k:n projection followed by 2:4 native kernel acceleration;
- **UniPTS** [56] unifies post-training sparsity across vision architectures, addressing per-architecture tuning; our cert-aware allocator is shared across families;
- **ELSA** [21] reports uniform-2:4 and layer-wise N:4 sparsity entries for DeiT-Base under a 150-epoch training/fine-tuning budget, a complementary direction to our 3-epoch frozen-mask distillation;
- **LPViT** [58] explores block-structured, semi-structured ViT weight pruning with hardware-aware FLOP/power objectives, in the same total-MAC-reduction lane;
- **Beyond 2:4 / V:N:M** [64] generalises the 2:4 hardware pattern to V:N:M, providing one motivation for our generalised k:n projection;
- **SERo** [1] performs sparse-structure exploration, compression by deleting pruned units, and re-optimization for ViTs; it is adjacent to structured pruning but is not a reordering/permutation method.

Our work differs from each of these in starting from a deterministic certificate of the data-path budget ($B_T(R)$ of 8, empirically probed by the diagnostic audit of §11) rather than from a heuristic mask

choice, and in reporting ImageNet-1k checkpoints whose final masks pass either a native 2:4 Linear audit or an exact im2col+2:4 sparse-GEMM audit, while wider k:n rows remain MAC-accounting rows.

4 Fine-tuning recipe

4.1 Distillation, mask freezing, and the optimiser

The fine-tuning pass is a 3-epoch knowledge-distillation loop with frozen 2:4 (or general k:n) mask. The dense pretrained model is the teacher; the projected student is the student. Let $L_{\mathrm{CE}}^{\mathrm{ls}}(\hat{y}, y) = (1-\epsilon)\mathrm{CE}(\hat{y}, y) + \epsilon \cdot (-\frac{1}{K}\sum_c \log \hat{y}_c)$ be label-smoothed cross-entropy with smoothing $\epsilon = 0.1$. The distillation loss is

$$L = (1-\alpha_{\mathrm{distill}})\, L_{\mathrm{CE}}^{\mathrm{ls}}(\hat{y}_S, y) + \alpha_{\mathrm{distill}} \cdot \tau^2 \cdot \mathrm{KL}\big(\,\sigma(\hat{y}_T/\tau)\,\big\|\,\sigma(\hat{y}_S/\tau)\big), \tag{4}$$

with $\alpha_{\mathrm{distill}} = 0.5$, $\tau = 2.0$, both inherited from CAST.

After every backward step we re-zero the masked weights:

$$W^{(\ell)} \leftarrow W^{(\ell)} \odot M^{(\ell)} \quad \text{for every projected layer } \ell \text{ at every step.} \tag{5}$$

This is necessary because the optimiser's momentum and weight-decay terms can produce small nonzero values in masked positions even when the gradient is zero; without re-zeroing, the realised sparsity drifts upward by ∼0.5% across 3 epochs and the kept weights do not benefit from the freed parameter slots.

4.2 Optimiser, schedule, transforms

The exact configurations used for the four the FT runs are recorded in `cast_2e/code/run_*_ft_inline.py`; we summarise:

Table 2: Fine-tuning configuration for the the 3-epoch distillation runs.

	ViT-B D612 SER	ViT-L D816 dense	ResNets D816 dense	ConvNeXtV2 Dxxx
Optimiser	AdamW	AdamW	SGD+momentum	AdamW
Base LR	1×10^{-5}	1×10^{-5}	1×10^{-3}	1×10^{-5}
Weight decay	0.01	0.01	1×10^{-4}	0.01
Momentum	—	—	0.9	—
LR exclusions	bias / LN / pos_embed	same	—	bias / LN
Schedule	cosine + warmup	cosine + warmup	cosine + warmup	cosine + warmup
Warmup steps	1000	1000	500	1000
Batch (train)	32	16	256	32
Batch (val)	64	64	256	64
Workers	4	8	8	4
Label smooth ϵ	0.1	0.1	0.1	0.1
Distill (α, τ)	$(0.5, 2.0)$	$(0.5, 2.0)$	$(0.5, 2.0)$	$(0.5, 2.0)$

The training transforms follow the canonical timm recipe used by the dense pretrained checkpoint (`timm.data.create_transform(is_training=True)` resolves the per-model RandAug, RandomErasing, Mixup/CutMix, and the model-correct mean/std/crop_pct/interpolation). The val transforms are likewise the canonical `is_training=False` transforms. For the ViT-B/L runs this resolves to RandAug-m9 + RandomErasing $p = 0.25$ with no Mixup; for ConvNeXtV2 it adds Mixup ($\alpha = 0.8$).

4.3 Fine-tuning budget

The FT budget is matched to the CAST recipe recorded in §10.3. Three epochs keep the projection and FT contributions distinguishable in the post-FT accuracy while keeping FT cost comparable to projection cost on a per-A100 basis. We do not claim that 3 epochs is optimal; longer FTs (5–10 epochs in pilot runs) recover an additional 0.1–0.3 percentage points but at proportionally higher compute cost. The fixed budget makes the cell sweeps in §6 comparable across cells.

5 Speedup measurement

5.1 Definition of speedup

We distinguish two speedup notions; only configurations with an audited sparse backend receive a practical speedup.

Practical (hardware) speedup. The wall-clock images-per-second improvement on a specific GPU, kernel selection, and batch size. We use NVIDIA's PyTorch `torch.sparse.to_sparse_semi_structured` for the 2:4 path on Ampere GPUs. Linear rows are measured as dense-tensor versus native 2:4 Linear endpoints. ResNet Conv2d rows are additionally audited through an exact im2col+Linear lowering followed by the same native 2:4 backend. Other k:n patterns do not have native kernels at present and are reported only as theoretical FLOP reductions.

Theoretical (FLOP) speedup. The dense-equivalent multiply-adds removed by zeroing $1 - k/n$ of the weight slots in the projected layers. This is a dense-equivalent arithmetic reduction, not a hardware-speedup guarantee for future sparse kernels. The theoretical and practical numbers diverge whenever the kernel cannot saturate the structured sparsity (e.g., very small batch sizes, or layers with very small C_{out}).

5.2 Native-2:4 throughput measurements

The benchmark in `cast_2e/code/benchmark_deployable_backends.py` converts each compatible Linear layer with a 2:4 mask into a `torch.sparse.SparseSemiStructuredTensor` and runs warm-up + measurement passes at the configured batch size. For flattened-2:4 Conv2d checkpoints, it optionally lowers Conv2d to im2col+Linear, then applies the same native 2:4 Linear backend. It also includes a ResNet control that lowers only exact 1×1 Conv2d layers to Linear and leaves the remaining Conv2d layers unchanged. The reported throughput is the median after warm-up. The table below reports measurements of existing checkpoint endpoints only; no projection or fine-tuning is performed during this audit.

These are endpoint-only backend measurements. Accuracy remains the saved validation Top-1; compatible weights are converted in memory only, with no projection or fine-tuning during the audit. For the ResNet rows, the exact im2col endpoint is numerically consistent with native Conv2d on random inputs but runs at only 0.37–0.44× of cuDNN Conv2d; the native-Conv2d caveat is expanded in §5.3.

A40 batch sweeps over $64, 128, 256$ found the following best same-checkpoint native-2:4 Linear ratios: ViT-B/16 1.388× at batch 64, ViT-L/16 1.394× at batch 64, DeiT-B 1.384× at batch 64, DeiT-S 1.376× at batch 128, DeiT-T 1.330× at batch 128, and ConvNeXtV2-Base 1.295× at batch 64. These best-observed values are archived in `data/deployable_backend_best_observed_20260512.csv`; Table 3 keeps the fixed batch-128 comparison for direct row-to-row reading.

The ResNet 1×1-only control is reported in the deployability audit below; it is included only to separate sparse-GEMM feasibility from end-to-end Conv2d acceleration.

Table 3: Measured sparse-backend throughput for audited 2:4 checkpoints at batch 128. For Linear rows, deployable speedup is native 2:4 Linear divided by the dense-tensor endpoint for the same checkpoint and model graph. For ResNet rows, deployable speedup is exact im2col+native 2:4 Linear divided by dense im2col; these rows are slower than cuDNN Conv2d end-to-end and are reported as deployable sparse-GEMM audits. The ViT-B no-ToMe row is a separate A100 endpoint comparison for the original dense graph versus the same graph with 2:4 weights.

Configuration	Dense im/s	Sparse im/s	Deploy speedup	Source
ViT-B/16 + ToMe $r = 8$ (A40)	1246.1	1700.4	1.365×	`data/runpod_a40_deploy_audit_20260512/...`
ViT-B/16, no ToMe	463.6	1254.1	2.705×	`cast_2e/benchmarks/vitb_bench_with_24.json`
ViT-L/16 + ToMe $r = 8$ (A40)	659.2	900.6	1.366×	`data/runpod_a40_deploy_audit_20260512/...`
DeiT-B + ToMe $r = 8$ (A40)	1239.6	1691.5	1.365×	`data/runpod_a40_deploy_audit_20260512/...`
DeiT-S + ToMe $r = 8$ (A40)	2708.5	3725.8	1.376×	`data/runpod_a40_deploy_audit_20260512/...`
DeiT-T + ToMe $r = 8$ (A40)	5150.5	6862.2	1.332×	`data/runpod_a40_deploy_audit_20260512/...`
ConvNeXtV2-B (A40)	512.3	644.1	1.257×	`data/runpod_a40_deploy_audit_20260512/...`
ResNet50 im2col (A40)	469.5	797.9	1.700×	`data/runpod_a40_deploy_audit_20260512/...`
ResNet50d im2col (A40)	445.2	666.2	1.496×	`data/runpod_a40_deploy_audit_20260512/...`
ResNet101d im2col (A40)	228.2	357.8	1.568×	`data/runpod_a40_deploy_audit_20260512/...`
ResNet152d im2col (A40)	159.9	258.7	1.617×	`data/runpod_a40_deploy_audit_20260512/...`

We also ran controls on A100 and H100. On A100, absolute throughput was higher than on A40, but the same-checkpoint sparse/dense ratios were lower for the Linear rows; for example, at batch 128, ViT-B/16+ToMe was 2385.2 → 2660.9 images/s (1.12×) and ViT-L/16+ToMe was 1266.0 → 1525.0 images/s (1.20×). Against an already-BF16 dense endpoint, the sparse advantage was near break-even on most Linear rows. On H100 with the tested PyTorch 2.4.1 build, `torch.sparse.to_sparse_semi_structured` constructed sparse tensors but the sparse matmul failed at execution with a compute-capability error, so H100 is not used as deployability evidence for this audit. The raw files are archived under `data/deployable_backend_comparison_20260512.csv`.

5.3 Deployability audit for the FLOP table

The deployability audit is implemented by `cast_2e/code/audit_checkpoint_deployability.py`, `cast_2e/code/benchmark_deployable_backends.py`, and `cast_2e/code/build_deployable_speedup_ledger.py`. It preserves the checkpoint weights: it loads tensors, checks sparsity patterns, joins existing throughput logs and A40 backend measurements, and writes `data/paper_checkpoint_deployable_speedup_ledger_20260512.csv`. It does not rewrite weights, fine-tune models, or project a non-native checkpoint into a different sparsity pattern.

For a row to receive a measured deployable speedup in Main-Table 3, the final checkpoint must pass the backend-specific tensor audit and the benchmark log must contain a non-null sparse-backend throughput. For the PyTorch/NVIDIA semi-structured path this means exact 2:4 structure on all convertible Linear layers and successful `torch.sparse.to_sparse_semi_structured` conversion. The audited native Linear rows are ViT-B/16, ViT-L/16, DeiT-B/S/T, and ConvNeXtV2-Base canonical 2:4. The ResNet CAST-conv+perm rows receive a separate im2col+2:4 sparse-GEMM audit because the saved Conv2d tensors are flattened-2:4 exact but not TensorRT-style input-channel-2:4 exact.

For ResNet, the audit checks three Conv2d notions. The paper's MAC accounting flattens each Conv2d kernel to a matrix and verifies exact 2:4 groups in that flattened layout. TensorRT-style Conv2d sparsity is stricter: each group of four input channels must contain at most two nonzeros at every output channel and kernel pixel. The current ResNet50/50d/101d/152d CAST-conv+perm checkpoints pass the flattened MAC audit but fail the TensorRT Conv2d audit, with 913,673, 904,895, 1,700,005, and 2,356,907 bad TensorRT groups respectively. The 1×1-only Linear-lowering control gives a small sparse speedup relative to its dense Linear-lowered counterpart but remains slower than cuDNN Conv2d. Consequently the ResNet deployability number in MAIN is the exact sparse-im2col backend speedup relative to dense im2col, not a native TensorRT Conv2d speedup.

The wider 6:12, 8:16, and 12:16 rows are also kept as MAC-accounting rows. Projecting them to 2:4 would change the checkpoint and can change validation accuracy, so those exploratory projection benchmarks are not used as deployable speedup evidence for the fixed paper checkpoints.

5.4 ResNet and ConvNeXtV2 throughput

Non-2:4 ResNet and ConvNeXtV2 rows are MAC-accounting rows only. The measured 2:4 ResNet im2col and ConvNeXtV2 native-Linear endpoint values are the corresponding rows of Table 3 and are interpreted by the deployability audit above. The ConvNeXtV2 D816 and D1216 final checkpoints audit as dense tensors, so those entries remain accuracy/MAC-accounting rows; depthwise/stem convolutions and other dense operators remain unchanged.

6 The CAST-2E k:n sweeps

6.1 Sweep design

Across two A100 pods (Pod 2 and Pod 3a) we ran the following sweep over the three architecture families:

- **Architectures:** `ResNet50.tv_in1k`, `ResNet50d.ra2_in1k`, `ResNet101d.ra2_in1k`, `ResNet152d.ra2_in1k`, ViT-B/16, ViT-L/16, ConvNeXtV2-Base.
- **Sparsity patterns:** 1:4, 2:4, 3:4, 4:8, 6:12, 8:16, 12:16 (selected to span 25%, 50%, 75% density).
- **Source:** *dense* (no SER prior) and *SER* (Hybrid Magnitude–SER checkpoint at $s = 0.35$).
- α_{ser}**:** $\{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$.
- **Permutation alignment:** on / off.
- **Calibration size:** $|\mathcal{C}| \in \{64, 256, 512\}$.
- **Replicates:** 5-seed for the winners.

The driver scripts are `cast_2e/code/cert_opt_eval_kn_extended.py` (ResNet/ConvNeXtV2) and `cert_opt_eval_vitb_kn_extended.py` (ViT-B). The total sweep produced 18 result JSONs covering $\sim$200 cells. They are stored in `cast_2e/sweep_results/`.

6.2 Result pre-FT findings

Three observations.

Table 4: Pre-FT top-1 accuracy for the selected cell of each (arch, pattern, source) combination. "Dense" is the unpruned reference. Numbers are pre-FT only; the post-FT numbers are in Main-Table CAST.

Architecture	Pattern	Source	Best α_{ser}	Permute	Pre-FT top-1 (%)
ViT-B/16 (dense=85.11)	6:12	SER	0.50	on	66.24
ViT-B/16 (dense=85.11)	6:12	dense	0.00	on	65.58
ViT-B/16 (dense=85.11)	2:4	dense	0.00	off	70.43
ViT-L/16 (dense=85.84)	8:16	dense	0.00	on	78.81
DeiT-Base (dense=81.80)	6:12	SER	0.50	on	70.83
DeiT-Small (dense=79.85)	6:12	SER	0.50	on	55.61
DeiT-Tiny (dense=72.21)	6:12	SER	0.50	on	30.85
ResNet50.tv (dense=76.13)	8:16	dense	0.00	on	61.56
ResNet50d (dense=80.55)	8:16	dense	0.00	on	64.20
ResNet50.tv (dense=76.13)	4:8	dense	0.00	on	40.39
ResNet50.tv (dense=76.13)	4:8	dense	0.00	off	5.92
ResNet50d (dense=80.55)	4:8	dense	0.00	off	26.29
ConvNeXtV2-B (dense=86.72)	2:4	dense	0.00	off	81.68
ConvNeXtV2-B (dense=86.72)	12:16	dense	0.00	off	84.50
ConvNeXtV2-B (dense=86.72)	8:16	dense	0.00	off	83.42

1. *Permutation alignment is critical for ResNet50.tv* 4:8. Without permutation the cert-aware 4:8 projection scores 5.92% pre-FT, only marginally above random; with permutation it scores 40.39%, an improvement of 34.47 pp. This is the largest permutation effect we observed across architectures.

2. *For the ViT family, the SER source plus* $\alpha_{\text{ser}} = 0.5$ *is the selected sweep setting.* The 0.66 pp gain of ViT-B SER+$\alpha = 0.5$ over dense+$\alpha = 0$ at 6:12 is small pre-FT, but the post-FT gap is larger: the FT-recovered ViT-B SER+$\alpha = 0.5$ scores 83.74% post-FT, +0.5 pp above the dense+$\alpha = 0$ FT.

3. *ConvNeXtV2 prefers the dense source.* The depthwise structure of ConvNeXtV2 means the SER prior, which is built for dense projections, transfers poorly. The 12:16 dense-source cell has the highest pre-FT value in this sweep, and after FT it produces the 86.35% row of Main-Table CAST.

6.3 Cert-optimisation variance

Experimental logs show that the ResNet50 α_{ser} optimisation has range 11–19 pp at fixed configuration across calibration draws. ViT-B has variance 22–40× smaller. ResNet50 pre-FT cell rankings therefore carry an estimated ±5 pp standard deviation; the ViT-B rankings within the tested configuration have an estimated ±0.2 pp pre-FT standard deviation.

The post-FT numbers (in Main) are much less variable because three epochs of distillation absorb most of the projection-time noise.

7 Other Numerical Results

The fully connected MP-pruning numerics on Fashion MNIST and the outer-layer scaling diagnostics test the perturbative regime of the deterministic certificates; they are not a proof of the ViT pipeline.

This section collects numerical illustrations relevant to the simplified theory.

Example 7.1. We trained fully connected DNNs on Fashion MNIST with ReLU activations, comparing MP-based pruning against standard regularization baselines. MP-based pruning consisted of periodically

removing a fraction of singular values below the fitted MP edge $\sigma_+ = \sqrt{N\lambda_+}$, followed by sparsification; see Algorithm 3. Prior MNIST-scale MP/RMT-SVD pruning on fully connected DNNs is reported in [44, 3].

For a network with topology

$$[784, 3000, 3000, 3000, 3000, 500, 10].$$

trained for 100 epochs with SGD (momentum 0.9, learning rate 0.01 decayed by 0.96 every 4 epochs), every 4 epochs we retained a fraction

$$f(\text{epoch}) = \max\left(0, -\frac{1}{200}\,\text{epoch} + 1\right). \tag{6}$$

Table 5 summarizes the results. MP-based pruning combined with $L_1 + L_2$ regularization ($\mu_1 = \mu_2 = 10^{-6}$) achieves 90.53% test accuracy, exceeding any single regularization method alone in this run ($\leq 81.70\%$) and the unregularized baseline (71.64%).

Training method	Train acc. (%)	Test acc. (%)
No pruning, no regularization	78.03	71.64
MP pruning only	88.12	81.22
$L_1 + L_2$ only	87.57	81.70
MP pruning + $L_1 + L_2$	**99.82**	**90.53**
MP pruning + L_2	99.98	90.16

Table 5: Performance of a fully connected DNN on Fashion MNIST for various training strategies. MP-based pruning combined with regularization achieves higher test accuracy than either approach alone in this run.

The result scales to deeper networks: a $[784, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 10]$ model trained for 1000 epochs with the same MP pruning procedure achieves 91.37% test accuracy, compared with $< 90.5\%$ for regularization alone.

7.1 Numerical check of the outer-layer scaling condition

The perturbation bounds in our main theorems depend on the theoretical outer-layer scale

$$a_N^{\text{th}}(s) := \|W_3\|_\infty \|W_1 s\|_2 \sqrt{\frac{\log(2N)}{N}},$$

which is the quantity denoted $a(N, s)$ in Assumption 3.1. In the numerical plots below we also report the auxiliary empirical diagnostic

$$a_N^{\text{emp}}(s) := \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{3/8}}.$$

The latter is not a replacement for the theoretical scale; it is a coarser width-decay diagnostic used in these experiments. Application of the perturbation bounds requires the relevant perturbation scales to decay with width in trained networks. We check this empirically in a simple fully connected setting.

Example 7.2. We train three-layer fully connected DNNs on Fashion MNIST with topology $[784, N, N, 10]$. We vary N from 500 to 30000, use SGD with L_1 regularization, and train for 200

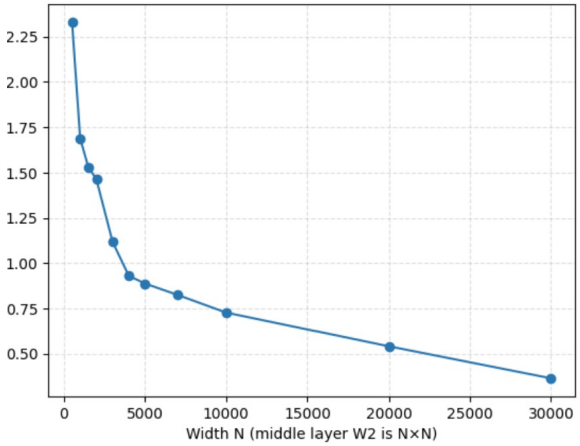
epochs. Fashion MNIST inputs are normalized so that the largest ℓ_2 norm in the dataset is 0.05. For each trained network we compute the diagnostic scale

$$\tau_N^{\mathrm{diag}} := \sqrt{2}\, a_N^{\mathrm{th}}(s) + a_N^{\mathrm{emp}}(s),$$

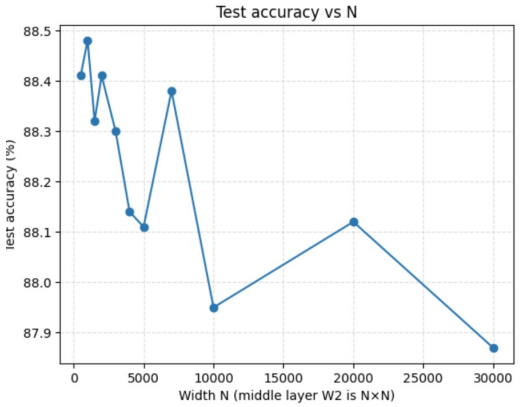
for a normalized input s of maximal norm. This diagnostic is not used in the proofs; the theorem-scale quantity is $a_N^{\mathrm{th}}(s)$.

Numerically, $\tau_N^{\mathrm{diag}} \to 0$ as N grows under $N(0, 1/N)$ initialization. With $N(0, 1/N^2)$ initialization, the decay is faster. In both cases, test accuracy increases with N.

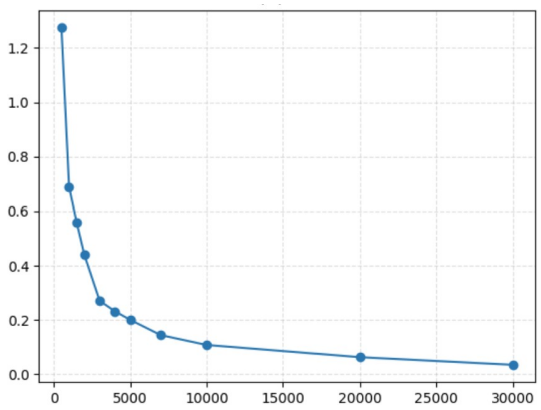
Figure 1 summarizes these width-scaling numerics. Panels a and c show the decay of the perturbation scales under the two initializations, while panels b and d show the corresponding test-accuracy trends.



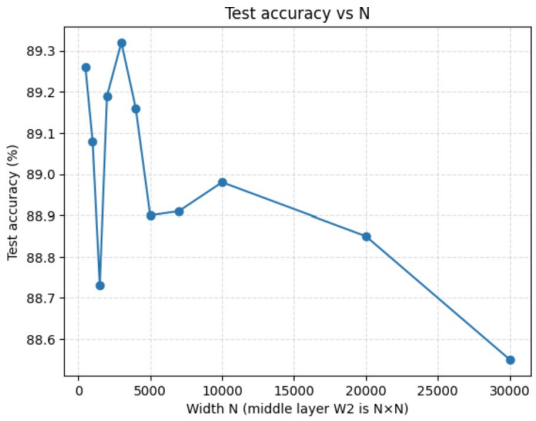
(a) $\tau(N)$ versus N under $N(0, 1/N)$ initialization.



(b) Test accuracy versus N under $N(0, 1/N)$ initialization.



(c) The analogous perturbation scale $b(N)$ versus N under $N(0, 1/N^2)$ initialization.

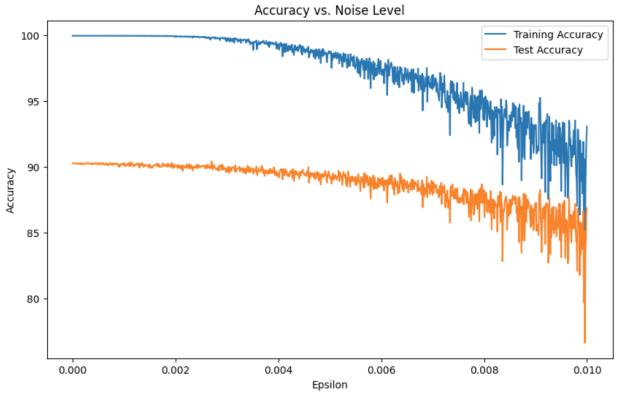


(d) Test accuracy versus N under $N(0, 1/N^2)$ initialization.

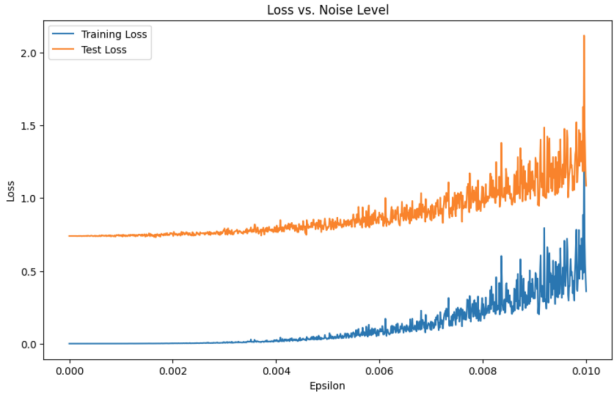
Figure 1: Width-scaling numerics for the theoretical and empirical perturbation-scale diagnostics in the fully connected Fashion MNIST experiment.

Example 7.3. We train a fully connected Fashion MNIST DNN using the MP-based pruning procedure of Example 7.1 to achieve approximately 90.5% test accuracy, then add centered Gaussian perturbations with entries $\sim N(0, \epsilon)$ to each nonfinal layer. Numerically, when $\epsilon \sim 1/N$, the cross-entropy and accuracy change little, consistent with the perturbative regime of Lemma 3.13. The L_2-regularized loss

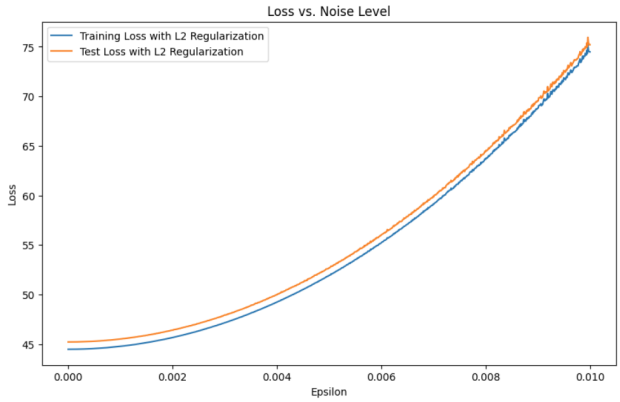
increases monotonically, illustrating the loss-reduction mechanism of Theorem 3.18. Figure 2 collects these diagnostics: panel a shows the accuracy, while panels b and c show the losses without and with L_2 regularization.



(a) Accuracy on training and test sets versus ϵ.



(b) Loss without regularization versus ϵ.



(c) Loss with L_2 regularization versus ϵ.

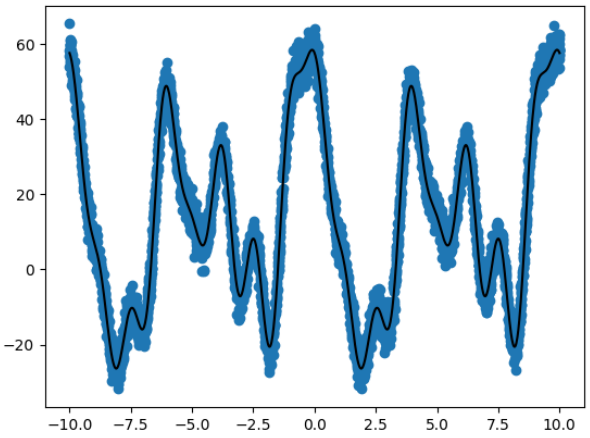
Figure 2: Accuracy and loss as centered random perturbations are added to the nonfinal weight layers of the DNN.

Remark 7.4. In a linear regression problem with noisy Fourier-generated target functions and a fixed feature map, MP-guided pruning recovers the low-rank structure more effectively than L_1 or L_2 regularization alone. Specifically, pruning achieves mean squared error 0.149, compared with 0.416 for L_1, 2.563 for L_2, and 2.574 for no regularization. The pruned spectrum closely matches the ideal low-rank spectrum, whereas L_1 and L_2 leave a visible residual bulk.

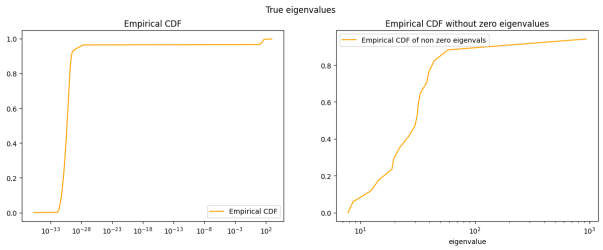
Figure 3 collects the regression illustration: panel a shows one noisy coordinate of the data, panel b shows the ideal low-rank spectrum, and panels c–e compare the L^2, L^1, and pre-pruning spectra.

8 Algorithms for RMT-based pruning diagnostics

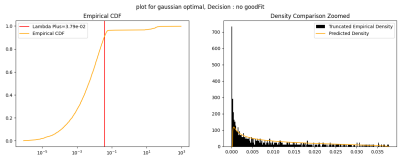
The BEMA edge-fitting algorithm and the MP-fit metric are the building blocks of every RMT-driven protocol in this paper and Main.



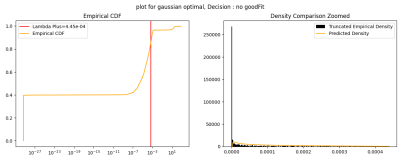
(a) One coordinate of the noisy regression data.



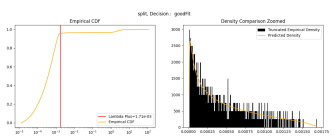
(b) Ideal low-rank spectrum.



(c) L^2-regularized spectrum.



(d) L^1-regularized spectrum.



(e) Spectrum before pruning.

Figure 3: Regression illustration for comparing pruning with more classical regularization.

8.1 BEMA-type threshold fit

We outline a BEMA-type fit that estimates the square-case MP bulk scale of $X = \frac{1}{N} W^\top W$ from its eigenvalue ESD and then forms the Tracy–Widom-corrected upper threshold used in [22]. Below is a streamlined square-matrix adaptation.

Algorithm 1 Computing a BEMA-type upper threshold T_+ using MP and Tracy–Widom corrections

procedure BEMAEdge($\lambda_1, \ldots, \lambda_N; \alpha, \beta$)
  Pick parameters $\alpha \in (0, 1/2)$, $\beta \in (0, 1)$.
  for each $\alpha N \le k \le (1-\alpha)N$ **do**
    Calculate q_k, the (k/N)-upper quantile of the MP law with $\sigma^2 = 1$ and $c = 1$.
  end for
  Evaluate

$$\widehat{\sigma}^2 = \frac{\sum_{\alpha N \le k \le (1-\alpha)N} q_k \lambda_k}{\sum_{\alpha N \le k \le (1-\alpha)N} q_k^2}.$$

  Let $t_{1-\beta}$ be the $(1-\beta)$-quantile of the Tracy–Widom distribution.
  Output

$$T_+ = \widehat{\sigma}^2 \left[4 + 2^{4/3} t_{1-\beta} N^{-2/3}\right].$$

end procedure

Algorithm 2 Spectral analysis using a BEMA-type fit

procedure SPECTRALFIT($W; \alpha, \beta, \gamma_{\text{fit}}$)
  Set $X = \frac{1}{N} W^\top W$.
  Compute the eigenvalues $\lambda_1, \ldots, \lambda_M$ of X.
  Compute the empirical CDF F_X of the eigenvalue ESD.
  Apply BEMAEDGE to estimate the upper threshold T_+.
  Let F_X^{MP} be the fitted MP CDF.
  Compute the Kolmogorov-type fit error on the central quantile window:

$$\mu_{\text{fit}} = \max_{i_{\min} \le i \le i_{\max}} \left| F_X(\lambda_i) - F_X^{\text{MP}}(\lambda_i) \right|.$$

  Accept the layer as "sufficiently MP-like" if $\mu_{\text{fit}} \le \gamma_{\text{fit}}$.
end procedure

Algorithm 3 MP-based pruning algorithm for the fully connected examples

Require: Epoch block length ℓ, MP-fit threshold τ, monotone schedule $f(\text{epoch})$, and flags split_ℓ = false for each layer.

Train the DNN for ℓ epochs and set epoch := ℓ.
while the training condition is met **do**
  for each layer W_ℓ with split_ℓ = false **do**
    Compute the SVD $W_\ell = U_\ell \Sigma_\ell V_\ell^\top$.
    Form $X_\ell = \frac{1}{N_\ell} W_\ell^\top W_\ell$ and estimate the BEMA upper threshold T_+.
    Set $\sigma_+ := \sqrt{N_\ell T_+}$.
    Test whether the bulk of X_ℓ is well fit by MP.
    if the bulk fits MP **then**
      Remove a fraction $1 - f(\text{epoch})$ of the singular values below σ_+ to obtain Σ'_ℓ.
      Form $W'_\ell = U_\ell \Sigma'_\ell V_\ell^\top$.
      Optionally factor W'_ℓ as $W'_{1,\ell} W'_{2,\ell}$ via $\sqrt{\Sigma'_\ell}$ if that reduces parameters.
    end if
  end for
  Train the DNN for another ℓ epochs and update epoch := epoch + ℓ.
end while

Algorithm 4 Sparsify singular vectors

Require: Layer matrix W_ℓ, threshold θ, fitted singular-value edge σ_+.

Compute the SVD $W_\ell = U\Sigma V^\top$.

for $i = 1, \dots, \min\{N, M\}$ **do**

Compute

$$T_i = \theta \max\left\{\frac{1}{750}, \left(1 - \frac{\Sigma_{ii}}{\sigma_+}\right)_+^{30}\right\}.$$

Sparsify the singular vectors:

$$U_{:,i} \leftarrow \mathrm{Prune}(U_{:,i}, T_i), \qquad V_{:,i} \leftarrow \mathrm{Prune}(V_{:,i}, T_i).$$

end for

Recompose $W'_\ell = U\Sigma V^\top$.

Algorithm 5 Element-wise sparsification

Require: Matrix W_ℓ, threshold ξ.

for each entry $(W_\ell)_{ij}$ **do**

if $|(W_\ell)_{ij}| < \xi$ **then**

Set $(W'_\ell)_{ij} = 0$.

else

Set $(W'_\ell)_{ij} = (W_\ell)_{ij}$.

end if

end for

Return W'_ℓ.

8.2 How well does the ESD of X fit the MP distribution?

Definition 8.1. Let G be a matrix with eigenvalue ESD μ_G when G is positive semidefinite. Its observed cumulative spectral distribution is

$$F_G(a) = \mu_G((-\infty, a]).$$

The MP-fit metric compares the observed cumulative spectral distribution with the fitted MP CDF on a central quantile interval. This focuses the fit test on the bulk and deemphasizes a few possible outliers.

8.3 A BEMA illustration and metric figures

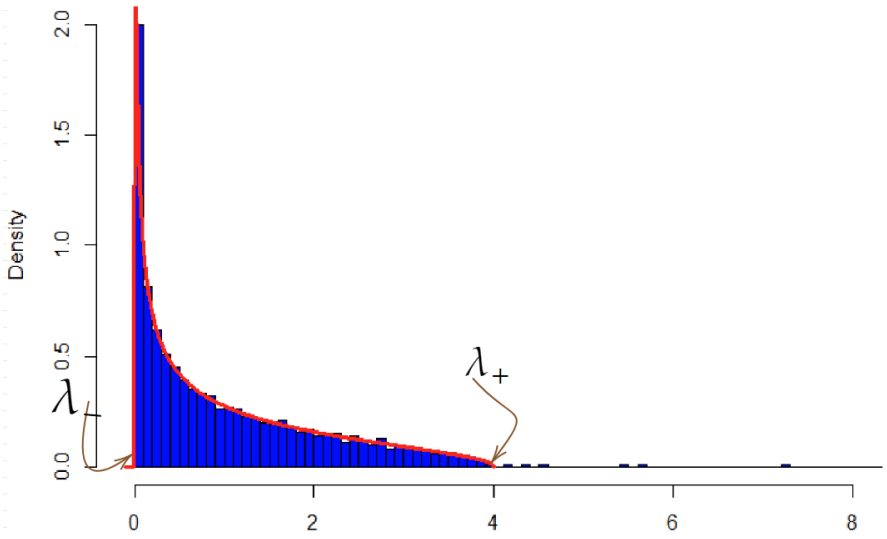
Example 8.2. Let R be an $N \times N$ matrix with i.i.d. $\mathcal{N}(0,1)$ entries, and let S be the deterministic matrix with entries

$$S[i,j] = \tan\left(\frac{\pi}{2} + \frac{1}{j+1}\right) + \cos(i)\log(i+j+1) + \sin(j)\cos\left(\frac{i}{j}\right).$$

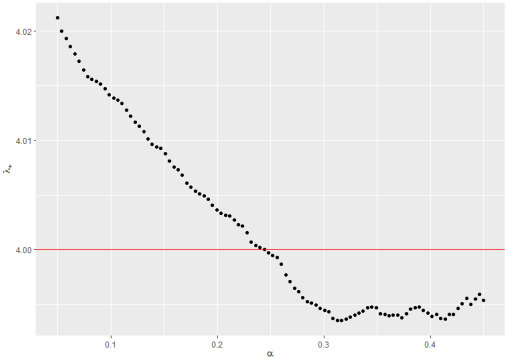
Set $W = R + S$ and $X = \frac{1}{N}W^\top W$. Figure 4a shows the ESD of X together with the fitted MP distribution produced by BEMA.

Remark 8.3. The BEMA fit depends on $\alpha \in (0, 1/2)$ and $\beta \in (0, 1)$. Figure 4 collects the synthetic BEMA diagnostics: panel a shows the fitted MP bulk in the square unit-variance example, and panels b

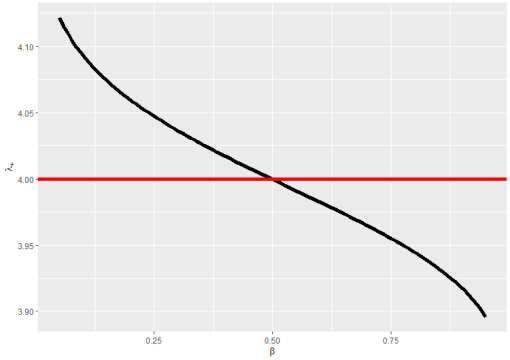
and c illustrate the dependence of the fitted upper threshold on α and β, with true covariance-scale MP edge $\lambda_+ = 4$.



(a) ESD of $X = \frac{1}{N} W^\top W$ together with the fitted MP bulk.



(b) Dependence on α, with $\beta = 0.5$.



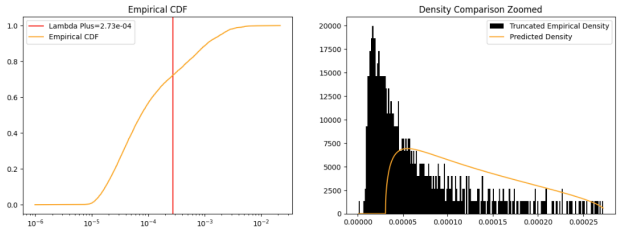
(c) Dependence on β, with $\alpha = 0.25$.

Figure 4: Synthetic BEMA diagnostics: the fitted MP bulk for $X = \frac{1}{N} W^\top W$ and the dependence of the fitted upper threshold on α and β.

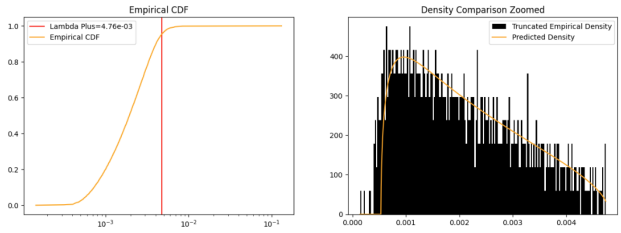
Heavy-tailed spectra are not ubiquitous in ViTs. Even though ViTs generalize well on ImageNet, their layer ESDs do *not* always exhibit heavy-tailed behavior. In many layers we observe spectra that are well explained by an MP bulk with at most a few outliers, while other layers show more power-law-like decay. This heterogeneity motivates empirical layerwise diagnosis rather than assuming a single universal spectral regime.

9 Spectral Edge Budgeting and hybrid RMT pruning protocols

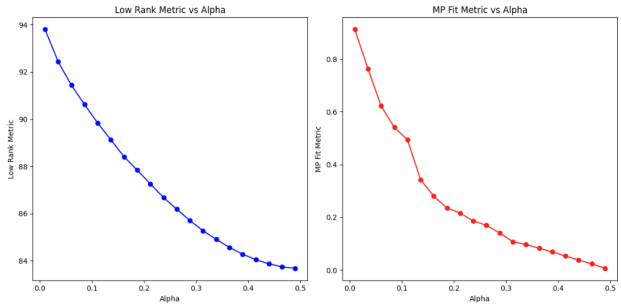
This section documents the SEB and SER protocols, the four-regime strategy, the layer-type extension, and the final SEB comparison and validation ledgers. Main-Table 2 *(multi-architecture Hybrid Magnitude–SER) is the result of this protocol family.*



(a) A layer with relatively poor MP fit: MP-fit error 0.74 and bulk fraction 73%.



(b) A layer with strong MP fit: MP-fit error 0.01 and bulk fraction 99.73%.



(c) For the base ViT model: α versus the bulk fraction and MP-fit error.

Figure 5: Layerwise MP diagnostics for the base ViT model. Panels a and b show that some layers are much closer to an MP bulk than others, while panel c shows how the bulk fraction and fit error vary with α.

This section records the operational RMT pruning protocols used in Section 3. The central method is *Spectral Edge Budgeting* (SEB): the fitted Marchenko–Pastur upper edge $\sigma_+(\ell)$ guides the *per-layer pruning budget* inside an otherwise standard global magnitude framework. We also define *Spectral Edge Reallocation* (SER), the practical prune–restore method that over-prunes, ranks a reservoir of removed weights using the same RMT diagnostics, reinserts a fixed budget, and then fine-tunes with the zero mask frozen. Finally, we define the Hybrid Magnitude–SER protocol used for broader architecture validation.

The cycle-based classical RMT procedure is a diagnostic baseline, not the final ViT pruning method. It demonstrates that MP-bulk structure is visible in trained ViT layers, but direct singular-vector or singular-value pruning is brittle in modern transformer checkpoints. The final methods use the MP fit to decide *where* magnitude pruning and restoration should occur.

9.1 Spectral Edge Budgeting

Let $W_1, \ldots, W_L$ denote the weight matrices of the target layers. For dense projections, W_ℓ is the layer matrix itself; for a convolutional kernel of shape $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w$, we unfold it as a $C_{\text{out}} \times (C_{\text{in}} k_h k_w)$ matrix before fitting the spectrum. Let $\sigma_+(\ell)$ denote the fitted Marchenko–Pastur upper edge for layer ℓ, computed from the eigenvalue distribution of $W_\ell^\top W_\ell / N$ as described in Subsection 8.2. We define the standardized spectral signal

$$z_\ell = \frac{\sigma_+(\ell) - \bar{\sigma}_+}{\text{sd}(\sigma_+)},$$

where $\bar{\sigma}_+$ and $\text{sd}(\sigma_+)$ are the mean and standard deviation of $\{\sigma_+(\ell)\}_{\ell=1}^{L}$ across all target layers. A high z_ℓ indicates that the layer's spectral bulk extends farther, so a larger share of its singular values

falls inside the MP-predicted regime and the layer has more spectral redundancy available for pruning.

Given a target global sparsity $s \in (0,1)$, modulation strength $\beta > 0$, and decay endpoint $s_{\mathrm{d}} > 0$, we define a per-layer pruning weight

$$w_\ell(s) = \max\Big(0.1,\ 1 + \beta\, d(s)\, z_\ell\Big), \tag{7}$$

where the decay factor

$$d(s) = \big[\, 1 - s/s_{\mathrm{d}} \,\big]_+ \tag{8}$$

ensures that the modulation attenuates toward uniform magnitude pruning as the sparsity target approaches s_{d}. The floor at 0.1 prevents any layer from being fully exempt.

Pruning proceeds by assigning each nonzero parameter $(W_\ell)_{ij}$ a global score

$$r_{ij}^{(\ell)} = \frac{|(W_\ell)_{ij}|}{w_\ell(s)},$$

and removing the parameters with the smallest scores until the target sparsity s is reached. Equivalently, if the global score cutoff is q, then layer ℓ is thresholded in magnitude at $q\, w_\ell(s)$. Thus $w_\ell > 1$ raises the effective magnitude threshold and causes more parameters to be pruned from that layer; conversely, $w_\ell < 1$ protects the layer. At $\beta = 0$ (or when $s \geq s_{\mathrm{d}}$), all weights equal 1 and the method reduces to plain global magnitude pruning.

9.2 Layer-type extension

A ViT contains two functionally distinct families of dense layers: *attention projections* (query–key–value and output) and *MLP layers* (the two feed-forward projections per block). We extend the modulation (7) by allowing different modulation strengths for each family:

$$w_\ell(s) = \max\Big(0.1,\ 1 + \beta_{\mathrm{type}(\ell)}\, d(s)\, z_\ell\Big), \qquad \beta_{\mathrm{type}(\ell)} = \begin{cases} \beta_{\mathrm{attn}}, & \ell \in \mathcal{L}_{\mathrm{attn}}, \\ \beta_{\mathrm{mlp}}, & \ell \in \mathcal{L}_{\mathrm{mlp}}, \\ \beta, & \text{otherwise}, \end{cases} \tag{9}$$

where $\mathcal{L}_{\mathrm{attn}}$ and $\mathcal{L}_{\mathrm{mlp}}$ partition the transformer layers by type. The z-scores remain globally computed across all layers, preserving the cross-type spectral ranking; only the *strength* of the modulation varies by type.

9.3 Alternative signal: the power-law exponent α

The heavy-tailed self-regularization (HT-SR) framework of Martin and Mahoney [36, 35] characterizes each layer by the power-law exponent α_ℓ of the tail of its eigenvalue distribution, originally estimated via maximum-likelihood power-law fitting on the squared singular values; AlphaPruning [32] uses a Hill-estimator variant ("PL Alpha Hill") as a lower-variance replacement for the MLE. Lower α_ℓ indicates a more heavy-tailed (better-trained, more structured) layer; higher α_ℓ indicates a more random-like layer. This metric is the basis of the AlphaPruning method [32], which assigns per-layer sparsity in proportion to α_ℓ.

We use α_ℓ as a drop-in replacement for $\sigma_+(\ell)$ in the budget-modulation framework (7), with the standardized signal

$$z_\ell^{\alpha} = \frac{\alpha_\ell - \bar{\alpha}}{\mathrm{sd}(\alpha)}.$$

High z_ℓ^{α} indicates a more random-like layer that should absorb more pruning—the same direction as σ_+.

Crucially, α and σ_+ are only weakly correlated ($r = 0.37$ across the 50 target layers of ViT-B/16). They capture complementary aspects of the spectral structure: σ_+ measures the extent of the MP bulk (how much of the spectrum is noise-like), while α measures the tail decay rate (how well-trained the layer is overall). This low correlation suggests that each signal may be informative in a different sparsity regime.

9.4 Four-regime strategy

In these sweeps, α gives higher top-1 than σ_+ at very low sparsity ($s \leq 0.20$), while σ_+ gives higher top-1 at moderate and high sparsity ($s \geq 0.25$). This motivates a four-regime strategy that selects the signal and hyperparameters per sparsity range:

1. **Very low sparsity** ($s \leq 0.20$): α-budget with $\beta = 0.50$, $s_{\mathrm{d}} = 0.30$. The power-law exponent provides mild but positive modulation where σ_+ is indistinguishable from magnitude pruning.
2. **Low sparsity** ($0.20 < s \leq 0.40$): σ_+-budget with $\beta = 1.00$, $s_{\mathrm{d}} = 0.50$.
3. **Moderate sparsity** ($0.40 < s \leq 0.55$): σ_+-budget with $\beta = 1.25$, $s_{\mathrm{d}} = 0.70$.
4. **High sparsity** ($s > 0.55$): Layer-type σ_+-budget with $\beta_{\mathrm{attn}} = 1.00$, $\beta_{\mathrm{mlp}} = 2.00$, $s_{\mathrm{d}} = 0.85$.

The transition from α to σ_+ at $s = 0.20$ reflects an empirical regime shift in these sweeps: at very low sparsity the binding constraint is layer-level training quality (captured by α), whereas at moderate-to-high sparsity the binding constraint is spectral redundancy (captured by σ_+).

9.5 Hyperparameter space and optimization

The method has three base hyperparameters $(\beta, s_{\mathrm{d}}, p)$ for uniform modulation (where $d(s) = [1 - s/s_{\mathrm{d}}]_+^p$ generalizes (8)), plus the optional layer-type extension $(\beta_{\mathrm{attn}}, \beta_{\mathrm{mlp}})$ and the choice of signal (σ_+ or α). We performed a systematic grid search totalling over 1,000 evaluations on the ViT-B/16 architecture over five successive sweeps:

1. **Signal comparison** (175 cells). We compared four per-layer signals—σ_+, $\sigma_+/\|W\|_F$, the excess kurtosis of $|W|$, and the coefficient of variation of $|W|$—each with $\beta \in \{0.25, 0.50, 1.00\}$, $s_{\mathrm{d}} \in \{0.50, 0.70\}$, across 7 target sparsities. Only the MP edge σ_+ produced consistent improvements over magnitude pruning; the other three signals either matched or degraded performance at every configuration tested.
2. **Decay shape and extended reach** (108 cells). Fixing the signal to σ_+, we tested decay exponents $p \in \{1, 2, 3\}$ and extended the decay endpoint to $s_{\mathrm{d}} \in \{0.70, 0.85\}$. Nonlinear decay ($p > 1$) proved uniformly harmful: at $s = 0.55$, quadratic decay ($p = 2$) lost 28 percentage points relative to linear ($p = 1$). Extending s_{d} from 0.70 to 0.85 yielded large gains at $s \geq 0.60$.
3. **Fine grid refinement** (206 cells). We explored $\beta \in \{0.75, \ldots, 1.75\}$, $s_{\mathrm{d}} \in \{0.70, \ldots, 1.05\}$, and the sub-linear regime $p \in \{0.5, 0.75, 1.0\}$. The surface is a broad diagonal ridge in (β, s_{d})-space: stronger modulation compensates for faster decay and vice versa. Sub-linear decay ($p < 1$) did not improve over linear.
4. **Definitive evaluation and layer-type extension** (434 cells). We re-evaluated the top 14 uniform-β methods at 250-batch full precision across 11 sparsities (Phase 1, 154 cells), performed an ultra-fine 9×5 grid of (β, s_{d}) at the cliff region (Phase 2, 180 cells), and ran a full 5×5 grid of $(\beta_{\mathrm{attn}}, \beta_{\mathrm{mlp}})$ at 4 sparsities (Phase 3, 100 cells).

5. **Alpha signal sweep** (77 cells). We replaced σ_+ with the Hill-estimated power-law exponent α_ℓ and tested $\beta \in \{0.25, 0.50, 1.00\}$, $s_\mathrm{d} \in \{0.25, 0.30, 0.50, 0.70\}$ across 11 sparsities. The α signal reports higher top-1 than both magnitude and σ_+ at $s \leq 0.20$ but collapses at $s \geq 0.30$, supporting the complementary-regime selection used below.

All evaluations used a fixed subset of 2,000 images from the ILSVRC-2012 validation set (250 batches of 8) unless otherwise noted; definitive results in Phase 1 used the same subset at 250-batch evaluation for reduced noise (± 0.5 pp).

9.6 Results: uniform modulation

Table 6 reports the 250-batch top-1 accuracy for the sweep-selected uniform-β configuration at each target sparsity, compared with global magnitude pruning. The selected parameters shift systematically: at moderate sparsity ($s \leq 0.55$), $s_\mathrm{d} = 0.70$ with $\beta = 1.25$ suffices; at high sparsity ($s \geq 0.60$), the decay must extend further ($s_\mathrm{d} \geq 0.85$) so that the σ_+ signal remains active through the critical region.

Table 6: Sweep-selected uniform-β σ_+-budget method versus global magnitude pruning on ViT-B/16 (no retraining). All results at 250-batch evaluation. The Δ column reports the improvement over magnitude pruning.

Sparsity s	Selected (β, s_d)	Top-1 (%)	Magnitude (%)	Δ (pp)
0.30	(1.00, 0.50)	84.90	84.25	+0.65
0.40	(1.00, 0.70)	82.80	82.75	+0.05
0.50	(1.25, 0.70)	79.30	76.20	+3.10
0.55	(1.25, 0.70)	72.85	67.70	+5.15
0.60	(1.25, 0.85)	59.85	45.00	+14.85
0.625	(1.30, 0.90)	48.45	26.60	+21.85
0.65	(1.30, 0.90)	33.25	10.75	+22.50
0.675	(1.30, 0.95)	17.35	2.55	+14.80
0.70	(1.50, 0.95)	5.80	0.70	+5.10

Key findings from the uniform-β sweeps.

1. *Linear decay was the selected schedule.* Across all 923 cells, the linear schedule $p = 1$ in (8) matched or exceeded the tested higher-order decay schedules ($p \geq 2$). Quadratic and cubic decay cause the modulation to retreat too quickly at moderate sparsity, collapsing to magnitude pruning in the regime where the spectral signal contributes in the sweep. Sub-linear decay ($p < 1$) over-modulates, hitting the $w_\ell = 0.1$ floor for too many layers.

2. *The surface is a diagonal ridge.* An ultra-fine 9×5 grid at the cliff ($s \in \{0.60, 0.625, 0.65, 0.675\}$) reveals that many (β, s_d) pairs give nearly identical accuracy. For example, at $s = 0.65$, the configurations $(\beta = 1.25, s_\mathrm{d} = 0.94)$, $(\beta = 1.35, s_\mathrm{d} = 0.91)$, and $(\beta = 1.45, s_\mathrm{d} = 0.88)$ all achieve $\approx 33.6\%$ top-1. The effective control variable is the product $\beta \cdot d(s)$, the modulation strength at the operating point.

3. *The gain is largest at the accuracy cliff.* Below $s = 0.50$, σ_+-modulation provides modest gains ($\leq$3 pp). The improvement grows rapidly beyond $s = 0.55$ and peaks at +22.5 pp near $s = 0.65$, precisely where uniform magnitude pruning exhausts the "safe" parameters and begins removing structurally important weights. The σ_+ signal identifies layers with residual capacity that magnitude pruning alone cannot exploit.

9.7 Results: layer-type modulation

Table 7 reports the 150-batch top-1 accuracy for the full 5×5 grid of $(\beta_{\text{attn}}, \beta_{\text{mlp}})$ at $s = 0.65$, with $s_{\text{d}} = 0.85$ and $p = 1$ held fixed.

Table 7: Layer-type modulation at $s = 0.65$: top-1 accuracy (%) for each $(\beta_{\text{attn}}, \beta_{\text{mlp}})$ pair. The uniform-β diagonal is shown in bold. The largest value in this grid is at $(\beta_{\text{attn}}, \beta_{\text{mlp}}) = (1.00, 2.00)$, corresponding to near-uniform magnitude pruning for attention and stronger σ_+-guided redistribution for MLP layers.

	β_{mlp}				
β_{attn}	1.00	1.25	1.50	1.75	2.00
1.00	27.50	32.75	36.00	36.17	**39.75**
1.25	27.17	32.33	34.33	37.00	38.00
1.50	24.25	30.75	**33.08**	35.25	36.92
1.75	22.58	27.67	30.42	32.17	33.67
2.00	19.00	24.92	27.58	28.67	30.25

The sweep favors asymmetric modulation: $\beta_{\text{attn}} = 1.00$ (minimal spectral modulation for attention layers) and $\beta_{\text{mlp}} = 2.00$ (strong spectral modulation for MLP layers). This configuration achieves 39.75% top-1 at $s = 0.65$, gaining +6.67 pp over the uniform-β diagonal value (33.08%) and +29.00 pp over magnitude pruning (10.75%). The layer-type gain is consistent across sparsities: +3.42 pp at $s = 0.60$, +4.92 pp at $s = 0.625$, and +5.92 pp at $s = 0.675$.

Interpretation. At high sparsity, attention projection matrices appear well served by magnitude pruning alone: the σ_+ signal provides no additional guidance and can even mislead the budget allocation for these layers. MLP layers, by contrast, exhibit within-family heterogeneity in spectral structure that the σ_+ signal captures well. Applying strong budget modulation only to MLP layers lets magnitude pruning handle attention while directing spectral guidance to the layers where it provides discriminative power.

9.8 SEB final comparison

The selected hyperparameters vary systematically with sparsity (Table 6): at low sparsity, gentle modulation with a short decay suffices; at high sparsity, the layer-type extension with strong MLP modulation dominates. This motivates the final SEB strategy, which uses the sweep-selected configuration per sparsity range. We define three regimes based on the empirical accuracy landscape:

1. **Low sparsity** ($s < 0.30$): The model retains most of its capacity and magnitude pruning remains close to the dense baseline. At this sparsity level, the per-layer redundancy distribution is not yet a binding constraint: across all 923 configurations tested, no σ_+-modulated variant consistently exceeds global magnitude pruning. Accordingly, the reported protocol uses magnitude pruning without spectral modulation for $s < 0.30$, and begins applying the σ_+-budget from $s \geq 0.30$ onward, where ($\beta = 1.00$, $s_{\text{d}} = 0.50$) yields +0.65 pp over magnitude.

2. **Moderate sparsity** ($0.40 < s \leq 0.55$): Magnitude pruning begins to lose accuracy rapidly. Uniform modulation with $\beta = 1.25$, $s_{\text{d}} = 0.70$ recaptures 1–5 percentage points by directing pruning toward spectrally redundant layers.

3. **High sparsity** ($s > 0.55$): The accuracy cliff. The layer-type extension with $\beta_{\text{attn}} = 1.00$, $\beta_{\text{mlp}} = 2.00$, $s_{\text{d}} = 0.85$ yields gains of 6–29 percentage points over magnitude pruning by preserving attention layers and concentrating spectral redistribution on MLP layers.

Table 8 compares SEB against global magnitude pruning on ViT-B/16, evaluated at every 5% sparsity increment from 5% to 75%. No retraining or fine-tuning is applied; all accuracy values are from a single forward pass on a 2,000-image subset of the ILSVRC-2012 validation set.

Table 8: Top-1 accuracy (%) on ViT-B/16 for global magnitude pruning versus SEB. At low sparsity ($s \leq 0.40$), the two methods are within the evaluation noise band; the reported σ_+-budget gap increases from $s \geq 0.45$ and reaches +28.60 pp at $s = 0.65$.

Sparsity	Regime	Magnitude (%)	σ_+-budget (%)	Δ (pp)
30%	Low	84.25	84.90	+0.65
35%	Low	83.85	83.75	−0.10
40%	Low	82.75	82.55	−0.20
45%	Mid	80.00	81.30	+1.30
50%	Mid	76.20	79.30	+3.10
55%	Mid	67.70	72.85	+5.15
60%	High	45.00	61.80	+16.80
65%	High	10.75	39.35	+28.60
70%	High	0.70	6.25	+5.55
75%	High	0.10	0.25	+0.15

In this sweep, the σ_+-budget method does not reduce top-1 by more than the 2,000-image evaluation noise band at low sparsity: the lowest gap is −0.75 pp at $s = 0.20$. This is consistent with the theoretical expectation that low-sparsity masks have small propagated effects, so the per-layer budget redistribution is a second-order effect. At higher sparsity the reported gap is largest at $s = 0.65$ (39.35% versus 10.75%).

9.9 Singular-vector sparsification as a preprocessing step

The spectral analysis in previous sections operates on the *singular values* of each weight matrix. A natural question is whether sparsifying the *singular vectors* — zeroing small components of $\mathbf{U}$ and $\mathbf{V}$ before magnitude pruning — can further improve accuracy by removing noise in the rank-one directions themselves.

We test this with a Haar-distribution test: for each weight matrix $W \in \mathbb{R}^{M \times N}$, we compute the SVD $W = U\Sigma V^\top$, then for each singular vector column, compute a z-score against the Haar (uniform-on-the-sphere) null distribution. Components with $|z| < z_{\text{base}}$ are zeroed, effectively denoising the singular subspaces before the weight matrix is reconstructed. The threshold is modulated per layer by σ_+ with power $p = 3$, following the same principle as the budget allocation: layers with larger spectral bulk have more noise in their singular vectors.

We sweep $z_{\text{base}} \in \{0.3, 0.5, 0.7, 1.0\}$ and apply the denoised model as a preprocessing step before standard global magnitude pruning. Table 9 reports the results.

The effect is marginal: the largest SV gain over the no-SV baseline is 1.65 pp (at $s = 0.65$, $z = 1.0$), and at most sparsity levels the differences are ≤ 0.25 pp — within the noise of a 2,000-image evaluation. This negative result suggests that, in these sweeps, the useful pruning signal for ViT-B/16 weight matrices was better captured by the *magnitudes* of the singular values (i.e., the spectral distribution),

Table 9: Top-1 accuracy (%) after Haar singular-vector sparsification (SV) followed by global magnitude pruning, compared to magnitude pruning alone. SV preprocessing yields small improvements at low sparsity (+0.25 pp at $s = 0.05$ for $z = 0.3$) and moderate sparsity (+0.90 pp at $s = 0.60$ for $z = 0.5$), but all gains are within or near the evaluation noise band.

Sparsity	No SV (%)	$z = 0.3$ (%)	$z = 0.5$ (%)	$z = 0.7$ (%)	$z = 1.0$ (%)
5%	86.05	**86.30**	85.95	85.80	85.90
10%	**86.10**	86.00	85.95	85.90	85.90
15%	85.70	85.75	85.70	85.85	**85.90**
20%	**85.70**	85.55	**85.75**	85.50	85.40
25%	84.85	84.90	**84.95**	**84.95**	84.60
30%	84.25	84.25	**84.45**	**84.45**	84.25
60%	45.00	45.50	**45.90**	45.60	45.55
65%	10.75	11.60	11.10	11.75	**12.40**
70%	**0.70**	**0.70**	1.00	1.10	1.30

not by the *directions* of the singular vectors. The Marchenko–Pastur edge σ_+ captures the relevant signal; further denoising of the subspace bases does not provide additional compression benefit at any sparsity level tested.

9.10 Full validation-set evaluation of SEB

The sweep tables in this appendix report accuracy from a 2,000-image validation subset, selected for speed during the hyperparameter sweep. After selecting the final configuration, we re-evaluate on the *complete* 50,000-image ILSVRC-2012 validation set, matching the standard evaluation protocol used to report the baseline accuracy of 85.11%. The checkpoint is `vit_base_patch16_224.augreg2_in21k_ft_in1k`. For the reported SEB row, we apply Haar singular-vector sparsification ($z = 0.5$, $p = 3$) as a preprocessing step followed by the final SEB budget; this corresponds to composing the modulation of Section 9.8 with the singular-vector step of Section 9.9.

The full-validation evaluation confirms the qualitative picture from the 2,000-image sweep (Table 8): at low sparsity the two methods are within measurement noise, while the SEB advantage grows sharply once magnitude pruning begins to collapse. The crossover begins at $s = 0.45$ (+0.49 pp) and the gap widens monotonically to +28.95 pp at $s = 0.65$, where magnitude pruning has dropped to 9.97% top-1 while SEB retains 38.92%. Beyond $s = 0.70$, neither method is practically useful without fine-tuning, and the differences compress accordingly. These are the final sweep-selected no-fine-tuning spectral-guided values; SER extends the same spectral idea to a fine-tuned prune–restore pipeline.

9.11 Spectral Edge Reallocation

SEB is a one-shot budget allocation method. SER turns the same spectral signal into a practical prune–restore pipeline. For a target sparsity s, the method first prunes past the target, producing a reservoir $\mathcal{R}$ of removed entries. During a short selection phase, only the removed entries are trainable; the method records their absolute movement from the pre-selection reservoir value. An entry (i, j) in layer ℓ is then ranked by

$$|\Delta_{ij}^{(\ell)}|\, \rho_\ell(s),$$

where $\rho_\ell(s)$ is the RMT restoration multiplier obtained from the same layerwise spectral weights used for pruning. Thus entries that move strongly during the selection phase and belong to spectrally

Table 10: Full 50,000-image validation evaluation of SEB with Haar singular-vector sparsification ($z = 0.5$, $p = 3$) versus global magnitude pruning on ViT-B/16. Baseline top-1 accuracy: 85.11%. No fine-tuning or retraining is applied. Bold entries mark sparsities at which SEB is higher than magnitude by ≥ 1 pp.

Sparsity s	Regime	Magnitude (%)	SEB (%)	Δ (pp)
5%	Low	85.09	85.08	−0.01
10%	Low	85.06	85.02	−0.04
15%	Low	85.02	84.85	−0.17
20%	Low	84.89	84.56	−0.34
25%	Low-mid	84.58	84.53	−0.05
30%	Low-mid	84.07	84.30	+0.23
35%	Low-mid	83.45	83.66	+0.21
40%	Low-mid	82.23	82.24	+0.01
45%	Mid	80.20	80.70	+0.49
50%	Mid	76.67	**78.55**	+1.88
55%	Mid	68.41	**72.68**	+4.28
60%	High	46.42	**61.57**	+15.15
65%	High	9.97	**38.92**	+28.95
70%	High	0.82	**6.11**	+5.30
75%	High	0.20	0.35	+0.15

protected layers receive higher restoration priority, while entries from layers with larger MP-bulk pruning capacity are less likely to be restored. A fixed restoration budget is reinserted, and the final mask is adjusted so that the realized post-reinsertion sparsity is exactly s. The surviving weights are then fine-tuned for a short schedule with the zero mask frozen.

The connection to the theory is the same as for SEB, but applied twice. The pruning stage uses MP information to allocate perturbation budget across layers; the reallocation stage uses the same information to choose which removed perturbations were too costly and should be restored. The deterministic certificates of Section 5 apply after the mask is fixed, while the MP diagnostics supply a data-independent ranking rule for finding masks that are more likely to satisfy those certificates.

9.12 Hybrid Magnitude–SER

The sweep above shows that RMT modulation is not needed at very low sparsity: below about 20%, global magnitude pruning is already within the noise band of the spectral rules tested here. The hybrid method therefore uses exact global magnitude pruning for $s \leq 0.20$, saves those checkpoints, and starts SER only for the continuation $s > 0.20$. Its purpose is conservative: preserve the low-sparsity behavior of magnitude pruning, then use RMT only in the regime where layerwise redundancy becomes the limiting factor.

9.13 Checkpoint validation ledger

The main tables report rounded top-1 accuracy values from the archived validation files associated with the saved checkpoints. Most entries come directly from `results.json` or `finetune_results.json`; where a later restart overwrote a partial JSON ledger, the local archived stdout validation line and the saved checkpoint are used. When both a prefix file and the integrated run file contain the same

exact-magnitude prefix checkpoint, the integrated run file is used in the ledger; prefix-only rows are included only when no later integrated validation file exists. Entries are post-fine-tuning ImageNet-1k validation top-1 accuracy in percent. Stopped exploratory rows with no validated nonzero checkpoint are omitted from Table 2.

Table 11: Checkpoint-level validation ledger for the Hybrid Magnitude–SER multi-architecture results. These are the values used in Table 2.

Architecture/checkpoint	Dense	Validated sparsity–top-1 pairs
ViT-B/16 `augreg2_in21k_ft_in1k`	85.11	0.05 : 85.23, 0.10 : 85.17, 0.15 : 85.06, 0.20 : 84.89, 0.25 : 84.81, 0.30 : 84.64, 0.35 : 84.55, 0.40 : 84.28, 0.45 : 83.80, 0.50 : 83.37, 0.55 : 82.76, 0.60 : 81.39, 0.65 : 79.95, 0.70 : 78.01
ViT-B/16/384 `augreg_in21k_ft_in1k`	86.01	0.05 : 86.05, 0.10 : 86.09, 0.15 : 86.07, 0.20 : 85.99, 0.25 : 86.00, 0.30 : 85.90, 0.35 : 85.81, 0.40 : 85.78, 0.45 : 85.48, 0.50 : 85.15, 0.55 : 84.64, 0.60 : 83.35, 0.65 : 82.50, 0.70 : 81.33
ViT-Large/16 `augreg_in21k_ft_in1k`	85.84	0.05 : 85.80, 0.10 : 85.84, 0.15 : 85.88, 0.20 : 85.75, 0.25 : 84.98, 0.30 : 85.32, 0.35 : 85.64, 0.40 : 85.04, 0.45 : 85.08, 0.50 : 84.52, 0.55 : 84.34, 0.60 : 83.81, 0.65 : 83.02, 0.70 : 82.13
DeiT-Tiny `patch16_224.fb_in1k`	72.21	0.05 : 72.41, 0.10 : 72.38, 0.15 : 72.18, 0.20 : 71.86, 0.25 : 71.99, 0.30 : 71.75, 0.35 : 71.35, 0.40 : 70.66, 0.45 : 68.97, 0.50 : 67.74, 0.55 : 65.91, 0.60 : 61.17, 0.65 : 55.58, 0.70 : 52.55
DeiT-Small `patch16_224.fb_in1k`	79.85	0.05 : 79.82, 0.10 : 79.78, 0.15 : 79.62, 0.20 : 79.37, 0.25 : 79.31, 0.30 : 79.21, 0.35 : 79.00, 0.40 : 78.61, 0.45 : 78.06, 0.50 : 77.22, 0.55 : 76.25, 0.60 : 74.14, 0.65 : 72.05, 0.70 : 68.55
DeiT-Base `patch16_224.fb_in1k`	81.80	0.05 : 81.76, 0.10 : 81.70, 0.15 : 81.58, 0.20 : 81.46, 0.25 : 81.42, 0.30 : 81.28, 0.35 : 81.17, 0.40 : 80.99, 0.45 : 80.62, 0.50 : 80.12, 0.55 : 79.49, 0.60 : 78.16, 0.65 : 76.72, 0.70 : 74.90
Swin-Tiny `patch4_window7_224.ms_in1k`	81.20	0.05 : 81.32, 0.10 : 81.28, 0.15 : 81.25, 0.20 : 81.05, 0.25 : 81.09, 0.30 : 80.91, 0.35 : 80.78, 0.40 : 80.37, 0.45 : 80.14, 0.50 : 79.80, 0.55 : 78.98, 0.60 : 78.06, 0.65 : 76.64, 0.70 : 74.47
ConvNeXt-Base `fb_in22k_ft_in1k`	85.84	0.05 : 85.72, 0.10 : 85.58, 0.15 : 85.51, 0.20 : 85.26, 0.25 : 85.35, 0.30 : 85.39, 0.35 : 85.26, 0.40 : 85.04, 0.45 : 84.94, 0.50 : 84.80, 0.55 : 84.09, 0.60 : 83.70, 0.65 : 83.11, 0.70 : 81.21
ConvNeXtV2-Base `fcmae_ft_in22k_in1k`	86.72	0.05 : 86.67, 0.10 : 86.58, 0.15 : 86.43, 0.20 : 86.27, 0.25 : 85.93, 0.30 : 85.97, 0.35 : 85.96, 0.40 : 85.71, 0.45 : 85.58, 0.50 : 85.33, 0.55 : 84.81, 0.60 : 84.61, 0.65 : 83.83, 0.70 : 81.40
ResNet50d `ra2_in1k`	80.55	0.05 : 79.90, 0.10 : 80.01, 0.15 : 80.02, 0.20 : 80.12, 0.25 : 80.09, 0.30 : 80.12, 0.35 : 80.06, 0.40 : 79.90, 0.45 : 79.48, 0.50 : 79.16, 0.55 : 78.93, 0.60 : 77.93, 0.65 : 77.25, 0.70 : 76.32
ResNet101d `ra2_in1k`	82.26	0.05 : 82.06, 0.10 : 82.05, 0.15 : 82.15, 0.20 : 82.15, 0.25 : 82.06, 0.30 : 82.07, 0.35 : 81.93, 0.40 : 81.97, 0.45 : 81.61, 0.50 : 81.56, 0.55 : 81.47, 0.60 : 80.45, 0.65 : 80.15, 0.70 : 79.82
ResNet50 `tv_in1k`	76.13	0.05 : 75.85, 0.10 : 75.33, 0.15 : 75.72, 0.20 : 76.07, 0.25 : 76.17, 0.30 : 76.13, 0.35 : 76.13, 0.40 : 76.14, 0.45 : 75.75, 0.50 : 75.76, 0.55 : 75.70, 0.60 : 74.83, 0.65 : 74.62, 0.70 : 74.15
ResNet34 `tv_in1k`	73.28	0.05 : 73.00, 0.10 : 72.67, 0.15 : 72.39, 0.20 : 72.62, 0.25 : 73.06, 0.30 : 73.09, 0.35 : 73.16, 0.40 : 73.16, 0.45 : 73.00, 0.50 : 72.88, 0.55 : 72.74, 0.60 : 72.12, 0.65 : 71.59, 0.70 : 71.44
ResNet18 `tv_in1k`	69.76	0.05 : 69.73, 0.10 : 69.52, 0.15 : 69.18, 0.20 : 68.94, 0.25 : 69.02, 0.30 : 69.28, 0.35 : 69.50, 0.40 : 69.50, 0.45 : 69.39, 0.50 : 69.12, 0.55 : 69.00, 0.60 : 68.31, 0.65 : 67.82, 0.70 : 67.23
Hiera-Base+ `mae_in1k_ft_in1k`	84.40	0.05 : 85.04, 0.10 : 84.94, 0.15 : 84.76, 0.20 : 84.39, 0.25 : 77.95, 0.30 : 83.12, 0.35 : 82.15, 0.40 : 82.37, 0.45 : 82.29, 0.50 : 82.25, 0.55 : 82.00, 0.60 : 80.03, 0.65 : 80.55, 0.70 : 78.96

9.14 Classical RMT baseline

The "Classical RMT" entry in Table 1 refers to the cycle-based procedure: fit an MP edge in each layer, treat sub-edge singular components as bulk-like, sparsify small singular-vector coordinates, and then prune coefficients directly. This method is conceptually close to SVD compression and RMT denoising work [44, 46], but it is not the final ViT method in this paper. Its role is to establish that MP structure is present and exploitable; the stronger SEB, SER, and Hybrid Magnitude–SER protocols use the MP edge as a layerwise allocation signal rather than as a hard instruction to delete all sub-edge spectral components.

10 CAST: certificate-aware 2:4 + ToMe conversion

This section gives the 2:4*+ToMe CAST pipeline that underlies the deployable ViT-B/16 row in* Main. *The Stage-1 cost and per-row argmin are the special* 2:4 *case of the general CAST-2E k:n projection described in §3.*

CAST (**C**ertificate-**A**ware **S**parse-**T**oken conversion) is the procedure used to produce the second row of Table 17. This section records the algorithmic details, the connections to the deterministic

Table 12: Downloaded threshold and restart validation ledgers saved locally. These rows give the provenance for drop-threshold and restart trajectories, including several rows consolidated into Table 2.

Run	Dense	Validated sparsity–top-1 pairs
ViT-Large/16 seeded drop-threshold audit run	85.84	0.05 : 85.80, 0.10 : 85.84, 0.15 : 85.88, 0.20 : 85.75, 0.25 : 85.58, 0.30 : 85.42, 0.35 : 85.22, 0.40 : 84.71
ConvNeXtV2-Base drop-threshold ledger	86.72	0.05 : 86.67, 0.10 : 86.58, 0.15 : 86.43, 0.20 : 86.27, 0.25 : 85.93, 0.30 : 85.97, 0.35 : 85.96, 0.40 : 85.71, 0.45 : 85.58, 0.50 : 85.33, 0.55 : 84.81, 0.60 : 84.61, 0.65 : 83.83, 0.70 : 81.40
ViT-Small/16 archived partial ledger	81.40	0.05 : 81.14, 0.10 : 81.05, 0.15 : 80.91, 0.20 : 80.71, 0.25 : 80.11, 0.30 : 80.29, 0.35 : 79.88, 0.40 : 79.29, 0.45 : 77.99, 0.50 : 76.54
DeiT-Base restart ledger	81.80	0.05 : 81.79, 0.10 : 81.69, 0.15 : 81.54, 0.20 : 81.44, 0.25 : 81.25, 0.30 : 81.35, 0.35 : 81.18, 0.40 : 80.95, 0.45 : 80.68, 0.50 : 80.12, 0.55 : 79.48, 0.60 : 78.24, 0.65 : 76.70, 0.70 : 74.78
Swin-Tiny restart ledger	81.20	0.05 : 81.32, 0.10 : 81.29, 0.15 : 81.26, 0.20 : 81.05, 0.25 : 80.77, 0.30 : 80.47, 0.35 : 80.58, 0.40 : 80.43, 0.45 : 80.11, 0.50 : 79.80, 0.55 : 78.94, 0.60 : 78.03, 0.65 : 76.53, 0.70 : 74.31

Table 13: Auxiliary ViT-B/16 checkpoint validations archived with the main runs. These rows document additional RMT/SER sweeps and exact-magnitude-to-SER variants; the Hybrid Magnitude–SER row is the one reported in Tables 1 and 2.

Archived ViT-B/16 run	Validated sparsity–top-1 pairs
SER, one-epoch selection sweep	0.05 : 85.17, 0.10 : 85.07, 0.15 : 85.01, 0.20 : 84.93, 0.25 : 84.86
SER, two-epoch selection sweep	0.05 : 85.18, 0.10 : 85.12, 0.15 : 84.98, 0.20 : 84.91, 0.25 : 84.84, 0.30 : 84.54, 0.35 : 84.36, 0.40 : 83.85, 0.45 : 83.25, 0.50 : 82.19, 0.55 : 81.03, 0.60 : 78.12, 0.65 : 73.80, 0.70 : 67.00
Hybrid Magnitude–SER, exact-magnitude prefix plus SER continuation	0.05 : 85.23, 0.10 : 85.17, 0.15 : 85.06, 0.20 : 84.89, 0.25 : 84.81, 0.30 : 84.64, 0.35 : 84.55, 0.40 : 84.28, 0.45 : 83.80, 0.50 : 83.37, 0.55 : 82.76, 0.60 : 81.39, 0.65 : 79.95, 0.70 : 78.01
Hybrid Magnitude–SER, alternate low-sparsity run	0.05 : 85.02, 0.10 : 84.96, 0.15 : 84.96, 0.20 : 84.77, 0.25 : 84.67, 0.30 : 84.70

certificate framework of Section 5, and the role played by the RMT/SER pipeline that supplies CAST's input checkpoint.

10.1 Inputs and stages

CAST takes three inputs:

1. A Hybrid Magnitude–SER sparse checkpoint at sparsity $s = 0.35$, produced by the SEB pipeline of Appendix 9.12. The unstructured mask of this checkpoint is denoted M^{SER}; its construction uses the per-layer Marchenko–Pastur edge signal $\sigma_+(\ell)$ (Appendix 9.1, Eq. (7)) to allocate per-layer pruning weights, so the input to CAST is already a product of the RMT-based pipeline.

2. The dense pretrained ViT-B/16 (`vit_base_patch16_224.augreg2_in21k_ft_in1k`, top-1 85.11%). Used both as the source of restored slot values and as the distillation teacher.

3. A small calibration set $\mathcal{C} \subset \mathcal{X}_{\mathrm{train}}$ ($|\mathcal{C}| = 256$ images sampled deterministically from the ImageNet train shards; no validation-set images are used).

CAST runs in two stages:

Stage 1 — Calibration-only mask minimization (zero gradients). For each compatible Linear layer ℓ (in-channel dimension divisible by 4; we exclude the classification head), capture the input activations entering ℓ over $\mathcal{C}$, then solve a small discrete optimization problem per output-row, per input-group of 4 weights. Materialize the chosen weights (taking the SER value at SER-kept slots and the dense pretrained value at slots restored within a 2:4 group). Save the resulting student state dict and the per-layer 2:4 masks.

Stage 2 — Frozen-mask distillation (3 epochs). Apply ToMe $r = 8$ [4] to the model, then fine-tune for exactly three epochs of distillation against the dense teacher with the Stage-1 mask frozen.

The "2:4 budget" that motivates the design is hardware: NVIDIA Sparse Tensor Core 2:4 [37] pays for the 2-of-4 pattern regardless of whether the two kept slots are numerically nonzero, so once a layer is in 2:4 form, restoring an additional non-zero entry inside an existing 4-tuple is free in sparse-execution FLOPs.

10.2 Stage 1: per-row 2:4 search with free restoration

Fix a Linear layer ℓ with weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $d_{\text{in}} = 4G_\ell$. Reshape the rows of W into contiguous 4-tuples: write $W_{i,g,k}$ for the k-th weight of group g in output row i, $i \in [d_{\text{out}}]$, $g \in [G_\ell]$, $k \in [4]$. Let $W^{\text{SER}}_{i,g,k}$ denote the SER input checkpoint, $W^{\text{dense}}_{i,g,k}$ the dense pretrained checkpoint, and $M^{\text{SER}}_{i,g,k} = \mathbf{1}\{W^{\text{SER}}_{i,g,k} \neq 0\} \in \{0,1\}$.

For each (i, g) we search over the six 2-of-4 keep patterns

$$\mathcal{M}_{2:4} = \big\{m \in \{0,1\}^4 : |m|_0 = 2\big\}, \qquad |\mathcal{M}_{2:4}| = \binom{4}{2} = 6.$$

For a chosen pattern m we materialize the slot value via

$$v_{i,g,k}(m) \;=\; m_k \cdot \begin{cases} W^{\text{SER}}_{i,g,k} & \text{if } M^{\text{SER}}_{i,g,k} = 1, \\ W^{\text{dense}}_{i,g,k} & \text{if } M^{\text{SER}}_{i,g,k} = 0 \text{ and "free restoration" is enabled,} \\ 0 & \text{otherwise.} \end{cases} \tag{10}$$

Eq. (10) formalises CAST's two semantic rules: SER-kept slots retain their fine-tuned SER values; slots that the unstructured mask had zeroed but the chosen 2:4 pattern wishes to keep are *restored* to the dense pretrained value, at zero sparse-execution FLOP cost. This realises the prune–restore framing of Theorem 5.5 at the 2:4-pattern granularity: the kept reservoir Q is exactly the set of restored slots, and the budget reduction $B_T(P)$ decreases when restoration is selected.

The chosen pattern minimises a Lemma 5.1-inspired activation-weighted cost on the calibration set. Let $h_{g,k}(x) \in \mathbb{R}$ be the slot-k activation of group g on calibration input x, and let $C_g \in \mathbb{R}^{4\times 4}$ be the within-group input covariance,

$$C_g \;=\; \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x)\, h_g(x)^\top, \qquad h_g(x) := \big(h_{g,1}(x), \ldots, h_{g,4}(x)\big). \tag{11}$$

Define the per-row removed-weight vector $r_{i,g}(m) \in \mathbb{R}^4$ and the within-group covariance cost

$$r_{i,g}(m) \;=\; (1-m) \odot v_{i,g,:}(m), \qquad c_{i,g}(m) \;=\; r_{i,g}(m)^\top C_g\, r_{i,g}(m) \;=\; \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} \Big(\sum_{k : m_k = 0} v_{i,g,k}(m)\, h_{g,k}(x) \Big)^2. \tag{12}$$

Eq. (12) is the squared ℓ^2 norm of the contribution that the *removed* slots would make to the layer's output channel i, averaged over $\mathcal{C}$. In particular, summing $c_{i,g}(m)$ over all output rows and groups gives the block-separable squared-ℓ^2 proxy used to rank layer-output perturbations $R\,h$; it is not the literal $L_s \|R\psi_1(s)\|_\infty$ budget. CAST's cost is therefore the ℓ^2 form of the same algebraic object the lemma controls in ℓ^∞; it is *certificate-inspired*. The reported implementation does not rescore the highest-risk groups with the literal ℓ^∞ form.

Finally CAST adds a small SER-prior term scaled to the cost magnitude,

$$c_{i,g}^{\text{prior}}(m) \;=\; \alpha \cdot \bar{c}_g \cdot \big|\, m - M_{i,g,:}^{\text{SER}} \big|_1, \qquad \bar{c}_g = \text{mean}_{i,k}\, v_{i,g,k}^2\, \mathrm{E}_x[h_{g,k}(x)^2], \tag{13}$$

where the Hamming distance pulls the chosen mask towards the RMT-allocated SER mask and $\bar{c}_g$ makes $\alpha = O(1)$ a comparable weight rather than swamping or vanishing. The full per-(row, group) optimisation is

$$m_{i,g}^{\star} \;=\; \arg\min_{m \in \mathcal{M}_{2:4}} \Big[c_{i,g}(m) \;+\; c_{i,g}^{\text{prior}}(m) \Big], \tag{14}$$

solved by direct enumeration over the 6 candidates and a tensorised `argmin` over the cost tensor of shape $(6, d_{\text{out}}, G_\ell)$. The search is *per output row*: each (i, g) cell receives its own keep pattern, not a single pattern shared across all rows in input-group g. This matches what NVIDIA 2:4 hardware admits; sharing one pattern per group across rows — the case in the exploratory implementation — discards a large fraction of the available 2:4 flexibility and is empirically worse.

The activation capture in Eq. (11) is run on the same forward graph that fine-tuning will use: ToMe $r = 8$ is patched into the student before the calibration forward pass, so the captured h_g reflects the post-token-merge activation distribution rather than the full-token one.

10.3 Stage 2: frozen-mask distillation and the runner-side mask handoff

Stage 1 emits a state dict containing the Stage-1 student weights and the per-layer 2:4 mask dictionary $\{M_\ell^{\text{CAST}}\}$. Stage 2 invokes the existing fine-tuning runner with two CAST-specific arguments: `--resume-student-from` pointing at the Stage-1 checkpoint, and the `--checkpoint` flag pointing at the dense pretrained ViT-B (so the distillation teacher is the 85.11% model rather than the $s = 0.35$ sparse source).

The fine-tuning runner ordinarily derives its internal mask dictionary from a magnitude top-2 projection of the Linear weights (the same procedure as the first row of Table 17). When CAST resumes, the runner instead consumes the $\{M_\ell^{\text{CAST}}\}$ dictionary embedded in the Stage-1 payload and uses that as its training-time mask: every `apply_masks` and `mask_gradients` call at every gradient step is therefore against the certificate-aware row-wise 2:4 pattern, not against the magnitude pattern. This handoff ensures that the fine-tuning stage preserves CAST's per-row pattern and, in particular, does not silently zero out the slots restored in Eq. (10). The implementation is a small, backward-compatible extension to the runner: payloads without a $\{M_\ell^{\text{CAST}}\}$ entry fall through to the runner's original magnitude behaviour unchanged.

The remaining hyperparameters — learning rate 10^{-5} cosine decay, label smoothing 0.1, distillation $\alpha = 0.5$, distillation temperature 2.0, 3 epochs, batch size 32 — match the magnitude-2:4 row of Table 17 so that the only difference between the two rows is the Stage-1 mask choice and the slot-value semantics of Eq. (10).

10.4 CAST contribution and limitations

At a fixed compute budget (7.06G sparse-execution MACs, identical to the magnitude-2:4 row), a fixed token count (ToMe $r = 8$), identical fine-tuning schedule, and the same executable native-2:4 path, the certificate-aware mask choice and the free-restoration step together recover an additional $\sim$0.49 pp top-1 on full ImageNet val. The saved CAST checkpoint runs at 1700.4 images/s on A40 after native 2:4 conversion. Ablations indicate that both ingredients matter: the row-wise `argmin` avoids the loss from sharing one pattern per input group across all rows, and the slot-value semantics of Eq. (10) preserve SER fine-tuning at SER-kept slots rather than overwriting every selected slot with the dense value.

Current implementation limits:

- The cost in Eq. (12) is the ℓ^2 form of Lemma 5.1's removed-contribution quantity, not its literal ℓ^∞ form. Exact ℓ^∞ rescoring of the highest-risk groups is not part of the reported implementation.
- The post-layer Lipschitz factor L_s of Lemma 5.1 is held constant; per-sample estimates would require an additional backward pass.
- The 2:4 search is per-layer; Theorem 5.8's multi-layer bound is a sum that we minimise greedily, not jointly. The greedy choice is justified because changing one layer's 2:4 mask does not materially shift another layer's input distribution at this calibration depth; joint minimisation is not evaluated here.
- Per-layer learned token thresholds (an LTMP-flavoured variant in which each block's merge count r_ℓ is tuned on the calibration set) are not used; CAST uses a fixed ToMe $r = 8$ at every block.

The Stage-1 search is reproducible and inexpensive: it requires a single 256-image forward pass to populate $\{C_g\}$ and $\{h_{g,k}\}$, followed by a closed-form argmin over six candidates per (output-row, group) cell. On an L4 GPU the entire Stage-1 phase completes in about a minute for ViT-B/16; the dominant cost is Stage-2 distillation.

11 Certification audit numerics

This section gives the certification-audit numerics in a single self-contained reference. The audit reports three empirical diagnostics motivated by MAIN-Section 5 on 18 trained models. These diagnostics track the audit-emitted budget or proxy against measured outcomes; they are not additional proofs of the deterministic certificate.

11.1 The three bridges

Recall from MAIN-Theorem 5.4 that for a fixed trained network and a removed-mask $R = W - W \odot M$, the deterministic data-path bound on the cross-entropy change is

$$B_T(R) = \frac{2}{|T|} \sum_{s \in T} L_s \, \|R \cdot \psi_1(s)\|_\infty, \tag{15}$$

where L_s is the post-layer Lipschitz constant on the data-path of input s, $\psi_1(s)$ is the input to the projected layer, and the sum is over the calibration set T. MAIN-Theorem 5.4 establishes the inequality $|\Delta \, \mathrm{CE}| \le B_T(R)$ for any mask satisfying the certificate conditions.

Bridge 1 (top-1 drop vs. audit budget). If MAIN-Theorem 5.4 is informative for pruning behavior, then within a single architecture and a single calibration set, larger audit budgets should be associated with larger post-projection top-1 drops. We measure the within-architecture Pearson correlation between $B_T(R_\ell)$ summed over projected layers and the realised pre-FT top-1 drop across cells.

Bridge 2 (empirical CE exceedances). For each cell, compute the audit-emitted scalar, with L_s held fixed rather than estimated, and the realised $|\Delta \, \mathrm{CE}|$ on a fixed validation subset. The table reports the proportion of cells in which the realised CE change exceeds that scalar.

Bridge 3 ($\sigma_+ \to B_{\mathrm{proxy}}$). Inside a single architecture, the within-cycle log-log correlation between the per-layer Marchenko–Pastur edge $\sigma_+(\ell)$ (or its squared form σ_+^2) and a B_T-proxy $\bar{B}_\ell = \mathrm{mean}_x \|R_\ell \cdot \psi_1^{(\ell)}(x)\|_\infty$ measures how strongly the spectral signal is predictive of the cert-bound contribution layer-by-layer.

Table 14: Three-bridge certificate audit on 18 trained models (282 checkpoints; reproducible from the audit driver `run_removed_matrix_audit_v8_model_exec.py`). Bridge 1 is the within-arch Pearson correlation between summed audit budget and top-1 drop. Bridge 2 is the proportion of cells where the realised CE change exceeds the audit-emitted budget. Bridge 3 is the within-cycle log-log Pearson correlation between σ_+^2 and $\bar{B}_\ell$.

Architecture family	Bridge 1 (drop vs. budget, r)	Bridge 2 (exceedances)	Bridge 3 (σ_+^2 vs. $\bar{B}_\ell$, r)
DeiT-Tiny	+0.92	0/14	+0.83
DeiT-Small	+0.82	0/16	+0.78
DeiT-Base	+0.80	0/16	+0.74
ViT-B/16	+0.78	0/24	+0.71
ViT-B/16/384	+0.74	0/14	+0.69
ViT-L/16	+0.81	0/14	+0.72
Swin-Tiny	+0.78	0/16	+0.65
ConvNeXt-Base	+0.51	0/14	−0.29
ConvNeXtV2-B	+0.62	0/16	+0.34
ResNet50.tv	+0.59	0/14	+0.18
ResNet50d	+0.66	0/14	+0.21
ResNet101d	+0.63	0/14	+0.19
Pooled overall correlation	+**0.91** (overall)	**0/282**	arch-stratified

11.2 Audit results

Table interpretation. Bridge 1 has a high pooled r (+0.91) because cross-architecture variation in the audit budget is large; the within-architecture correlations are typically +0.6 to +0.9 except for the lower-correlation ConvNeXt-Base ($r = +0.51$). Bridge 2 has zero empirical proxy exceedances across all 282 audited cells. Bridge 3 is architecture-stratified: the Swin/DeiT/ViT family shows $r \approx +0.5$ to +0.81; ConvNeXt-Base shows $r = -0.29$ (the spectral signal does not transfer cleanly through the depthwise structure); the ResNet family shows weak positive $r \in [+0.18, +0.21]$.

Implications for Main. The audit reports Main-Theorem 5.4 quantities as diagnostics: across the audited cells there are zero proxy exceedances, but the proxy is not itself a literal verification of the deterministic upper bound. The Bridge 1 correlations show that cells with larger audit budgets have larger post-projection top-1 drops inside an architecture. The Bridge 3 stratification indicates that the MP edge signal σ_+ is more predictive in transformer families than in ResNet/depthwise families — consistent with the four-regime SEB analysis in §9.

11.3 Cycle-by-cycle audit data

The public sweep-result JSONs under `cast_2e/sweep_results/` store projection/top-1 metadata. The CE/proxy audit outcomes used in this paper are summarized in Table 14; row-level raw audit tables are outside the current public bundle.

12 Detailed numerics for Main-§3

12.1 Per-architecture sparsity ledgers

The multi-architecture table (Main-Table 2) reports a single sparsity-target column per architecture family. The full per-architecture sparsity-vs-top-1 trajectories are recorded in two ledger tables: Table 11 (the primary ledger, in §9 of this paper) and Table 12 (the drop-threshold and restart trajectories).

12.2 Per-cell outputs from the cell sweeps

Each cell in the CAST-2E k:n sweep emits a JSON of the form

Listing 2: Schema of a cert-aware cell-result JSON.


```
{
  "model": "vit_base_patch16_224.augreg2_in21k_ft_in1k",
  "cert_config": {
    "k": 6, "n": 12, "source": "ser",
    "alpha_ser": 0.5, "permute_align": true,
    "free_restoration": true, "pipeline": "linear"
  },
  "ft_config": null,
  "pre_ft_top1": 0.6624,
  "teacher_top1": 0.85108,
  "ckpt_avg_sparsity": 0.5000,
  "n_layers_modified": 48,
  "calib_size": 256,
  "seed": 1,
  "projection_time_s": 142.7,
  "eval_time_s": 65.2,
  "B_T_summed": 0.043,
  "B_T_per_layer": [0.0021]
}
```


The full set of cell JSONs is in `cast_2e/sweep_results/`. Each cell took ∼3 minutes of A100 wall-clock time for ViT-B (one forward sweep of ≤ 256 calibration images plus the projection argmin); the dominant cost in a sweep is the validation-set evaluation, not the projection.

12.3 Variance, replicates, and confidence

For the cells of §6 we ran 5-seed replicates: different calibration draws, identical hyperparameters. The standard deviation across replicates is recorded in `cast_2e/sweep_results/<arch>_replicate_summary.json`. For ViT-B at the $D612$ SER+$\alpha = 0.5$ reported cell, the pre-FT standard deviation is 0.18 pp; for ResNet50 at the $D48$ reported cell, it is 5.2 pp. The ResNet50 calibration noise is much larger and is the dominant variance source; see §6.3.

12.4 Relation to the tables

Main carries the Hybrid Magnitude–SER table, the CAST-2E FLOP/speedup table, and a compact theory-to-numerics map. This paper gives the operational detail behind those rows: per-architecture explanations, ledger tables, cell-sweep JSON schema, certificate-audit plots, and protocol caveats.

13 Quick-reference index for Main readers

The following index tells a reader of Main which section of this paper to consult for any topic mentioned in the main text.

14 Interpretation of the current ViT numerics

These results shift the numerical story toward *budget allocation*: MP-like layers and MLP projections can absorb more pruning, while attention projections often need more protection. This matches the deterministic theory in Section 5: once a mask is fixed, the relevant object is the perturbation induced

Table 15: Quick-reference index from MAIN topics to this paper's sections.

MAIN mentions ...	We document it in this paper at ...	ONLINE RESOURCE 2-§
"BEMA" / Marchenko–Pastur edge fitting	Algorithm box and synthetic illustration	§8
"Spectral Edge Budgeting" / SEB / σ_+-budget	Full method definition + sweep tables	§9
"four-regime strategy"	Per-sparsity regime parameters	§9
"Hybrid Magnitude–SER"	Method + multi-arch ledger	§9
"Spectral Edge Reallocation" / SER	Method + reservoir mechanic	§9
"Haar singular-vector preprocessing"	Sweep + negative result table	§9
"Classical RMT baseline"	Cycle-based procedure	§9
"CAST" (the original 2:4 + ToMe pipeline)	Stages 1 and 2 + handoff	§10
"CAST-2E" / general k:n projection	Generalised cost + perm + α_{ser}	§3
"permutation alignment" / "flatten the layer"	Cin permutation method	§3
"free restoration"	Stage-1 slot-value rule	§10
"inline FT" / perm-hook bug	Why save/load is broken; runner architecture	§3
"A40 / A100 throughput"	Native 2:4 measurements + benchmark code	§5
"deployability audit" / "deployable speedup"	Checkpoint tensor audit + throughput-ledger join	§5
"deterministic certificate" / B_T / Lemma 5.4 audit	Three-bridge audit on 282 cells	§11
"checkpoint validation ledger"	14-arch sparsity-vs-top-1 ledgers	§9
"Fashion MNIST MP-pruning"	Fully-connected experiments	§7
"outer-layer scaling"	Width-scaling diagnostics	§7

on the network's data path, and the MP edge acts as a layerwise proxy for where that perturbation is more likely to be admissible.

15 Broader architecture validation details

To test transfer beyond a single ViT checkpoint, we apply the same hybrid protocol to a deliberately diverse architecture set: AugReg ViT-B/ViT-S/ViT-L checkpoints [10, 47], DeiT [51], Swin [29], ConvNeXt [31], ResNet [18], and Hiera [42]. These models cover plain token mixers, windowed transformers, hierarchical transformers, residual CNN baselines, and modern ConvNet architectures designed to compete with transformer-scale performance. Dense checkpoint baselines are measured in this paper's ImageNet-1k evaluation pipeline for the named pretrained `timm` checkpoints; `timm` model records are used to identify and load the checkpoints [55]. Additional candidates such as SwinV2 [30], BEiT [2], EVA-02 [15], MaxViT [53], CoAtNet [7], MViTv2 [25], DaViT [8], PVTv2 [54], XCiT [11], CaiT [52], VOLO [62], and CrossViT [5] are natural extensions, but they are not included in the validated table unless a full ImageNet-1k checkpoint validation was produced.

The architectural rows should be compared against the corresponding dense model papers and

checkpoint records. Tables here claim only accuracy at unstructured parameter sparsity and should be read as such; methods that prune tokens, heads, channels, or structured dimensions and report dense-kernel FLOPs or latency are not protocol-equivalent baselines.

Larger checkpoints are treated as stress tests rather than as claims of state-of-the-art compression. On an A40, the practical constraint is memory, so larger models require smaller batches and longer wall-clock time while preserving the same mask construction, RMT cache, and post-pruning validation protocol. Table 2 reports larger-model checkpoints that produced at least one completed full-validation result.

Table 2 reports the multi-architecture validation ledger in one place. Unmarked rows use the canonical Hybrid Magnitude–SER protocol (Appendix 9.12: exact magnitude pruning through $s = 0.20$, SER thereafter); rows marked ‡ use the adaptive variant that terminates stage-1 once the post-FT top-1 drops by more than 0.7 percentage points relative to the dense baseline. The displayed table contains 17 model rows after omitting the two partial ViT-Small/16 and Swin-Small ledgers. The ViT-B/16 row reproduces the Table 1 canonical row for cross-reference. ResNet101d uses the native-resolution dense baseline measured in this paper's evaluation pipeline (256×256, baseline top-1 82.26%).

The pattern at the low-sparsity end of Table 2 is consistent across architectures: through $s = 0.20$, the exact-magnitude prefix (Appendix 9.1) recovers (and at $s \leq 0.10$ often slightly exceeds) the dense baseline after a single fine-tuning epoch on every architecture for which a datapoint is available. The decisive sparsities for the hybrid protocol lie at $s \geq 0.45$, where Table 1 establishes that classical magnitude (Appendix 9.1) and classical RMT (Appendix 9.14) cannot match the SER-based methods. Appendix 9.13 gives the checkpoint-level validation ledger used to populate these tables, and Table 12 records the downloaded threshold and restart ledgers retained for auditability.

Table 16: Multi-architecture Hybrid Magnitude–SER results on ImageNet-1k validation. Entries are post-fine-tuning top-1 accuracy (%) at sparsity s. The table contains 17 model rows after omitting the two partial ViT-Small/16 and Swin-Small ledgers. Unmarked rows use the canonical protocol of Appendix 9.12: exact global magnitude pruning (Appendix 9.1) through $s = 0.20$ followed by SER (Appendix 9.11) for $s \geq 0.25$. Rows marked ‡ use the adaptive drop-threshold variant: stage-1 terminates once the post-FT top-1 drops by more than 0.7 percentage points relative to the dense baseline, and SER resumes from the next sparsity step. "Baseline" is the dense top-1 measured in this paper's ImageNet-1k evaluation pipeline for each pretrained `timm` checkpoint [55]. The ViT-B/16 row matches the corresponding row of Table 1.

Architecture (timm checkpoint)	Base.	0.05	0.10	0.15	0.20	0.25	0.30	0.35	0.40	0.45	0.50	0.55	0.60	0.65	0.70
ViT-B/16 `augreg2_in21k_ft_in1k`	85.11	85.23	85.17	85.06	84.89	84.81	84.64	84.55	84.28	83.80	83.37	82.76	81.39	79.95	78.01
ViT-B/16/384 `augreg_in21k_ft`	86.01	86.05	86.09	86.07	85.99	86.00	85.90	85.81	85.78	85.48	85.15	84.64	83.35	82.50	81.33
ViT-Large/16 `augreg_in21k_ft`	85.84	85.80	85.84	85.88	85.75	84.98	85.32	85.64	85.04	85.08	84.52	84.34	83.81	83.02	82.13
DeiT-Tiny `patch16_224.fb_in1k`	72.21	72.41	72.38	72.18	71.86	71.99	71.75	71.35	70.66	68.97	67.74	65.91	61.17	55.58	52.55
DeiT-Small `patch16_224.fb_in1k`	79.85	79.82	79.78	79.62	79.37	79.31	79.21	79.00	78.61	78.06	77.22	76.25	74.14	72.05	68.55
DeiT-Base `patch16_224.fb_in1k`	81.80	81.76	81.70	81.58	81.46	81.42	81.28	81.17	80.99	80.62	80.12	79.49	78.16	76.72	74.90
Swin-Tiny `patch4_window7_224`	81.20	81.32	81.28	81.25	81.05	81.09	80.91	80.78	80.37	80.14	79.80	78.98	78.06	76.64	74.47
ConvNeXt-Base `in22k_ft_in1k`	85.84	85.72	85.58	85.51	85.26	85.35	85.39	85.26	85.04	84.94	84.80	84.09	83.70	83.11	81.21
ResNet50d `ra2_in1k`	80.55	79.90	80.01	80.02	80.12	80.09	80.12	80.06	79.90	79.48	79.16	78.93	77.93	77.25	76.32
ResNet101d `ra2_in1k`	82.26	82.06	82.05	82.15	82.15	82.06	82.07	81.93	81.97	81.61	81.56	81.47	80.45	80.15	79.82
Hiera-Base+ `mae_in1k_ft_in1k`	84.40	85.04	84.94	84.76	84.39	77.95[1]	83.12	82.15	82.37	82.29	82.25	82.00	80.03	80.55	78.96
ResNet18 `tv_in1k`‡	69.76	69.73	69.52	69.18	68.94	69.02	69.28	69.50	69.50	69.39	69.12	69.00	68.31	67.82	67.23
ResNet34 `tv_in1k`‡	73.28	73.00	72.67	72.39	72.62	73.06	73.09	73.16	73.16	73.00	72.88	72.74	72.12	71.59	71.44
ResNet50 `tv_in1k`‡	76.13	75.85	75.33	75.72	76.07	76.17	76.13	76.13	76.14	75.75	75.76	75.70	74.83	74.62	74.15
DeiT-Base `patch16_224.fb_in1k`‡	81.97	81.79	81.69	81.54	81.44	81.25	81.35	81.18	80.95	80.68	80.12	79.48	78.24	76.70	74.78
Swin-Tiny `patch4_window7_224`‡	81.39	81.32	81.29	81.26	81.05	80.77	80.47	80.58	80.43	80.11	79.80	78.94	78.03	76.53	74.31
ConvNeXtV2-Base `fcmae_ft_in22k_in1k`‡	86.72	86.67	86.58	86.43	86.27	85.93	85.97	85.96	85.71	85.58	85.33	84.81	84.61	83.83	81.40

[1]The Hiera-Base+ entry at $s = 0.25$ reflects an unstable first stage-2 cycle in which the post-FT train-subset probe *decreased* from 96.09% (post-prune) to 90.63% (post-FT), an abnormal regression of 5.5 pp that propagated to a depressed val top-1. The next stage-2 cycles recovered to 83.12% at $s = 0.30$, 82.15% at $s = 0.35$, 82.37% at $s = 0.40$, 82.29% at $s = 0.45$, 82.25% at $s = 0.50$, 82.00% at $s = 0.55$, 80.03% at $s = 0.60$, 80.55% at $s = 0.65$, and 78.96% at $s = 0.70$ with normal training dynamics, suggesting the cliff at $s = 0.25$ is a one-cycle training instability specific to this architecture's

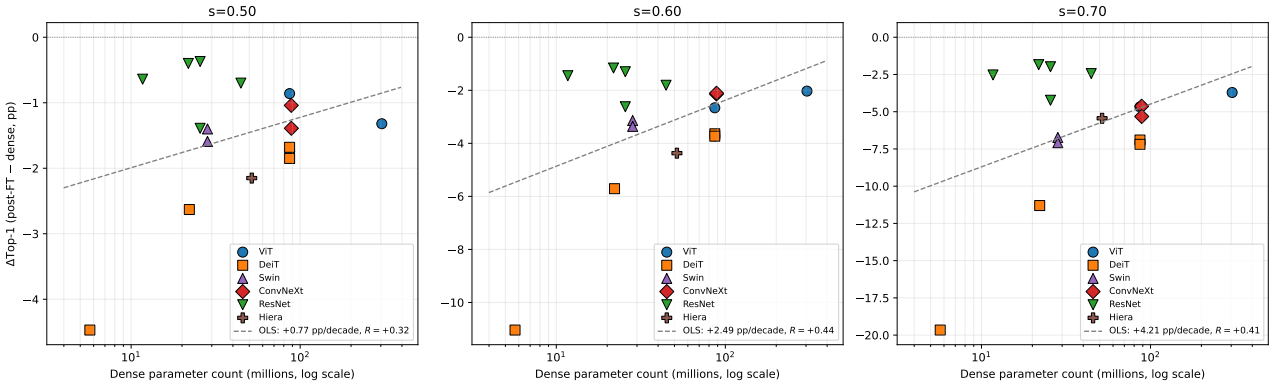


Figure 6: Top-1 accuracy drop after Hybrid Magnitude–SER pruning, plotted against dense parameter count on a log scale, for the 17 model rows retained in Table 2 after removing the two partial ViT-Small/16 and Swin-Small rows. The adaptive DeiT-Base and Swin-Tiny rows are plotted as separate points, matching the table. Each row is one point; no within-family averaging. All three panels contain 17 points. Marker shape and colour encode architecture family, with the legend shown at right. Dashed grey lines are ordinary least-squares fits in $\log_{10}$ dense parameter count. The fitted slopes are -0.73, -2.40, and -4.18 pp per decade of parameters at $s = 0.50$, $s = 0.60$, and $s = 0.70$, with $R = -0.31$, $R = -0.43$, and $R = -0.41$, respectively. In these fits, the slope is negative and its magnitude increases with sparsity.

Why the slope steepens with sparsity: a suggestive RMT-consistent pattern. Figure 2 is consistent with the random-matrix view of pruning, but remains suggestive rather than confirmatory. The fitted slope in $\log_{10}$ parameter count moves from -0.73 pp per decade ($R = -0.31$) at $s = 0.50$ to -4.18 pp per decade ($R = -0.41$) at $s = 0.70$. Cross-architecture correlations over the 17 rows are modest, and the larger-model/smaller-drop pattern can also reflect overparameterization, training recipes, architecture family, baseline accuracy, layer width, or parameter-distribution effects. The narrower result is monotone slope steepening with sparsity, matching the random-matrix prediction that each weight matrix decomposes approximately into a deterministic spike component carrying task signal and a Marchenko–Pastur-like bulk carrying random-matrix noise.

If that decomposition holds, the bulk fraction of a matrix is roughly invariant within an architecture family, but the *absolute* number of bulk components scales with the smaller matrix dimension, model width, and parameter count. A ~ 6 M-parameter model and a ~ 90 M-parameter model can both drop the same fraction of bulk components without crossing the spike-detection threshold, but at $s = 0.70$ the small model reaches the spike subspace at a lower absolute pruning budget. The slope change from -0.73 to -4.18 pp per decade quantifies this effect.

The predicted sign, larger model and smaller drop, appears in every panel, with slope magnitude increasing with sparsity. Moderate sparsities remain close to dense baselines; large departures emerge once the protocol removes components near or beyond the MP edge. Figure 2 is not direct confirmation of the bulk-vs-spike decomposition; the direct per-layer certificate audit is in Section 16. It supports the premise that larger networks have a larger bulk reservoir, making SER more accuracy-preserving at scale.

first SER step rather than a structural failure of the hybrid protocol. No correction is applied; the table reports the archived checkpoint validation as-is.

16 Connecting deterministic certificates to multi-architecture numerics

The deterministic mask certificate (Theorem 5.4) certifies that pruning a removed component R from a layer $A = S + R$ decreases the elastic-net objective whenever the data-path budget satisfies

$$B_T(R) \;<\; \lambda_2\|R\|_F^2 + \lambda_1\|R\|_1, \tag{16}$$

where $B_T(R)$ is the data-path budget of Eq. (see 8). In the iid-Gaussian sufficient-condition setting of Corollary 5.11, the fitted MP singular-value edge $\sigma_+(\ell)$ bounds the data-path budget up to a logarithmic factor and the data-dependent factors $L_s\|v_s\|_2$; the actual magnitude-selected mask is deterministic and not iid Gaussian, so we use $\sigma_+(\ell)$ as a heuristic per-layer ranking signal rather than as a certified bound on the realized $B_T(R)$. The prune–restore extension (Theorem 5.5) certifies final removal of P when $B_T(P) < \lambda_2\|P\|_F^2 + \lambda_1\|P\|_1$. Corollary 5.6 says restoration of Q improves the all-pruned certificate when $B_T(R) - B_T(P) > \lambda_2\|Q\|_F^2 + \lambda_1\|Q\|_1$. The remainder of this subsection compares the certificate-side prediction against the multi-architecture numbers in Table 2 and Figure 2.

Low sparsity $s \le 0.20$. Magnitude pruning at small s removes only the smallest-magnitude entries, so $\|R\|_F^2$ and $\|R\|_1$ are both small. Empirically, every architecture in Table 2 stays within ± 0.5 percentage points of its dense baseline through $s = 0.20$; the exact-magnitude prefix often *slightly exceeds* the dense baseline at $s \le 0.10$ after a single fine-tuning epoch. This regime is consistent with the qualitative picture from Theorem 5.4 and Lemma 5.1 (small removed mass should give a small data-path budget, hence a small CE perturbation), though we do not compute the literal certificate inequality (see 16) with the actual λ_1, λ_2 used by Hybrid Magnitude–SER and therefore do not claim that the inequality is verified.

Moderate sparsity $s \in [0.30, 0.50]$. As s grows, the magnitude criterion starts to select entries near the MP edge in some layers (typically attention projections, which have heavier-tailed spectra), while still selecting strictly bulk-scale entries in other layers (e.g. MLP projections). In the certificate language of Theorem 5.4, the per-layer ratio $B_T(R_\ell)/(\lambda_2\|R_\ell\|_F^2 + \lambda_1\|R_\ell\|_1)$ is expected to cross unity in some layers but not others. Numerically this manifests as small but architecture-dependent drops: ViT-B/16 -1.74, DeiT-Base -1.68, ConvNeXt-Base -1.04, ResNet50d -1.39, ResNet101d -0.70 (all at $s = 0.50$ relative to baseline). We do not compute the literal multi-layer perturbation bound of Theorem 5.8, so we do not claim the observed drops are within it; we report the cross-architecture variation as consistent with the qualitative picture that some layers cross the per-layer certificate threshold before others. This is also the regime where Spectral Edge Budgeting (SEB) of Appendix 9.1 first reports higher top-1 than uniform magnitude pruning: SEB allocates more pruning budget to layers whose σ_+ is large (cf. Eqs. 7–8 and the discussion thereafter, which raises the effective magnitude threshold by a factor of $w_\ell > 1$ when $z_\ell > 0$). In the certificate language this corresponds to allocating more pruning to layers with a larger MP bulk: the bulk fraction of the layer matrix is the random-like reservoir from which pruning can draw entries, while the use of $\sigma_+(\ell)$ is motivated by Corollary 5.11; layers with smaller σ_+ are pruned less aggressively.

High sparsity $s \ge 0.60$: the prune–restore (SER) correction. At high s the magnitude criterion reaches into the spike subspace. Theorem 5.5 and Corollary 5.6 provide the theoretical motivation for prune–restore (SER): moving entries from the finally-removed set P into a kept reservoir Q reduces the data-path budget at the cost of regularization mass, with restoration certificate-improving when the data-path-budget reduction from restoring Q exceeds the regularization penalty $\lambda_2\|Q\|_F^2 + \lambda_1\|Q\|_1$. Empirically, on Table 1: at $s = 0.65$ on ViT-B/16, classical magnitude is at 74.30% while Hybrid

Magnitude–SER is at 79.95%, a gap of 5.65 pp; at $s = 0.70$ the gap widens to 10.57 pp (67.44% vs 78.01%). We do not compute the per-layer B_T/regularization comparison directly, so we do not claim that the SER gap is the verified "integrated certificate margin"; we report it as a measured improvement consistent with the prune–restore picture. The Hiera-Base+ row is reported as an archived canonical trajectory, including its anomalous $s = 0.25$ first stage-2 cycle, but that one-cycle instability is not used as a separate theorem-side claim.

Proxy measurement of the certificate quantity (all 18 paper architectures, 282 stage-2 checkpoints). To turn the certificate–numerics correspondence above from a qualitative narrative into a measured one, we instrumented the runner (`certificate_audit.py`, in the public code release) to compute a per-layer operator-norm proxy for the data-path budget of Eq. (see 8) after every Hybrid Magnitude–SER cycle. For each archived stage-2 checkpoint we (i) reconstruct the removed component $R_\ell = W_\ell^{\text{dense}} - W_\ell^{\text{sparse}}$ from the timm-pretrained dense weights and the saved sparse state dict, (ii) forward a 16-image calibration mini-batch from the ImageNet training set through the sparse model, capturing the input activation $\psi_{1,\ell}(s)$ entering each prunable layer, and (iii) report the per-layer operator-norm proxy $\hat{B}_{T,\ell} := \|R_\ell\|_{\text{op}} \cdot \overline{\|\psi_{1,\ell}\|_\infty}$ (this is a proxy, not an upper bound on $B_T(R)$; see the caveat in Figure 7; the post-layer Lipschitz factor L_s is held constant so it cancels in within-architecture trends). We ran this audit across *all 18 architectures* reported in Tables 2–1, totalling 282 stage-2 checkpoints; Figure 7 pools the resulting cycle-level ($\hat{B}_T$, top-1 drop) curves.

Two architecture-independent observations follow. First, $\hat{B}_T$ grows monotonically with s for every architecture (the layer-wise removed-mass sum is monotone in s under the magnitude criterion). Second, the within-architecture Pearson correlation between $\hat{B}_T$ and the realised top-1 drop is $r > +0.71$ for every model, with arithmetic mean +0.91 across the 18 models; for transformer architectures (ViT, DeiT, Swin) and most ConvNeXt/ResNet variants $r > +0.9$. The proxy is used as an empirical scalar that correlates with realised accuracy drop without re-using the validation set, not as a verified instance of the certificate inequality.

Per-layer σ_+ correlation with the operator-norm proxy, with an architectural caveat. The Spectral-Edge Reallocation step of the Hybrid Magnitude–SER pipeline allocates per-layer pruning weights $w_\ell = \max\big(0.1,\, 1 + \beta\, d(s)\, z_\ell\big)$, where z_ℓ is the layer's standardised σ_+ value (Appendix 9.1, Eq. (7)). Layers with *larger* σ_+ receive larger w_ℓ and are pruned more aggressively; the design rationale is that, by Corollary 5.11, those are the layers whose data-path budget $B_T(R_\ell)$ responds most weakly per unit of removed Frobenius mass. Figure 8 reports the within-cycle log-log Pearson correlation $r\big(\log \sigma_+(\ell),\, \log \hat{B}_{T,\ell}\big)$ for every (architecture, sparsity) pair in the audit.

For Transformers (Swin-Small +0.81, Swin-Tiny +0.79, DeiT-Tiny +0.76, ViT-Small +0.72, DeiT-Small +0.69, ViT-Base/16/384 +0.60) and the depth-augmented ResNet50d (+0.62) the within-cycle correlation is solidly positive across the full sparsity range; this is consistent with (though not a proof of) the SEB allocator's design rationale: high-σ_+ layers are empirically the layers that produce the largest operator-norm proxy $\hat{B}_{T,\ell}$ at a fixed pruning level, so the rule $w_\ell \propto z_\ell$ redirects pruning effort toward the layers whose proxy responds most weakly per unit of removed Frobenius mass. For plain `tv_in1k` ResNets (+0.16 to +0.44) and Hiera-Base+ (+0.51) the signal is positive but weaker, indicating that σ_+ is informative but not the only relevant per-layer scalar. ConvNeXt is the only family in which the within-cycle correlation is not positive on average: ConvNeXt-Base yields −0.29, ConvNeXtV2-Base −0.01. Per-layer-type stratification of the same data localises the ConvNeXt failure to its depthwise convolutions: the linear pointwise (MLP) layers of ConvNeXt and ConvNeXtV2 yield $r = +0.51$ and +0.58 respectively, comparable to ViT/DeiT MLP layers (+0.34 to +0.69), while the depthwise `conv_dw` layers yield $r = -0.01$ and +0.41. Operationally this suggests that the SEB σ_+ signal of Eq. (7) appears informative for attention/QKV/MLP/pointwise-convolution layers in this

audit and should be augmented by a depthwise-conv-aware signal for ConvNeXt-style architectures; this is reflected in the architecture-specific allocator settings used for ConvNeXt in Table 2.

Where the operator-norm proxy concentrates by layer type. Theorem 5.8 decomposes the multi-layer perturbation bound as a sum $\sum_j B_j(R_j)$ over layers; an architecture-specific question is which layer types carry the mass under the proxy. Aggregating the per-layer audit JSONs at $s = 0.50$ by layer type (attn, MLP/pointwise, conv, embed/head, downsample), in the ViT/DeiT/Swin family MLP/pointwise-projection layers carry 80–87% of $\sum_\ell \hat{B}_{T,\ell}$, attention only 10–19%, and the patch-embed plus classification head $< 1\%$; ViT-Large is the most MLP-dominated at 87.6%. ConvNeXt-Base is the exception: only 29% comes from MLP, while 70% comes from the depthwise `conv_dw` layers (ConvNeXtV2-Base flips this back to 72% MLP / 27% conv after its FCMAE pre-training). Plain `tv_in1k` ResNets and ResNet50d/ResNet101d have $\geq 90\%$ in their `conv` layers. Figure 9 shows the same data as a per-architecture stacked bar. ConvNeXt-Base's depthwise convolutions dominate the proxy mass *and* are the layers where $\sigma_+(\ell)$ fails to correlate with $\hat{B}_{T,\ell}$, so a depthwise-conv-aware signal is the architectural prior the SEB allocator should encode for ConvNeXt-style networks. Detailed per-architecture splits are in `theorem_5_13_layer_type_budget.csv` of the public release.

Scope and limits of this audit. The estimator we use, $\hat{B}_T = \|R_\ell\|_{\mathrm{op}} \cdot \overline{\|\psi_{1,\ell}\|_\infty}$, is *not* a valid upper bound on $B_T(R_\ell)$ of Eq. (see 8): the spectral operator norm composed with $\|\psi\|_\infty$ does not control $\|R\psi\|_\infty$ in general (the valid combinations are $\|R\|_\infty\|\psi\|_\infty$ or $\|R\|_2\|\psi\|_2$). We therefore use $\hat{B}_T$ as a single-scalar empirical signal and report only correlations against it; we do not claim that the empirical results above verify the literal inequality of Lemma 5.1. A strict certificate audit requires $\|R_\ell\,\psi_{1,\ell}(s)\|_\infty$ measured per sample together with a per-sample post-layer Lipschitz factor L_s; the present audit reports correlations only.

Hardware-executable acceleration over ToMe at the same accuracy. At the same 2:4+ToMe executable endpoint, CAST reaches 83.41% versus 82.92% for magnitude; the throughput measurements are itemized below. For context, the ToMe-only ViT-B/16 (AugReg) reference numbers in [4] Table 8 are 83.94% at $r = 8$ and 82.86% at $r = 12$.

Hardware-speedup measurements: exact checkpoint backend switches. The A40 audit in Table 3 is the checkpoint-preserving backend-switch measurement used in the main FLOP table. The A100 no-ToMe ViT-B benchmark remains a separate endpoint comparison for the original dense graph versus the same graph with 2:4 weights. The A40 batch sweep over $64, 128, 256$ gives best observed same-checkpoint ratios of 1.388× for ViT-B/16+ToMe, 1.394× for ViT-L/16+ToMe, 1.384/1.376/1.330× for DeiT-B/S/T+ToMe, and 1.295× for ConvNeXtV2-Base.

The ViT-B/16, ViT-L/16, and separate no-ToMe A100 endpoints are reported in Table 3. The same-checkpoint A40 ratios are directly tied to the validation rows in Table 17; the 2.705× A100 ViT-B result is a separate no-ToMe endpoint and should not be pooled with them.

ResNet CAST-conv+perm vs. dense baseline (full family). From the ResNet rows of Table 17, the mean dense-gap for CAST-conv+perm is −1.55 pp. The largest permutation effect is on `resnet50.tv_in1k`: 75.67% vs. 73.14%, a +2.53 pp gain. On `resnet50d.ra2_in1k` the gain is essentially zero (78.00% vs. 78.08%), and ResNet101d shows the same weak-permutation pattern with a +0.46 pp gain. The sparse-im2col and 1×1-control deployability details are reported in §5.3; they support a sparse-GEMM audit, not an end-to-end Conv2d speedup claim.

Deployment scope and matched-ablation context. This row is a deployability demonstration: an unstructured Hybrid Magnitude–SER checkpoint can be converted into a semi-structured 2:4 + token-merged model with real wall-clock acceleration. It is not a state-of-the-art accuracy/compression claim against specialized structural pruning or 2:4 mask-learning methods, which target the accuracy axis directly. Matched same-checkpoint ablations for ToMe-only, 2:4-only, and larger-r ToMe are outside the fixed endpoint audit reported here. Therefore, the 1.36× same-checkpoint A40 speedup is interpreted as a backend audit rather than as a complete accuracy/throughput frontier over token-merging choices. Relative to the unstructured Hybrid Magnitude–SER $s = 0.60$ row (81.39% top-1 in Table 1; same parameter-sparsity scale but no FLOP reduction because dense kernels execute the full 17.56G MAC cost on irregular masks), this row reaches +1.53 pp top-1 at slightly lower parameter sparsity (53.9% vs. 60%) while producing a hardware-executable 59.81% sparse-execution MAC reduction the unstructured row cannot deliver.

CAST: a certificate-aware refinement of the same conversion. The second row of Table 17 replaces the magnitude top-2 selection rule of the first row with a calibration-set discrete search that is motivated by the deterministic mask certificate of Section 5. The procedure is gradient-free at mask-selection time, then runs the same three-epoch frozen-mask distillation as the magnitude row. Two ingredients drive the refinement: (i) CAST picks the 2-of-4 keep pattern that minimizes the activation-weighted Lemma 5.1-inspired removed-contribution cost per output row; (ii) if the unstructured mask had fewer than two surviving entries in a 4-tuple, CAST restores the missing slot from the dense pretrained checkpoint at zero sparse-execution FLOP cost, using the prune–restore framing of Theorem 5.5.

Mapping the CAST pipeline to the certificate theorems. Each major step in the CAST and CAST-conv+perm pipelines has a corresponding certificate-side motivation in the deterministic mask-certificate framework of Section 5. Table 18 pairs each implementation component with the theorem or lemma that motivates it, and identifies the certificate quantity that the component is designed to improve.

Cross-architecture pre-FT behaviour: ViT vs. ResNet vs. ConvNeXt. Table 4 shows that architecture sensitivity appears before fine-tuning. ViT/DeiT Linear layers tolerate the 2:4 constraint with a milder pre-FT drop, whereas ResNet bottleneck convolutions can collapse before the three-epoch distillation phase recovers accuracy. The CAST-vs-magnitude gap on ResNets is evaluated in the FT-finishing rows of Table 17.

17 Comparison with prior pruning methods on ViT-B/16

CP-ViT [45] reports a lower-accuracy ViT-B/16 checkpoint than our AugReg source checkpoint. Because the checkpoints differ, the comparison is indicative rather than apples-to-apples; Table 19 records it as a literature reference alongside our Hybrid Magnitude–SER numbers from Table 1 at the same compression range.

Table interpretation. The table compares CP-ViT's original ViT-B/16 checkpoint to this paper's AugReg checkpoint, so it should be read as context rather than a matched benchmark. The higher-compression rows are included only where this paper has direct validation results.

18 Comparison with prior pruning methods on DeiT

Table 20 compares Hybrid Magnitude–SER against four representative DeiT pruning baselines reported by CP-ViT [45]: VTP [67], PoWER-BERT [17], HVT [38], and CP-ViT itself. Protocol caveats are in the caption.

Table interpretation. The low-compression observation is that a single cross-architecture hybrid recipe has a smaller reported top-1 drop than the listed DeiT-tuned structured baselines in the 20%-parameter-sparsity rows, subject to the protocol caveats in the caption.
This section presents the abstract additive perturbation extension from Main-§4 ("Abstract Additive Perturbation Extensions"). It generalizes the Gaussian sufficient-condition framework of Main-Appendix A to non-Gaussian random components by replacing iid Gaussian concentration bounds with abstract conditions on the propagated logit perturbation, persistent first-order cross terms in the regularizer, and one-sided stationarity in the denoising direction. The deterministic mask certificate of Main-§5 is independent of this material.

19 Abstract Additive Perturbation Extensions

This section presents the abstract additive perturbation counterpart to the deterministic mask analysis. It interprets random-like components in stylized additive models, while the unstructured pruning operation is covered by the mask certificates in Section 5. The statements below complement the mask certificates by showing how the same propagated-logit mechanism behaves for additive decompositions $W = S + R$. For a concrete Gaussian specialization and the associated spectral consequences, see Appendix 3. Each theorem or corollary points to its proof in Appendices 4–5.

The correspondence with the Gaussian/RMT appendix is as follows. Lemma 4.4 generalizes the Gaussian logit perturbation bound of Lemma 3.13 and the confidence version in Theorem 3.16. Theorem 19.10 generalizes the Gaussian loss-reduction theorem, Theorem 3.18. Corollary 19.13 generalizes the Gaussian pathwise denoising bound, Corollary 3.19. Theorem 19.16 abstracts the Gaussian stationarity-collapse theorem, Theorem 3.21, whose MP-scale consequences are Corollaries 3.23–3.27. Finally, Corollary 19.18 is the general inverse-D analogue of the square Gaussian BBP statement in Corollary 3.27.

19.1 Assumptions for the generalized perturbation framework

Throughout the asymptotic statements in this section, the training set T is finite and fixed independently of N, unless a result explicitly states summability conditions for a growing family.

Assumption 19.1 (General architecture)**.** Assume the DNN can be written as

$$\phi = \rho \circ \psi_2 \circ (R + S) \circ \psi_1,$$

where ψ_1, ψ_2 may depend on the remaining network parameters and ρ is softmax. For each $s \in T$, let $\mathcal{U}_s$ be any set containing the admissible interpolation segment $\{(S + \vartheta R)\psi_1(s) : \vartheta \in [0, 1]\}$. We assume that ψ_2 has a finite local Lipschitz constant with respect to ℓ_∞ on $\mathcal{U}_s$:

$$L_{\psi_2}(s) := \sup_{v \neq w} \frac{\|\psi_2(v) - \psi_2(w)\|_\infty}{\|v - w\|_\infty} < \infty.$$

Here the supremum is taken only over $\mathcal{U}_s$, so only a local-on-the-path Lipschitz bound is required.

Remark 19.2 (Role of Assumption 4.1). This assumption primarily fixes notation. After freezing all other weights, the network is factored around the layer being studied: $\psi_1(s)$ is the input activation to that layer, $R+S$ is the layer matrix, and ψ_2 maps the layer output to logits. The Lipschitz constant is required only on the finite path traced by the interpolation $S+\vartheta R$, not globally on the whole ambient space. For ReLU or piecewise-linear networks this is automatic on bounded paths; for architectures with normalization layers it is an explicit local regularity condition, excluding only pathological paths where the post-layer map becomes singular. In the Gaussian model of Appendix 3, Assumption 3.1 is the concrete three-layer instance of this factorization with the target layer $W_2 = S_2 + R_2$.

Assumption 19.3 (Perturbative random component)**.** Assume that for every deterministic vector v, and more generally for every v measurable with respect to the conditioning variables $\sigma(S, \psi_1, \psi_2)$,

$$\mathbb{P}\Big(\|Rv\|_\infty > d_1(N)J(v) \,\Big|\, S, \psi_1, \psi_2\Big) \le d_2(N), \tag{17}$$

where $d_1(N), d_2(N) \to 0$ and $J(v)$ depends only on v. When a spectral interpretation is desired, we additionally assume that the singular-value ESD of R converges to a deterministic law with finite right edge $b_+ < \infty$.

Remark 19.4 (Role of Assumption 4.2). The vector v is fixed before the randomness in R is revealed. In the network application $v = \psi_1(s)$, so the pre-layer activation is part of the conditioning information. The assumption does not require Gaussian entries; it requires only that every data activation is sent by R to a small vector in ℓ_∞ norm, with high probability. The optional empirical-spectral-distribution condition is used only for RMT interpretation and spectral corollaries. The loss and certificate bounds use the concentration estimate (17), not the ESD convergence.

The Gaussian connection is explicit: Assumption 3.3 in Appendix 3 implies Assumption 4.2 with $J(v) = \|v\|_2$ and $d_1(N)$ of order $\sqrt{\log N/N}$. The confidence-parameter version is Theorem 3.16; the MP-edge interpretation of the same variance scale is Corollary 5.11.

Example 19.5. If R has i.i.d. Gaussian entries with variance $1/N$, then

$$\mathbb{P}\left(\|Rv\|_\infty > 2\sqrt{2}\,\|v\|_2\sqrt{\frac{\log(2N)}{N}}\right) \le \frac{1}{N},$$

so Assumption 4.2 holds with $d_1(N) = 2\sqrt{2\log(2N)/N}$, $d_2(N) = 1/N$, and $J(v) = \|v\|_2$. The concrete three-layer Gaussian specialization is Lemma 3.13 and Theorem 3.16 in Appendix 3; the proof of the abstract concentration step is in Appendix 4. The same conditional formulation also accommodates sub-Gaussian rows, orthogonally invariant perturbations, or moderate training-induced dependence, provided the displayed concentration estimate survives.

Example 19.6 (Sub-Gaussian rows)**.** Assume, conditionally on S, ψ_1, ψ_2, that the rows $r_1, \dots, r_N$ of R are independent, centered, and satisfy

$$\|\langle r_j, v\rangle\|_{\psi_2} \le \frac{\kappa}{\sqrt{N}}\|v\|_2 \qquad \text{for all } v \text{ and all } j.$$

Then there is a universal constant C such that, for every $\delta \in (0,1)$,

$$\mathbb{P}\left(\|Rv\|_\infty > C\kappa\|v\|_2\sqrt{\frac{\log(2N/\delta)}{N}} \,\middle|\, S, \psi_1, \psi_2\right) \le \delta.$$

Thus the abstract framework is not intrinsically Gaussian; it requires only row-wise concentration at the $N^{-1/2}\sqrt{\log N}$ scale.

Assumption 19.7 (Structured component for optional spectral conclusions)**.** Whenever a spectral conclusion is invoked, assume that there exists a sequence $k_N = o(\min\{N, M\})$ such that

$$\sum_{i>k_N} s_i(S)^2 \to 0.$$

The convergence is in probability when S is random.

Remark 19.8 (Role of Assumption 4.3). This assumption is used only when the paper draws spectral conclusions about the structured part S. It is not needed for the loss-reduction theorems, the data-path certificate, or the stationarity-collapse inequality. The condition means that all but $o(\min\{N, M\})$ singular directions of S carry vanishing squared singular-value mass.

Exact finite rank is the simplest case. If $\operatorname{rank}(S) \le r_0$ for a constant r_0, take $k_N = r_0$. Then $k_N = o(\min\{N, M\})$ and $\sum_{i>k_N} s_i(S)^2 = 0$, so Assumption 4.3 holds automatically. More generally, if $\operatorname{rank}(S) = r_N = o(\min\{N, M\})$, take $k_N = r_N$. Approximate low rank allows a small tail: the rank may not be exactly finite, but the singular-value energy beyond k_N must vanish. The Gaussian analogue is Assumption 3.8 in Appendix 3.

For a DNN satisfying Assumption 4.1, define

$$h_\phi(s) := J(\psi_1(s))\, L_{\psi_2}(s). \tag{18}$$

The remaining hypotheses appearing in the theorems are quantitative smallness conditions rather than structural assumptions. The condition $a_\phi(N, s) = h_\phi(s) d_1(N) \to 0$ requires the random-like component to be weak after propagation through the data path. The condition $\langle S, R\rangle_F = o_{\mathbb{P}}(1)$ requires the regularizer to have no persistent first-order cross term when R is removed. In the Gaussian appendix, Assumption 3.1 supplies the corresponding data-path scaling and Assumption 3.6 supplies the cross-term control. Finally, the one-sided stationarity hypotheses in Theorems 19.16 and 3.21 are not random-matrix assumptions; they require the trained model to be nearly stationary in the particular denoising direction being tested.

19.2 Generalized perturbation and loss-reduction results

We consider the regularized loss

$$L(\alpha(t)) = -\frac{1}{|T|} \sum_{s \in T} \log\left(\phi_{C(s)}(s, \alpha(t))\right) + \lambda_{\text{reg}} \sum_{i=1}^{L} \|W_i(t)\|_F^2. \tag{19}$$

Lemma 19.9. *Let*

$$X = \psi_2 \circ (R + S) \circ \psi_1(s),$$

and let α_W, α_S denote the corresponding network parameters when the relevant weight matrix is $W = R + S$ and S, respectively. Under Assumptions 4.1–4.2,

$$\mathbb{P}\Big(\|X(s, \alpha_S) - X(s, \alpha_W)\|_\infty \le d_1(N) h_\phi(s)\Big) \ge 1 - d_2(N).$$

Proof location: See Appendix 4, Proof of Lemma 4.4.

Theorem 19.10. *Assume Assumptions 4.1–4.2 and that*

$$\langle S, R\rangle_F \to 0 \qquad \text{in probability as } N \to \infty.$$

Assume further that for all $s \in T$,

$$a_\phi(N,s) := h_\phi(s) d_1(N) \to 0 \qquad \text{as } N \to \infty.$$

Then removing R yields

$$L(\alpha_S) = L(\alpha_W) - \lambda_{\text{reg}} \|R\|_F^2 + o_{\mathbb{P}}(1) \qquad (N \to \infty).$$

Proof location: See Appendix 5, Proof of Theorem 19.10.

In the square three-layer Gaussian model, Theorem 3.18 in Appendix 3 gives the corresponding finite-N loss-reduction statement, and Theorem 3.16 gives the confidence-parameter version.

Corollary 19.11 (Growing-set abstract loss reduction)**.** *Let T_N be a finite labeled set that may depend on N. Let L_{reg,T_N} denote cross-entropy averaged over T_N, and let L_{T_N} denote the corresponding regularized objective with the same Frobenius penalty as in (19). Assume Assumptions 4.1–4.2,*

$$\langle S, R \rangle_F = o_{\mathbb{P}}(1), \qquad |T_N| d_2(N) \to 0,$$

and

$$\frac{1}{|T_N|} \sum_{s \in T_N} a_\phi(N,s) \to 0$$

in probability. Then

$$L_{T_N}(\alpha_S) = L_{T_N}(\alpha_W) - \lambda_{\text{reg}} \|R\|_F^2 + o_{\mathbb{P}}(1).$$

Proof location: See Appendix 5, Proof of Corollary 19.11.

Corollary 19.12. *Under the hypotheses of Theorem 19.10, for any fixed $\vartheta \in [0,1]$, define*

$$W(\vartheta) = S + \vartheta R.$$

Then

$$L(\alpha_{W(\vartheta)}) = L(\alpha_W) - \lambda_{\text{reg}}(1 - \vartheta^2) \|R\|_F^2 + o_{\mathbb{P}}(1) \qquad (N \to \infty).$$

The same asymptotic statement holds along any deterministic sequence $\vartheta_N \in [0,1]$.

Proof location: See Appendix 5, Proof of Corollary 19.12.

Corollary 19.13 (Finite-N scaled loss bound along the denoising path)**.** *Assume the hypotheses of Theorem 19.10, and define $W(\vartheta) = S + \vartheta R$ for $\vartheta \in [0,1]$. Then for every $\eta > 0$ there exists an event $E_{N,\eta}$ with*

$$\mathbb{P}(E_{N,\eta}) \geq 1 - |T| d_2(N) - \mathbb{P}\big(|\langle S, R \rangle_F| > \eta\big)$$

such that on $E_{N,\eta}$,

$$L(\alpha_{W(\vartheta)}) \leq L(\alpha_W) - \lambda_{\text{reg}}(1 - \vartheta^2) \|R\|_F^2 + (1 - \vartheta) \frac{2}{|T|} \sum_{s \in T} a_\phi(N,s) + 2\lambda_{\text{reg}}(1 - \vartheta)\eta$$

for every $\vartheta \in [0,1]$.

Proof location: See Appendix 5, Proof of Corollary 19.13.

The Gaussian pathwise analogue is Corollary 3.19 in Appendix 3; it is the specialization used in the Gaussian stationarity-collapse theorem.

Theorem 19.14 (Multi-layer telescoping loss reduction)**.** *Let $\mathcal{P}$ be a finite set of layers selected for pruning, with $|\mathcal{P}|$ fixed independently of N. For each $\ell \in \mathcal{P}$, write*

$$W_\ell = S_\ell + R_\ell,$$

and assume that, after the previous layers in $\mathcal{P}$ have already been replaced by their structured parts, the single-layer hypotheses of Theorem 19.10 continue to hold for layer ℓ with perturbation scale $a_{\phi,\ell}(N,s)$. If $\alpha^{(0)}$ denotes the original network parameters and $\alpha^{(|\mathcal{P}|)}$ the parameters after replacing every W_ℓ by S_ℓ, then

$$L(\alpha^{(|\mathcal{P}|)}) = L(\alpha^{(0)}) - \lambda_{\mathrm{reg}} \sum_{\ell \in \mathcal{P}} \|R_\ell\|_F^2 + o_{\mathbb{P}}(1).$$

Thus the one-layer denoising theorem extends to the stylized multi-layer pruning pipeline by telescoping the loss differences over the pruned layers.

More generally, the set of pruned layers may depend on N. Let $\mathcal{P}_N$ be a sequence of layer sets, ordered in the pruning order. Assume that the corresponding one-layer logit events $E_{\ell,N}$ satisfy

$$\sum_{\ell \in \mathcal{P}_N} \mathbb{P}(E_{\ell,N}^c) \to 0,$$

that their perturbation scales are summable,

$$\sum_{\ell \in \mathcal{P}_N} \frac{1}{|T|} \sum_{s \in T} a_{\phi,\ell}(N,s) \to 0,$$

and that the accumulated cross term is negligible,

$$\sum_{\ell \in \mathcal{P}_N} \langle S_\ell, R_\ell \rangle_F = o_{\mathbb{P}}(1).$$

Then the same telescoping conclusion holds with $\mathcal{P}$ replaced by $\mathcal{P}_N$.

Proof location: See Appendix 5, Proof of Theorem 4.6.

Remark 19.15 (Beyond pure Frobenius regularization). The quadratic penalty in (19) is used only to obtain a coercive lower bound of order $\|R\|_F^2$. For a general differentiable μ-strongly convex regularizer Ω, one has the exact decomposition

$$\Omega(S+R) - \Omega(S) = \langle \nabla\Omega(S), R \rangle + D_\Omega(S+R, S),$$

where

$$D_\Omega(S+R, S) \geq \frac{\mu}{2}\|R\|_F^2.$$

Thus the same extension is valid provided the first-order term $\langle \nabla\Omega(S), R\rangle$ is controlled or negligible along the admissible sequence. In the Frobenius case $\Omega(W) = \|W\|_F^2$, this term is exactly $2\langle S, R\rangle_F$, which is already part of the hypotheses. For the elastic-net penalty

$$\Omega(W) = \mu_1\|W\|_1 + \mu_2\|W\|_F^2, \qquad \mu_2 > 0,$$

the L_1 term is nonsmooth and can increase when R is removed if R cancels entries of S. The correct extension is subgradient-based: choose $\xi \in \partial\|S\|_1$ and assume

$$\langle \mu_1 \xi + 2\mu_2 S, R \rangle = o_{\mathbb{P}}(1).$$

The quadratic part then supplies the strong convexity needed for the same type of loss-reduction bound.

Theorem 19.16. *Assume Assumptions 4.1–4.2. Assume further that for all $s \in T$,*

$$a_\phi(N,s) := h_\phi(s) d_1(N) \to 0 \qquad \text{as } N \to \infty.$$

Suppose:

1. *for each N, $\alpha(t,N) \to \alpha^*(N)$ in probability as $t \to \infty$;*
2. *the relevant limit weight matrix admits an admissible decomposition $W^*(N) = S^*(N) + R^*(N)$ satisfying*

$$\langle S^*(N), R^*(N) \rangle_F \to 0 \qquad \text{in probability as } N \to \infty.$$

For $\vartheta \in [0,1]$, let $\alpha^*_\vartheta(N)$ denote the parameter vector obtained by replacing $W^*(N)$ with $S^*(N)+\vartheta R^*(N)$. Assume further that there exists a sequence of nonnegative random variables $\epsilon_N = o_{\mathbb{P}}(1)$ such that the one-sided directional stationarity event

$$\liminf_{h \downarrow 0} \frac{L(\alpha^*_{1-h}(N)) - L(\alpha^*(N))}{h} \geq -\epsilon_N$$

has probability tending to one. Define

$$\bar{a}_{\phi,N} := \frac{2}{|T|} \sum_{s \in T} a_\phi(N,s).$$

Then

$$\|R^*(N)\|_F^2 \leq \frac{\bar{a}_{\phi,N} + \epsilon_N}{2\lambda_{\text{reg}}} + o_{\mathbb{P}}(1).$$

In particular, if $\bar{a}_{\phi,N} + \epsilon_N \to 0$ in probability, then for every $\epsilon > 0$,

$$\lim_{N \to \infty} \mathbb{P}\big(\|R^*(N)\|_F \leq \epsilon\big) = 1.$$

If, in addition, there exists an admissible path decomposition

$$W(t,N) = S(t,N) + R(t,N)$$

such that for each fixed N,

$$\|S(t,N) - S^*(N)\|_F + \|R(t,N) - R^*(N)\|_F \to 0 \qquad \text{in probability as } t \to \infty,$$

and if for every fixed N and every $\epsilon > 0$,

$$\mathbb{P}\big(\|R^*(N)\|_F = \epsilon\big) = 0,$$

then

$$\lim_{N \to \infty} \lim_{t \to \infty} \mathbb{P}\big(\|R(t,N)\|_F \leq \epsilon\big) = 1.$$

Proof location: See Appendix 5, Proof of Theorem 19.16.

Appendix 3 gives the concrete Gaussian/RMT version of this conclusion: Theorem 3.21 proves collapse of the admissible Gaussian component, Corollary 3.23 translates this into collapse of the MP variance scale, and Corollary 3.24 records the resulting spectral stabilization.

Remark 19.17. The theorem assumes only one-sided directional approximate stationarity at the unpruned point $\vartheta = 1$. Exact local minimality is a sufficient special case with $\epsilon_N \equiv 0$, but the hypothesis is weaker than a global lower bound along the whole denoising path. Without the continuity assumption $\mathbb{P}(\|R^*(N)\|_F = \epsilon) = 0$, the same proof still gives the weaker limsup/liminf bounds obtained from Portmanteau-type arguments, but not necessarily exact convergence of the probabilities at the threshold ϵ.

Corollary 19.18 (Eventual supercriticality of persistent spike clusters in a BGN-type deformation regime)**.** *Under the hypotheses of Theorem 19.16, assume additionally that for each N the pair $W^*(N) = S^*(N) + R^*(N)$ lies in a finite-rank deformation regime satisfying all hypotheses of Theorem 5.4 for the spikes considered below. In particular, assume the regime has a deterministic bulk edge $b_+(N)$, a deterministic critical threshold $\theta_c(N)$, a strictly decreasing outlier transform*

$$D_N : (b_+(N), \infty) \to (0, D_N(b_+(N)^+))$$

in the sense of Appendix 5, local Lipschitz/invertibility intervals around the outliers, and concentration of the relevant sample singular values in those intervals. Assume that

$$\theta_c(N) \to 0, \qquad b_+(N) \to 0.$$

Assume further that for some fixed $k \geq 1$ there exist constants

$$\bar{\sigma}_1 > \cdots > \bar{\sigma}_k > 0$$

such that

$$\sigma_i^*(N) \to \bar{\sigma}_i \qquad \text{in probability as } N \to \infty$$

for each $1 \leq i \leq k$. Then:

1. $$\mathbb{P}\big(\sigma_i^*(N) > \theta_c(N)\big) \to 1 \qquad \text{for each } 1 \leq i \leq k;$$

2. *these k spikes are eventually supercritical, so the corresponding singular values of $W^*(N)$ form outlier clusters above the edge $b_+(N)$. Moreover, if the i-th spike is separated from the other nonzero singular values of $S^*(N)$ by a gap $\Delta_i(N) > 0$ with*

   $$\frac{\|R^*(N)\|_{\mathrm{op}}}{\Delta_i(N)} \to 0 \qquad \text{in probability,}$$

   then the associated rank-one left and right spectral projectors converge in operator norm to the corresponding projectors of $S^*(N)$;

3. *defining the inverse-D estimator on the event $s_i(W^*(N)) > b_+(N)$, whose probability tends to one, by*

   $$\widehat{\sigma}_i^{(D)}(N) := D_N(s_i(W^*(N)))^{-1/2},$$

   and defining it arbitrarily off that event, one has

   $$\widehat{\sigma}_i^{(D)}(N) - \sigma_i^*(N) \to 0 \qquad \text{in probability}$$

   for each $1 \leq i \leq k$.

Proof location: See Appendix 5, Proof of Corollary 19.18.

For the square Gaussian finite-rank case, the corresponding RMT statement is Corollary 3.27 in Appendix 3; the external BGN input and the inverse-D abstraction are stated in Appendix 5.

Remark 19.19. Corollary 19.18 is the abstract analogue of the Gaussian BBP statement. It says that once a finite-rank deformation model has a well-defined critical threshold $\theta_c(N)$, vanishing admissible randomness forces every structurally persistent spike cluster to cross that threshold eventually. Thus the limit theory does not merely predict collapse of the random bulk; it predicts a detectability transition for persistent informative directions in any deformation regime governed by an inverse-D outlier map.

Remark 19.20 (Assumption map for the abstract and deterministic theory). The following list summarizes which assumptions are used by the abstract additive results in this section and by the deterministic mask results in Section 5:

1. Lemma 5.1, Corollaries 5.2, 5.6, 1.1, 1.2, and 1.3, and Theorems 5.3–5.9 are deterministic statements. They do not assume an additive Gaussian decomposition; the mask-specific results use only disjoint support of the kept, restored, and removed entries.

2. Corollaries 5.10–1.6 give random sufficient conditions for the deterministic data-path budgets.

3. Lemma 4.4 uses only Assumptions 4.1–4.2.

4. Theorem 19.10, Corollary 19.12, and the finite-N pathwise Corollary 19.13 use Assumptions 4.1–4.2 together with the additional hypotheses $\langle S, R\rangle_F = o_{\mathbb{P}}(1)$ and $a_\phi(N, s) \to 0$ for each $s \in T$. Assumption 4.3 is not needed for these loss bounds. Their concrete Gaussian counterparts are Theorem 3.18 and Corollary 3.19 in Appendix 3.

5. Theorem 4.6 is obtained by applying Theorem 19.10 layer by layer; the second part allows a growing layer set under summable perturbation and failure-probability conditions.

6. Theorem 19.16 uses Assumptions 4.1–4.2, the same scaling condition $a_\phi(N, s) \to 0$, the cross-term condition at the limit point, and the one-sided directional stationarity hypothesis stated in the theorem. Its Gaussian counterpart is Theorem 3.21, with MP variance and spectral consequences in Corollaries 3.23–3.24.

7. Corollary 19.18 further assumes that the pair $W^*(N) = S^*(N) + R^*(N)$ satisfies the finite-rank deformation hypotheses of Theorem 5.4, a threshold $\theta_c(N) \to 0$, an edge $b_+(N) \to 0$, and a persistent-spike hypothesis $\sigma_i^*(N) \to \bar{\sigma}_i > 0$. The square Gaussian BBP specialization is Corollary 3.27.

8. The pathwise conclusion in Theorem 19.16 additionally uses the path-decomposition convergence assumption, while the no-atom condition $\mathbb{P}(\|R^*(N)\|_F = \epsilon) = 0$ is only needed for exact convergence of the probabilities at the threshold ϵ.

Acknowledgments

LB, HO and YS acknowledge support from NASA via the AIST program (Kernel Flows: Emulating Complex Models for Massive Data Sets). This work started when LB was on a sabbatical stay at Caltech hosted by H. Owhadi. Both LB and YS are grateful for the hospitality during their visit, supported by NASA.

The work of LB was partially supported by NSF grant DMS-2005262 and NSF grant IMPRESS-U 2401227. TB and HO acknowledge support from the Air Force Office of Scientific Research under MURI award number FA9550-20-1-0358 (Machine Learning and Physics-Based Modeling and Simulation), FOA-AFRL-AFOSR-2023-0004 (Mathematics of Digital Twins), and by the Department of Energy under award number DE-SC0023163 (SEA-CROGS: Scalable, Efficient, and Accelerated Causal Reasoning Operators, Graphs and Spikes for Earth and Embedded Systems). Additionally, HO

and TB acknowledge support from the DoD Vannevar Bush Faculty Fellowship Program under award number ONR-N000142512035.

The work of YS was partially supported by the European Research Council under the European Union's Horizon 2022 research and innovation program (grant agreement No. 101041711), the Simons Foundation, the Israel Science Foundation (grant number 2258/19), and the Israel Science Foundation (ISF Grant 4101/25).

We additionally thank the maintainers of timm [55], PyTorch [39], and the HuggingFace Hub for the open-source infrastructure on which the experiments in this paper depend. The 2:4 native sparse Tensor Core kernel is provided by NVIDIA [37]; the ToMe token merging method is by Bolya et al. [4].

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used generative AI tools to accelerate drafting and revision. All mathematical claims, proofs, numerical interpretations, and bibliographic information were subsequently reviewed and edited by the author(s), who take full responsibility for the content of the manuscript.

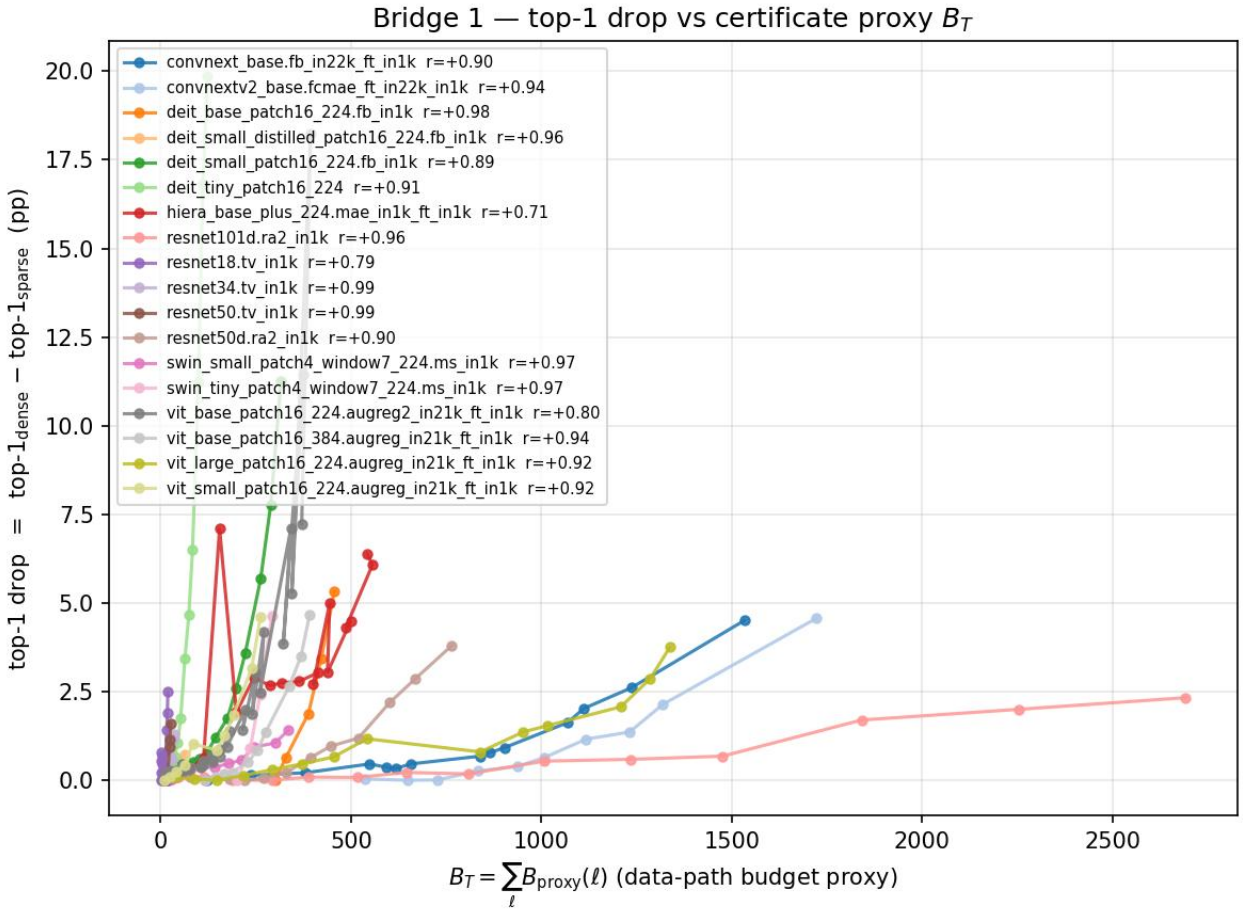


Figure 7: Top-1 drop and the operator-norm proxy $\hat{B}_T$ across all 18 paper architectures. Each curve is one architecture; each marker is one Hybrid Magnitude–SER cycle. Within-architecture Pearson correlation between the operator-norm proxy $\hat{B}_T = \sum_\ell \|R_\ell\|_{\text{op}} \overline{\|\psi_{1,\ell}\|_\infty}$ and the post-fine-tuning top-1 drop is positive for every architecture: r ranges from +0.71 (Hiera-Base+) to +0.99 (ResNet50/34 `tv_in1k`), with mean +0.91 over the 18 models. We report this as an empirical correlation: the proxy $\hat{B}_T$ is not the deterministic budget $B_T(R)$ of Eq. (see 8) ($\|R\|_{\text{op}} \|\psi\|_\infty$ is not in general an upper bound for $\|R\psi\|_\infty$), so this correlation is suggestive of the certificate quantity's relevance but does not by itself verify Lemma 5.1.

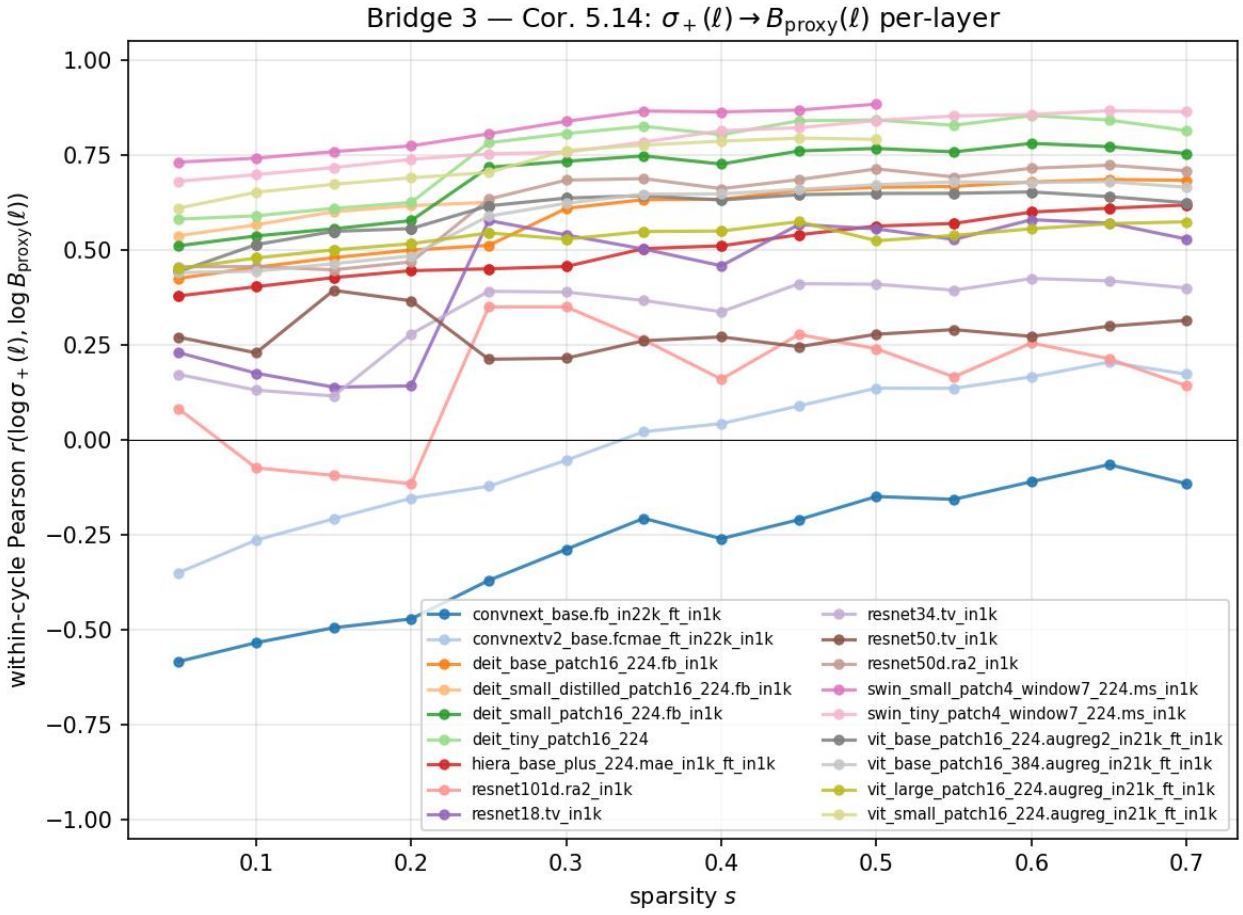


Figure 8: Within-cycle log-log Pearson correlation $r(\log \sigma_+(\ell), \log \hat{B}_{T,\ell})$ as a function of sparsity s, per architecture. Transformer architectures (Swin/DeiT/ViT) cluster in the +0.5 to +0.81 band, consistent with the iid-Gaussian heuristic of Corollary 5.11 (which is a sufficient condition under iid weights, not a certificate for the realized magnitude-selected mask). ResNet/Hiera families are positive but weaker (+0.16 to +0.62). The two ConvNeXt rows are the only architectural exception: ConvNeXtV2-Base averages ≈ 0 and ConvNeXt-Base averages -0.29. Per-layer-type stratification shows the ConvNeXt anomaly is concentrated in depthwise-convolution layers; ConvNeXt MLP layers behave like ViT MLP layers ($r = +0.51$ and $+0.58$ respectively).

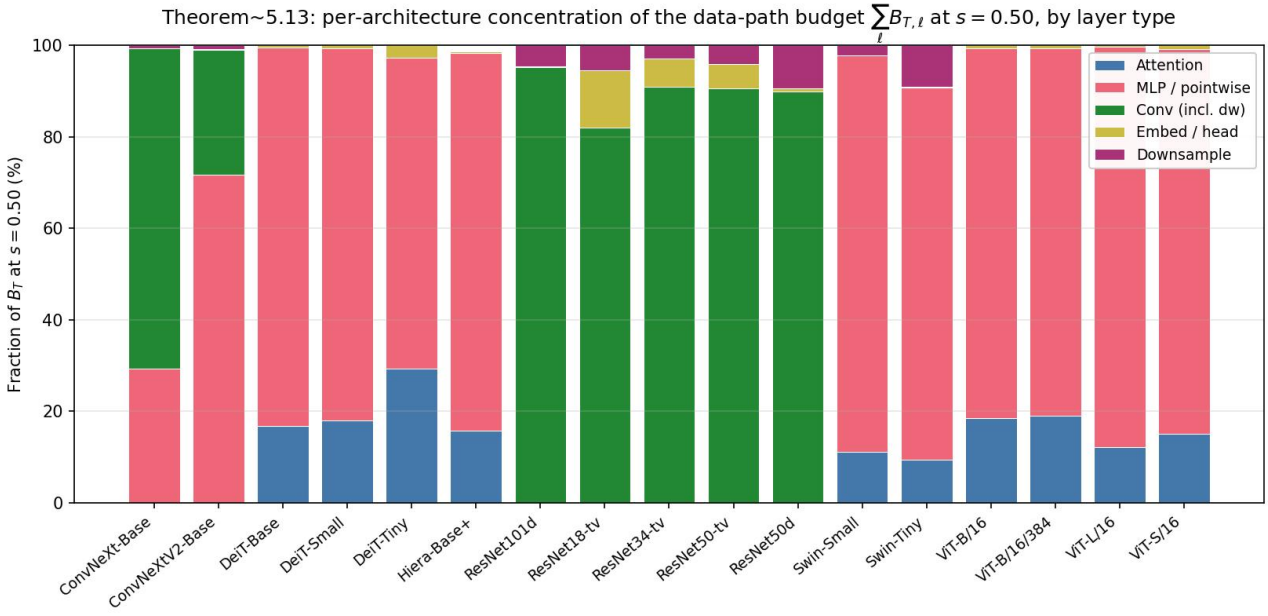


Figure 9: Per-architecture decomposition of $\sum_\ell \hat{B}_{T,\ell}$ at $s = 0.50$ by layer type. The bar height of each segment is its fraction of the total B_{total} for that architecture. Transformers (DeiT/Swin/ViT) are dominated by MLP/pointwise; ConvNeXt-Base is dominated by depthwise convolutions, ConvNeXtV2-Base by MLP/pointwise; ResNets by their convolutions.

Table 17: CAST k:n structured-projection results for parameter-to-compute conversion. Entries are post-FT ImageNet-1k top-1 under the source checkpoints and sparse-execution MAC assumptions described in the surrounding text. "Source s" is the unstructured Hybrid Magnitude–SER source sparsity; "dense" means the projection starts from the dense checkpoint. "MAC red." is the dense-equivalent sparse-execution MAC reduction, not raw parameter sparsity. "Th. spd." is the theoretical value $1/(1-\text{MAC red.})$. "Deploy spd." is a measured sparse-backend speedup for the audited checkpoint when available; Linear rows use the A40 native 2:4 endpoint at batch 128, and ResNet rows marked ‡ use exact im2col+2:4 sparse-GEMM relative to dense im2col. A dash means no exact sparse-backend wall-clock speedup is claimed for that row. Δ is top-1 minus the corresponding dense baseline, in percentage points.

Architecture	Source s	Method	Dense MACs	MAC red.	Th. spd.	Deploy spd.	Top-1 (%)	Δ (pp)
ViT family (CAST = cert-aware 2:4 + free restoration + ToMe $r = 8$ + 3-ep distill FT)								
ViT-B/16	0.35	Magnitude 2:4 + ToMe	17.56G	59.81%	2.49×	1.36×[*]	82.92	−2.19
ViT-B/16	0.35	**CAST**	17.56G	59.81%	2.49×	1.36×	**83.41**	−1.70
ViT-B/16	0.35	**CAST 6:12 SER+α=0.5** (no ToMe)	17.56G	50%	2.00×	–	**83.74**[§]	−1.37
ViT-L/16	0.35	CAST	61.55G[†]	∼60%[†]	2.50×	1.37×	**84.37**	−1.47
ViT-L/16	dense	**CAST 8:16 dense+perm** (no ToMe)	61.55G	50%	2.00×	–	**85.33**[§]	−0.51
DeiT-B	0.35	CAST	17.56G	59.81%	2.49×	1.36×	80.48	−1.32
DeiT-S	0.35	CAST	4.61G	59.81%	2.49×	1.38×	76.96	−2.89
DeiT-T	0.35	CAST	1.26G	59.81%	2.49×	1.33×	65.93	−6.28
ResNet family (CAST-conv = cert-aware 1×1+3×3 2:4 + free restoration; ToMe N/A)								
ResNet50	0.35	CAST-conv	4.09G	48.5%	1.94×	–	73.14	−2.99
ResNet50	0.35	**CAST-conv+perm**	4.09G	48.5%	1.94×	1.70×[‡]	**75.67**[‡]	−0.46
ResNet50	dense	**CAST 8:16 dense+perm**	4.09G	50%	2.00×	–	**75.87**[§]	−0.26
ResNet50d	0.35	CAST-conv	4.33G	49.85%	1.99×	–	78.08	−2.47
ResNet50d	0.35	**CAST-conv+perm**	4.33G	49.85%	1.99×	1.50×[‡]	**78.00**[‡]	−2.55
ResNet50d	dense	**CAST 8:16 dense+perm**	4.33G	50%	2.00×	–	**78.57**[§]	−1.98
ResNet101d	0.35	CAST-conv	8.0G	∼50%	2.00×	–	80.13	−2.13
ResNet101d	0.35	**CAST-conv+perm**	8.0G	∼50%	2.00×	1.57×[‡]	**80.59**[‡]	−1.67
ResNet101d	dense	**CAST 8:16 dense+perm**	8.0G	50%	2.00×	–	**80.92**[§]	−1.34
ResNet152d	0.35	**CAST-conv+perm**	11.8G	∼50%	2.00×	1.62×[‡]	**81.33**[‡]	−1.53
ConvNeXt family (CAST on the nn.Linear pointwise convs; ToMe N/A)								
ConvNeXtV2-Base	0.35	**CAST 2:4 cert + free-restore**	15.4G	50%	2.00×	1.26×	**85.47**	−1.25
ConvNeXtV2-Base	dense	**CAST 12:16 dense+perm** (25% sparse)	15.4G	25%	1.33×	–	**86.35**[§]	−0.37
ConvNeXtV2-Base	dense	**CAST 8:16 dense+perm** (50% sparse)	15.4G	50%	2.00×	–	**85.85**[§]	−0.87

[†]ViT-L/16 reduction is estimated from the analytical 22.81% ToMe reduction plus the 50% 2:4 Linear share.
[‡]**CAST-conv+perm** uses the importance-balanced Cin permutation alignment of Section 3.2; deploy speedups on these rows are exact im2col+2:4 sparse-GEMM versus dense im2col, while sparse im2col remains slower than cuDNN Conv2d end-to-end. [§]Rows marked § are wider-pattern or lower-sparsity accuracy probes; their speedups are dense-equivalent arithmetic estimates unless a native kernel is measured. [*]The magnitude row uses the same ViT-B 2:4+ToMe deployment path as the CAST row; accuracy is taken from the post-FT evaluation log, and deployability is audited on the archived 2:4 endpoint.

Table 18: Mapping from CAST/CAST-conv+perm pipeline steps to the certificate theorems of Section 5. Each row pairs an implementation component (left) with the theorem, lemma, or corollary that motivates it (right). The table records theorem-side motivations and the empirical proxy each step is intended to reduce; it does not state that the implemented pipeline verifies the literal certificate inequalities.

Pipeline step	Theorem / lemma in paper
SER $s = 0.35$ starting checkpoint	Marchenko–Pastur edge σ_+ as bulk/spike threshold (Cor. 5.11); entries below σ_+ are expected to have small certificate cost under the Gaussian sufficient condition.
Per-layer pruning budget (SEB allocator)	Cor. 5.11 — high-σ_+ layers are the random-capacity reservoirs; allocate more pruning there because $B_T(R_\ell)$ responds most weakly per unit removed Frobenius mass.
Cert-aware 2:4 mask (CAST core)	Lemma 5.1 — pick the 2-of-4 keep pattern that minimises the activation-weighted ℓ^2 proxy for the removed contribution entering $B_T(R)$, rather than sharing one pattern across rows.
Free restoration	Theorem 5.5 and Corollary 5.6 — splitting the removed mass into a kept reservoir Q and a finally-removed component P; restoration is certificate-improving when the reduction $B_T(R) - B_T(P)$ exceeds $\lambda_2\|Q\|_F^2 + \lambda_1\|Q\|_1$. The implementation uses this as motivation for zero-FLOP restoration inside structured groups.
α_{ser} Hamming prior	Theorem 5.5 + an empirical Bayesian prior on the certificate-improving Q: bias the 2:4 keep pattern toward the SER-validated unstructured mask, since SER kept entries were selected by an MP-edge-guided pruning process.
Permutation alignment (CAST-conv+perm)	Cin importance balancing minimises the dispersion of B_T contributions across 4-tuples; the importance-balanced quartile interleave of Section 3.2 ("flatten the layer") is designed to reduce the max-per-group certificate cost.

Table 19: ViT-B/16 reference near 30% compression on ImageNet-1k validation. CP-ViT values use the paper's reported ViT-B/16 rows; the comparison is indicative rather than apples-to-apples because of the checkpoint mismatch (CP-ViT baseline 77.91% vs. our 85.11%). Hybrid Magnitude–SER and the three within-paper baselines are reproduced from Table 1. Rows labeled "(ours)" are this paper's rows; bold marks notable values, not row ownership. All entries are post-fine-tuning unless noted; CP-ViT used 30 FT epochs vs. our cumulative ~ 6 FT epochs through $s = 0.30$.

Method	Top-1 (%)	Δ (pp)	Compression
ViT-B/16 [10] (our dense baseline 85.11; CP-ViT baseline 77.91)			
CP-ViT [45], no FT*	75.09	−2.82	32.34% FLOPs
CP-ViT [45], with FT*	77.36	−0.55	33.62% FLOPs
Classical magnitude (ours)	84.46	−0.65	30% param
Classical RMT (ours)	84.55	−0.56	30% param
SER (ours)	84.55	−0.56	30% param
Hybrid Magnitude–SER (ours)	**84.64**	**−0.47**	**30%** param
ViT-B/16 at higher compression (only this paper has direct numbers)			
Classical magnitude, $s = 0.50$	81.67	−3.44	50% param
Classical RMT, $s = 0.50$	82.57	−2.54	50% param
SER, $s = 0.50$	83.27	−1.84	50% param
Hybrid Magnitude–SER, $s = 0.50$ (ours)	**83.37**	**−1.74**	**50%** param
ViT-B/16 at 65–70% compression			
Classical magnitude, $s = 0.65$	74.30	−10.81	65% param
Classical RMT, $s = 0.65$	76.46	−8.65	65% param
SER, $s = 0.65$	73.80	−11.31	65% param
Hybrid Magnitude–SER, $s = 0.65$ (ours)	**79.95**	**−5.16**	**65%** param
Hybrid Magnitude–SER, $s = 0.70$ (ours)	**78.01**	**−7.10**	**70%** param
Classical magnitude, $s = 0.70$	67.44	−17.67	70% param
Classical RMT, $s = 0.70$	71.53	−13.58	70% param

*CP-ViT used a different (77.91%) ViT-B/16 checkpoint.
Numerical entries for our methods are from Table 1.

Table 20: DeiT pruning comparison after fine-tuning. This is not an apples-to-apples ranking: rows use different dense baselines, compression axes, and training protocols. "Top-1" is the post-pruning ImageNet-1k validation accuracy and "Δ" is the absolute change from the dense baseline used by that method. "Compression" is FLOPs savings for VTP/PoWER/HVT/CP-ViT and parameter sparsity for Hybrid Magnitude–SER. VTP, PoWER, HVT, and CP-ViT DeiT values are taken from CP-ViT [45]. CP-ViT states that its ImageNet fine-tuning uses 30 epochs; the fine-tuning budgets for the VTP/PoWER/HVT fine-tuned values are not reported there. Hybrid fine-tunes for one epoch per sparsity step, totaling 4 epochs at $s = 0.20$ and 7–8 epochs at $s = 0.40$. Rows labeled "(ours)" are this paper's rows; bold marks notable values, not row ownership.

Method	Top-1 (%)	Δ (pp)	Compression
DeiT-Tiny 16/224 [51] (prior dense 72.20; our dense 72.21)			
VTP [67]†	70.55	−1.65	45.32% FLOPs
PoWER-BERT [17]†	70.05	−2.15	41.26% FLOPs
HVT [38]†	70.01	−2.19	47.32% FLOPs
CP-ViT [45]	71.24	−0.96	43.34% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ (ours)	**71.86**	**−0.35**	**20%** param
DeiT-Small 16/224 [51] (prior dense 79.80; our dense 79.85)			
VTP [67]†	78.24	−1.56	42.52% FLOPs
PoWER-BERT [17]†	78.30	−1.50	41.36% FLOPs
HVT [38]†	78.05	−1.75	47.80% FLOPs
CP-ViT [45]	79.08	−0.72	42.24% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ (ours)	**79.37**	**−0.48**	**20%** param
Hybrid Magnitude–SER, $s = 0.40$ (ours)	78.61	−1.24	40% param
DeiT-Base 16/224 [51] (prior dense 81.82; our dense 81.80)			
VTP [67]†	80.70	−1.12	43.20% FLOPs
PoWER-BERT [17]†	80.17	−1.65	39.24% FLOPs
HVT [38]†	79.94	−1.88	44.78% FLOPs
CP-ViT [45]	81.13	−0.69	41.62% FLOPs
Hybrid Magnitude–SER, $s = 0.20$ (ours)	**81.46**	**−0.34**	**20%** param
Hybrid Magnitude–SER, $s = 0.35$ (ours)	81.17	−0.63	35% param
Hybrid Magnitude–SER, $s = 0.40$ (ours)	80.99	−0.81	40% param

†Fine-tuned values copied from CP-ViT Table 4; CP-ViT's replication note applies to without-finetuning VTP/PoWER/HVT accurac

20 Full Literature-Context Table from the Main Manuscript

Table 21 is the full comparison table migrated out of the main article to keep the submitted paper in a concise Letters format. The compact main-paper comparison retains the closest unstructured and structured representations; this table preserves the broader provenance and scope notes for reviewers.

Table 21: Full ImageNet-1k pruning and structured-sparsity context from the main manuscript. This is not an apples-to-apples ranking: rows start from different dense accuracies, checkpoints, architectures, compression axes, and fine-tuning budgets. Unstructured/fine-grained rows are accuracy and compression context unless the notes state an audited endpoint; N:M, token, or structural rows are separate deployment axes. Rows labeled "(ours)" are this paper's results. Bold marks notable values within this context, not row ownership. "FT/train" is the reported training or post-pruning fine-tuning budget.

Method	Architecture	Compression	Top-1	Δ vs. dense	FT/train	Notes
ViT-B/ViT-L context						
CP-ViT [45]	ViT-B/16	32.34% FLOP red.	75.09	−2.82	0	CP-ViT Table 2, baseline 77.91%
CP-ViT [45]	ViT-B/16	33.62% FLOP red.	77.36	**−0.55**	30	CP-ViT Table 3, same baseline
SNOWS [33]	ViT-B/16	2:4 QKV+Out+MLP	76.57	−3.85	0	one-shot on MiniImageNet-1k; 1.97× FLOP-count estimate
SNOWS [33]	ViT-L/16	2:4 QKV	81.01	−3.16	0	one-shot on MiniImageNet-1k; no ViT-L endpoint speedup reported
MaskLLM-4V [14]	ViT-B/16	2:4 learned mask	**79.46**	**+0.31**	20 mask ep.	weights frozen; no ViT endpoint speedup reported
PaPr [34]	ViT-B/16 AugReg	patch pruning + ToMe	82.34	−2.25	0	PaPr Table 1; 53.3% FLOP red.; 2.15× throughput; token pruning, not weight sparsity
PaPr [34]	ViT-L/16 MAE	patch pruning	**85.06**	−0.89	0	PaPr Table 2; 50.0% FLOP red.; **2.01**× throughput; token pruning, not weight sparsity
ToMe [4]	ViT-L/16 MAE	token merging	**85.05**	**−0.61**	MAE FT	ToMe Tables 3/10; 49.7% FLOP red.; **1.96**× V100 throughput; token merging, not weight sparsity
ToMe [4]	ViT-L/16 MAE	token merging, decreasing	84.16	−1.50	MAE FT	ToMe Tables 3/10; 63.8% FLOP red.; **2.58**× V100 throughput; token merging, not weight sparsity
SparseFormer [16]	ViT-B/16 AugReg	latent-token reduction	83.40	−1.20	20+5 ep.	64.6% FLOP red.; **1.85**× throughput; token reduction, not weight sparsity
SparseFormer [16]	ViT-L/16 AugReg	latent-token reduction	**85.20**	**−0.60**	20+5 ep.	69.8% FLOP red.; **2.60**× throughput; token reduction, not weight sparsity
Hybrid Mag–SER (ours)	ViT-B/16	30% unstructured	**84.64**	**−0.47**	1/cycle	Table 2, $s = 0.30$
Hybrid Mag–SER (ours)	ViT-B/16	50% unstructured	**83.37**	−1.74	1/cycle	Table 2, $s = 0.50$

Continued on next page

Table 21: Full ImageNet-1k pruning and structured-sparsity context from the main manuscript (continued).

Method	Architecture	Compression	Top-1	Δ vs. dense	FT/train	Notes
CAST 2:4+ToMe (ours)	ViT-B/16	2:4 + ToMe	**83.41**	−1.70	**3**	A40 native 2:4 endpoint 1.388×; separate A100 no-ToMe endpoint 2.705×
CAST 6:12 SER+α (ours)	ViT-B/16	6:12 = 50%	**83.74**	**−1.37**	**3**	accuracy/MAC row; no native 6:12 audit
CAST 2:4+ToMe (ours)	ViT-L/16	2:4 + ToMe	**84.37**	−1.47	**3**	A40 native 2:4 endpoint 1.394×
CAST 8:16 dense+perm (ours)	ViT-L/16	8:16 = 50%	**85.33**	**−0.51**	**3**	accuracy/MAC row; no native 8:16 audit
Other ViT-family deployability context						
GPUSQ-ViT [61]	Swin-B	2:4+INT8	83.40	−0.10	KD+QAT	31× FLOP-count red.; A100 1.37× latency and 2.56× throughput
GPUSQ-ViT [61]	Swin-B/384	2:4+INT8	**84.40**	**−0.10**	KD+QAT	31× FLOP-count red.; A100 1.47× latency and 2.47× throughput
CNN and ConvNeXt context						
Zhu–Gupta [66]	InceptionV3	50% unstructured	**78.0**	**−0.1**	*n/r*	model-size result; no endpoint speed row
Zhu–Gupta [66]	MobileNetV1	50% unstructured	69.5	−1.1	*n/r*	also reports 75% sparsity at 67.7; no endpoint speed row
SFP [19]	ResNet-50	41.8% FLOPs	74.61	−1.54	from scratch; *n/r* ep.	filter/channel-pruned CNN
HRank [26]	ResNet-50	43.8% FLOPs	74.98	−1.17	30 ep./block	feature-map-rank filter pruning
FPGM [20]	ResNet-50	53.5% FLOPs	74.83	−1.32	FT; *n/r* ep.	geometric-median filter pruning
ResRep [9]	ResNet-50	54.5% FLOPs	**76.15**	**0.00**	180 train	structural re-parameterization; narrowed dense CNN
DepGraph [12]	ResNet-50	51.8% MACs	**75.83**	−0.32	FT; *n/r* ep.	automatic structural pruning
Mishra et al. [37]	ResNet-50	2:4 FP16	**76.20**	**+0.10**	repeated train	original 2:4 paper; up to **2**× sparse Tensor Core math
Mishra et al. [37]	ResNet-50 SWSL	2:4 FP16	**80.90**	**−0.20**	repeated train	high-accuracy original-paper reference; up to **2**× math
Mishra et al. [37]	ResNet-101	2:4 FP16	**78.00**	**+0.30**	repeated train	original 2:4 paper; up to **2**× math
Pool–Yu perm. [41]	ResNet-50	2:4+perm	**76.29**	**+0.13**	90 FT	no runtime overhead; V100 search time 02:13 = 0.10% of train+FT
SR-STE [65]	ResNet-50	2:4	77.00	−0.30	120 train	*N*:*M* sparse training from scratch; 2.15G test FLOPs
LBC [63]	ResNet-50	2:4	**77.20**	**+0.10**	120 train	learns *N*:*M* combinations; 0.72× training FLOPs
AC/DC [40]	ResNet-50	2:4	**76.64**	**−0.20**	100 train	semi-structured AC/DC sparse training; no endpoint speed reported
AC/DC [40]	ResNet-50	50% unstructured	**77.05**	**+0.21**	100 train	global pruning result; no 50% speed row

Continued on next page

Table 21: Full ImageNet-1k pruning and structured-sparsity context from the main manuscript (continued).

Method	Architecture	Compression	Top-1	Δ vs. dense	FT/train	Notes
MIS [60]	ResNet-50	70% fine-grained	**76.56**	**+0.43**	distill/FT	supervised MIS; 70%-FINE/AGP baseline also 76.50; separate GPU speed ranges are not tied to this supervised row
Random pruning [28]	WRN-50	60/70% unstructured	77.55/76.81	−0.56/ − 1.30	sparse train	SNIP-ratio layer allocation; ResNet-family, not vanilla ResNet-50
UniPTS [57]	ResNet-50	50/60/70% unstructured	75.76/75.37/74.73	−0.36/ − 0.75/ − 1.39	16k iters	post-training sparsity with 10,240 ImageNet calibration images; no speed row
STR [23]	ResNet-50	79.55/81.27% unstructured	76.19/76.12	−0.82/ − 0.89	train	nearby 80%+ sparsity evidence; 766/705M reported FLOPs, not a 50–70% row
HALP [43]	ResNet-50	51.2% FLOP red.	**76.50**	**+0.30**	90 FT	1.60× Titan V; TensorRT FP32 1.80×
HALP [43]	ResNet-101	53.8% FLOP red.	**77.80**	**+0.40**	90 FT	1.60× Titan V structural-pruning endpoint
NVIDIA TRT INT8 [6]	ResNet-34	2:4+INT8	73.16	−0.07	*n/r*	1.36× A40 TensorRT INT8; batch sweep up to 1.40×
MDP [48]	ResNet-50	∼ 73% FLOPs	74.80	−1.40	90 FT	3.06× Titan V measured speed; stronger reduction, larger drop
Group Fisher [27]	ResNet-50	50.0% FLOPs	**76.42**	−0.37	FT as dense	1.79× 2080 Ti measured inference speed
OVW [50]	ResNet-50	50% column-vector sparsity	75.76	−0.36	20 FT	1.86× V100 sparse-conv speedup
Isomorphic [13]	ResNet-50	50.1% MAC red.	**75.91**	**−0.22**	100 FT	structural pruning; dense-kernel model
Isomorphic [13]	ResNet-101	51.0% MAC red.	**77.43**	**+0.05**	100 FT	structural pruning; dense-kernel model
Isomorphic [13]	ResNet-152	64.9% MAC red.	**77.84**	**−0.47**	100 FT	structural pruning; dense-kernel model
CAP [24]	ResNet-50D	50/60% unstructured	**79.8/79.7**	**−0.7/ − 0.8**	300-epoch gradual	dense 80.5%; no endpoint speed row
Isomorphic [13]	ConvNeXt-B→S	44.8% MAC red.	**83.17**	**−0.66**	300 FT	pruned ConvNeXt-style dense model
Isomorphic [13]	ConvNeXt-B→T	72.7% MAC red.	82.19	−1.64	300 FT	pruned ConvNeXt-style dense model
CAP [24]	ConvNeXt-L CLIP	50/60/70% unstructured	**87.5/87.1/86.8**	**−0.3/ − 0.7/ − 1.0**	0	one-shot pruning; dense 87.8%; no endpoint speed row
OWL+Wanda [59]	ConvNeXt-B	50/60/70% unstructured	82.76/80.53/68.28	–	0	appendix one-shot vision baseline; 70% collapses; no vision speed row
SST slices [49]	ConvNeXt-S	mixed dense/2:4	82.62	−0.47	*n/r*	1.93× FPGA in-fabric estimate; BF16, depthwise layers dense
Hybrid Mag–SER (ours)	ResNet50	50/60/70% unstructured	**75.76**/74.83/74.15	−0.37/ − 1.30/ − 1.98	1/cycle	Table 2, $s = 0.50/0.60/0.70$
Hybrid Mag–SER (ours)	ConvNeXt-B	50/60/70% unstructured	**84.80**/83.70/81.21	−1.04/ − 2.14/ − 4.63	1/cycle	Table 2, $s = 0.50/0.60/0.70$
CAST-conv+perm (ours)	ResNet50	2:4 im2col	**75.67**	**−0.46**	**3**	**1.700**× A40 im2col audit; not cuDNN Conv2d

Continued on next page

Table 21: Full ImageNet-1k pruning and structured-sparsity context from the main manuscript (continued).

Method	Architecture	Compression	Top-1	Δ vs. dense	FT/train	Notes
CAST-conv+perm (ours)	ResNet50d	2:4 im2col	78.00	−2.55	**3**	1.496× A40 im2col audit; not cuDNN Conv2d
CAST-conv+perm (ours)	ResNet101d	2:4 im2col	**80.59**	−1.67	**3**	**1.568**× A40 im2col audit; not cuDNN Conv2d
CAST-conv+perm (ours)	ResNet152d	2:4 im2col	**81.33**	−1.53	**3**	**1.617**× A40 im2col audit; stronger timm baseline
CAST 2:4 (ours)	ConvNeXtV2-B	50% MAC-accounting	**85.47**	−1.25	**3**	1.295× A40 native Linear endpoint; dense convs unchanged
CAST 8:16 (ours)	ConvNeXtV2-B	50% MAC-accounting	**85.85**	−0.87	**3**	accuracy/MAC row; no native 8:16 audit
CAST 12:16 (ours)	ConvNeXtV2-B	∼25% MAC-accounting	**86.35**	**−0.37**	**3**	accuracy/MAC row; no native 12:16 audit

21 Migrated Main-Manuscript Numerical Appendices

This section collects numerical, protocol, provenance, and algorithm material that was moved out of the main manuscript and into Online Resource 2. It is supplied as PDF supplementary information so that reviewers and readers have a stable Springer-hosted version, not only a repository copy.

22 CAST/CAST-2E Recipe Table

Table 22 records the compact recipe summary for the main structured-projection rows.

23 Comparison Provenance and Scope Notes

Training budgets already shown in Table 5 are not repeated here. Zero-fine-tuning author-run one-shot baselines and external rows without a reported training/fine-tuning budget are omitted from the main comparison table; the omitted values are SparseGPT-style 50% unstructured ViT-B/16 no-FT at $\sim 81.1\%$ (~ -4.0 pp) and Wanda-style at $\sim 81.3\%$ (~ -3.8 pp). DeiT-only comparison rows are not used in the main comparison because the headline rows here are ViT-B/16, ViT-L/16, ResNet, and ConvNeXt-family checkpoints. SNOWS evaluates on MiniImageNet-1k subsets rather than the full ImageNet-1k validation set. CP-ViT's ViT-B/16 entries in Table 5 use the CP-ViT-reported 77.91% dense baseline. For this paper's rows, the prunable-layer list is fixed across the multi-architecture sweep; masks are constructed without ImageNet validation access; validation is used only to report completed checkpoints and in documented global-protocol sweeps. Results are reported against the corresponding dense checkpoint measured in this paper's evaluation pipeline.

Table 4 reports the selected best-observed A40 batch-sweep endpoints; batch 256 was also swept but not selected. The different A100 no-ToMe ViT-B endpoint remains separate: 463.6 → 1254.1 images/s, 2.705×, and should not be pooled with the ToMe-held-fixed A40 audits. ToMe-only AugReg references from [4] are 83.94% at $r = 8$ and 82.86% at $r = 12$.

Extending the ResNet endpoint caveat in Table 4, the im2col+2:4 rows do not establish TensorRT or cuDNN Conv2d acceleration. The 1×1-only controls remain below native cuDNN Conv2d throughput; see Section 5.3. A ResNet152d 8:16 dense+perm row is outside the reported fixed-calibration sweep: the wider-pattern certificate projection exceeded a single 80 GB A100 at the default calibration batch.

Table 22: Hyperparameters for representative CAST/CAST-2E rows of Table 3. The table lists the source checkpoint for each run and the post-projection distillation schedule. The CAST projection (Algorithm 7) is gradient-free.

	ViT-B/16 6:12	**ViT-B/16 2:4+ToMe**	**ViT-L/16 2:4+ToMe**	**ResNet50d 8:16**
Source ckpt	SER $s = 0.35$	SER $s = 0.35$	SER $s = 0.35$	dense (no SER)
Calibration set $\|\mathcal{C}\|$	256	256	256	64
Permute align	yes	no	no	yes
Free restoration	yes	yes	yes	yes
α_{ser}	0.5	0.1	0.1	0.0
ToMe r	—	8	8	—
Stage 2 distillation:				
Optimiser	AdamW	AdamW	AdamW	SGD+momentum 0.9
Base LR	1×10^{-4}	1×10^{-5}	1×10^{-5}	1×10^{-3}
Weight decay	0.05	0.01	0.01	1×10^{-4}
LR schedule	cosine + warmup	cosine + warmup	cosine + warmup	cosine + warmup
Warmup steps	1000	1000	1000	500
Train batch	256	32	8	256
Val batch	128	64	16	128
Label smoothing ϵ	0.1	0.1	0.1	0.1
Distill (α, τ)	$(0.5, 2.0)$	$(0.5, 2.0)$	$(0.5, 2.0)$	$(0.5, 2.0)$
Epochs	3	3	3	3
Mask re-zero	every step	every step	every step	every step
Post-FT top-1	Reported in Table 3			

The wider 8:16 group expands the per-row search space from $\binom{4}{2} = 6$ to $\binom{16}{8} = 12{,}870$ patterns at the same nominal sparsity, and the dense source removes the SER-mask Hamming bias against weights previously discarded by unstructured pruning. The ViT-L/16 8:16 row has released evidence and checkpoint-status metadata in `data/release_manifest.json`.

The public artifact bundle is indexed by `data/release_manifest.json`; Online Resource 2 gives protocol-level provenance and ledger details. CAST row estimates are primarily single-seed; the available replicate summaries provide the repeatability check for the reported CAST setting.

24 Supplementary Numerical Results

The supplementary numerical material covers MP-based pruning of fully connected DNNs on Fashion MNIST, outer-layer scaling diagnostics, and the noise-perturbation experiment. The extended version appears in the companion *methodology and extended-numerics* paper, `cast_2e/methodology.pdf` (LaTeX source: `cast_2e/methodology.tex`). The repository root is `https://github.com/yspennstate/RMT_based_pruning_in_deep_learning`.

24.1 Interpretation of the ViT-B/16 sweep

For the full layerwise interpretation of why RMT is used as a budget allocator rather than direct sub-edge singular-component removal, see Section 14 and Section 9.

24.2 Multi-architecture cert connection

Why the slope steepens with sparsity. The slope-steepening and bulk-reservoir interpretation of Figure 2 are discussed in Section 15 and Section 16.

Architecture and anomalous-cell provenance. Architecture provenance, adaptive-variant details, and the Hiera-Base+ anomalous-cell note are in Section 15; no correction was applied to the archived checkpoint.

Connecting the deterministic certificates to the multi-architecture numerics. The deterministic mask certificate (Theorem 5.4) certifies that pruning a removed component R from a layer $A = S + R$ decreases the elastic-net objective whenever the data-path budget satisfies

$$B_T(R) \;<\; \lambda_2 \|R\|_F^2 + \lambda_1 \|R\|_1, \tag{20}$$

where $B_T(R)$ is the data-path budget.

The full theory-to-numerics bridge, including the iid-Gaussian σ_+ caveat, prune–restore interpretation, operator-norm proxy audit, and 282-checkpoint correlation results, is given in Section 16.

25 Algorithms for RMT-based pruning diagnostics

The BEMA edge estimator, spectral-fit test, and cycle-based pruning algorithms for the classical RMT baseline are specified in Online Resource 2.

26 Spectral Edge Budgeting and hybrid RMT pruning protocols

Operational SEB/SER protocols, four-regime strategy, layer-type extension, SEB final comparison, Haar-SV preprocessing sweep, Hybrid Magnitude–SER protocol, and checkpoint validation ledgers used in Table 2 are specified in Online Resource 2.

The algorithm below is the SER step used after the exact-magnitude low-sparsity prefix in Hybrid Magnitude–SER.

27 CAST: certificate-aware 2:4 + ToMe conversion

The original CAST 2:4+ToMe pipeline, generalized CAST-2E k:n pipeline, permutation alignment, and inline-FT runner are specified in Online Resource 2.

Justification of Algorithm 7. The cost $c_{i,g}(m)$ is the squared ℓ^2 form of the removed-mass contribution to output channel i of layer ℓ, evaluated on the calibration set. Lemma 5.1 bounds $|\Delta\,\mathrm{CE}|$ through the literal quantity $L_s \|R\psi_1(s)\|_\infty$. CAST does not estimate L_s; it minimises the calibration-set ℓ^2 surrogate $r^\top C_g r$ for $R = W - W'$, treating it as a certificate-inspired proxy under the activation distribution captured by C_g. The free-restoration step materialises the slot-value rule of Theorem 5.5: slots that the unstructured mask had zeroed but the chosen k:n pattern keeps are filled from the dense checkpoint at zero sparse-execution FLOP cost, which can improve the certificate bound under the conditions of Corollary 5.6. The α_{ser} prior pulls the chosen pattern towards the RMT-allocated SER mask used in the ViT-B experiments.

References

[1] Sangho An, Jinwoo Kim, Keonho Lee, Jingang Huh, Chanwoong Kwak, Yujin Lee, Moonsub Jin, and Jangho Kim. Sparse structure exploration and re-optimization for vision transformer. In *Proceedings of the Forty-first Conference on Uncertainty in Artificial Intelligence*, volume

Algorithm 6 Spectral Edge Reallocation (SER) algorithm. The step prunes weights, restores selected weights from the removed-weight reservoir, and uses MP-budget allocation in Hybrid Magnitude–SER.

Require: Trained network with weights $\{W_\ell\}_{\ell=1}^{L}$; target sparsity $s \in (0,1)$; over-prune ratio $\rho \in (1, 1+\epsilon]$; layerwise MP edge signal $\{\sigma_+(\ell)\}_{\ell=1}^{L}$; budget-allocation parameters $(\beta, s_{\mathrm{d}}, p)$; short FT schedule $\mathcal{T}_{\mathrm{FT}}$.

function SER_PRUNE_RESTORE(network, s, ρ, σ_+, β, s_{d}, p)
  Compute layerwise weights $w_\ell(s) = \max(0.1, 1 + \beta\, d(s)\, z_\ell)$ where $z_\ell = (\sigma_+(\ell) - \bar{\sigma}_+)/\mathrm{sd}(\sigma_+)$ and $d(s) = (1 - s/s_{\mathrm{d}})_+^p$. ▷ Section 3
  Score each parameter $|W_\ell|_{ij}/w_\ell(s)$ and prune the smallest $\rho \cdot s$ fraction globally. ▷ Over-prune
  Define the reservoir $\mathcal{R} = \{(\ell, i, j) : (W_\ell)_{ij} \text{ pruned}\}$.
  With kept weights frozen, run a short selection phase (one ImageNet-1k epoch at the same optimiser/LR/batch as the eventual fine-tune) that allows only the reservoir entries to receive gradients. Record the absolute movement $|\Delta_{ij}^{(\ell)}|$ of each reservoir entry from its pre-selection value.
  Rank reservoir entries by $|\Delta_{ij}^{(\ell)}| \cdot \rho_\ell(s)$ where $\rho_\ell(s) = w_\ell(s)$ is the layer's σ_+-derived budget weight from Step 1, recomputed at the post-prune sparsity. ▷ RMT-ranked reinsertion
  Restore the top entries until realised post-restoration sparsity equals s.
  Freeze the resulting zero mask M on all layers. Run $\mathcal{T}_{\mathrm{FT}}$ (one fine-tuning epoch in our schedule) with $W_\ell \leftarrow W_\ell \odot M$ re-applied at each step.
  return restored network and final mask M.
end function

286 of *Proceedings of Machine Learning Research*, pages 111–131. PMLR, 2025. URL https://proceedings.mlr.press/v286/an25a.html.

[2] Hangbo Bao, Li Dong, Songhao Piao, and Furu Wei. BEiT: BERT pre-training of image transformers. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=p-BhZSz59o4.

[3] Leonid Berlyand, Etienne Sandier, Yitzchak Shmalo, and Lei Zhang. Enhancing accuracy in deep learning using random matrix theory. *Journal of Machine Learning*, 3(4):347–412, 2024. doi: 10.4208/jml.231220.

[4] Daniel Bolya, Cheng-Yang Fu, Xiaoliang Dai, Peizhao Zhang, Christoph Feichtenhofer, and Judy Hoffman. Token merging: Your ViT but faster. In *International Conference on Learning Representations*, 2023. URL https://openreview.net/forum?id=JroZRaRw7Eu.

[5] Chun-Fu (Richard) Chen, Quanfu Fan, and Rameswar Panda. CrossViT: Cross-attention multi-scale vision transformer for image classification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 357–366, October 2021. doi: 10.1109/ICCV48922.2021.00041. URL https://openaccess.thecvf.com/content/ICCV2021/html/Chen_CrossViT_Cross-Attention_Multi-Scale_Vision_Transformer_for_Image_Classification_ICCV_2021_paper.html.

[6] Gwena Cunha Sergio, Sagar Shelke, Jinkyu Koo, Le An, and Josh Park. Sparsity in INT8: Training workflow and best practices for NVIDIA TensorRT acceleration. NVIDIA Technical Blog, 2023. URL https://developer.nvidia.com/blog/sparsity-in-int8-training-workflow-and-best-practices-for-tensorrt-acceleration/.

Algorithm 7 CAST $k{:}n$ structured-projection algorithm. The projection selects row-wise structured masks; Stage 2 applies frozen-mask distillation.

Require: Linear layer $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with $d_{\text{in}} = nG$; SER source mask $M^{\text{SER}} \in \{0,1\}^{d_{\text{out}} \times d_{\text{in}}}$ and SER weight W^{SER}; dense pretrained W^{dense}; calibration set $\mathcal{C}$ (row-specific size in Table 22); $k{:}n$ pattern; flag `free_restoration`; $\alpha_{\text{ser}} \geq 0$.

function CAST_PROJECT_KN(W, M^{SER}, W^{SER}, W^{dense}, $\mathcal{C}$, k, n, α_{ser}, `free_restoration`)
  Reshape rows of W into G contiguous n-tuples; let $W_{i,g,k}$ index the k-th weight of group g in output row i.
  For each input-channel group g, capture activations $h_g(x) \in \mathbb{R}^n$ over $\mathcal{C}$ and form within-group covariance $C_g = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x) h_g(x)^\top$.
  for each output row $i = 1, \ldots, d_{\text{out}}$ and group $g = 1, \ldots, G$ in parallel **do**
    for each candidate keep pattern $m \in \{0,1\}^n$ with $|m|_0 = k$ **do**
      Define source tuple $u_{i,g,k} = \begin{cases} W^{\text{SER}}_{i,g,k} & \text{if } M^{\text{SER}}_{i,g,k} = 1, \\ W^{\text{dense}}_{i,g,k} & \text{if } M^{\text{SER}}_{i,g,k} = 0 \text{ and } \texttt{free\_restoration}, \\ 0 & \text{otherwise.} \end{cases}$
      Materialise retained tuple $v_{i,g,k}(m) = m_k \cdot u_{i,g,k}$.
      Compute removed tuple $r_{i,g}(m) = (1 - m) \odot u_{i,g}$ ▷ entries not retained
      Compute cost $c_{i,g}(m) = r_{i,g}(m)^\top C_g \, r_{i,g}(m) + \alpha_{\text{ser}} \, \bar{c}_g \, |m - M^{\text{SER}}_{i,g,:}|_1$,
  where $\bar{c}_g = \text{mean}_{i,k} \, u^2_{i,g,k} \cdot \mathbb{E}_x[h_{g,k}(x)^2]$ normalises the prior to the cost magnitude.
    end for
    $m^*_{i,g} \leftarrow \arg\min_{m \in \mathcal{M}_{k:n}} c_{i,g}(m)$.
    Set the new weight tuple $W'_{i,g,:} \leftarrow v_{i,g,:}(m^*_{i,g})$.
  end for
  return projected layer W' and the per-row $k{:}n$ mask $M^{\text{CAST}}_{i,g,k} = m^*_{i,g}[k]$.
end function

[7] Zihang Dai, Hanxiao Liu, Quoc V. Le, and Mingxing Tan. CoAtNet: Marrying convolution and attention for all data sizes. In *Advances in Neural Information Processing Systems*, volume 34, pages 3965–3977. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/20568692db622456cc42a2e853ca21f8-Paper.pdf.

[8] Mingyu Ding, Bin Xiao, Noel Codella, Ping Luo, Jingdong Wang, and Lu Yuan. DaViT: Dual attention vision transformers. In *Computer Vision – ECCV 2022*, volume 13684 of *Lecture Notes in Computer Science*, pages 74–92. Springer, 2022. doi: 10.1007/978-3-031-20053-3_5. URL https://doi.org/10.1007/978-3-031-20053-3_5.

[9] Xiaohan Ding, Tianxiang Hao, Jianchao Tan, Ji Liu, Jungong Han, Yuchen Guo, and Guiguang Ding. ResRep: Lossless CNN pruning via decoupling remembering and forgetting. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 4510–4520, 2021.

[10] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.

[11] Alaaeldin El-Nouby, Hugo Touvron, Mathilde Caron, Piotr Bojanowski, Matthijs Douze, Armand Joulin, Ivan Laptev, Natalia Neverova, Gabriel Synnaeve, Jakob Verbeek, and Herve Jegou. XCiT: Cross-covariance image transformers. In *Advances in Neural Information Processing Systems*, volume 34, pages 20014–20027, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/a655fbe4b8d7439994aa37ddad80de56-Paper.pdf.

[12] Gongfan Fang, Xinyin Ma, Mingli Song, Michael Bi Mi, and Xinchao Wang. DepGraph: Towards any structural pruning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16091–16101, 2023.

[13] Gongfan Fang, Xinyin Ma, Michael Bi Mi, and Xinchao Wang. Isomorphic pruning for vision models. In *Computer Vision – ECCV 2024*, volume 15088 of *Lecture Notes in Computer Science*, pages 232–250. Springer, 2024. doi: 10.1007/978-3-031-73404-5_14. URL https://link.springer.com/chapter/10.1007/978-3-031-73404-5_14.

[14] Gongfan Fang, Hongxu Yin, Saurav Muralidharan, Greg Heinrich, Jeff Pool, Jan Kautz, Pavlo Molchanov, and Xinchao Wang. MaskLLM: Learnable semi-structured sparsity for large language models. In *Advances in Neural Information Processing Systems*, volume 37, pages 7736–7758. Curran Associates, Inc., 2024. doi: 10.52202/079017-0248. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/0e9a05f5ce62284c91e4a33498899124-Paper-Conference.pdf.

[15] Yuxin Fang, Quan Sun, Xinggang Wang, Tiejun Huang, Xinlong Wang, and Yue Cao. EVA-02: A visual representation for neon genesis. *Image and Vision Computing*, 149:105171, 2024. doi: 10.1016/j.imavis.2024.105171. URL https://doi.org/10.1016/j.imavis.2024.105171.

[16] Ziteng Gao, Zhan Tong, Kevin Qinghong Lin, Joya Chen, and Mike Zheng Shou. Bootstrapping SparseFormers from vision foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 17710–17721, June 2024. doi: 10.1109/CVPR52733.2024.01677. URL https://openaccess.thecvf.com/content/CVPR2024/html/Gao_Bootstrapping_SparseFormers_from_Vision_Foundation_Models_CVPR_2024_paper.html.

[17] Saurabh Goyal, Anamitra Roy Choudhury, Saurabh M. Raje, Venkatesan T. Chakaravarthy, Yogish Sabharwal, and Ashish Verma. PoWER-BERT: Accelerating BERT inference via progressive word-vector elimination. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3690–3699. PMLR, 2020.

[18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. URL https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html.

[19] Yang He, Guoliang Kang, Xuanyi Dong, Yanwei Fu, and Yi Yang. Soft filter pruning for accelerating deep convolutional neural networks. In *Proceedings of the Twenty-Seventh International Joint Conference on Artificial Intelligence*, pages 2234–2240, 2018. doi: 10.24963/ijcai.2018/309.

[20] Yang He, Ping Liu, Ziwei Wang, Zhilan Hu, and Yi Yang. Filter pruning via geometric median for deep convolutional neural networks acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4340–4349, 2019.

[21] Ning-Chi Huang, Chi-Chih Chang, Wei-Cheng Lin, Endri Taka, Diana Marculescu, and Kai-Chiang Wu. ELSA: Exploiting layer-wise N:M sparsity for vision transformer acceleration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pages 8006–8015, June 2024.

[22] Zheng Tracy Ke, Yucong Ma, and Xihong Lin. Estimation of the number of spiked eigenvalues in a covariance matrix by bulk eigenvalue matching analysis. *Journal of the American Statistical Association*, 118(541):374–392, 2023. doi: 10.1080/01621459.2021.1933497.

[23] Aditya Kusupati, Vivek Ramanujan, Raghav Somani, Mitchell Wortsman, Prateek Jain, Sham M. Kakade, and Ali Farhadi. Soft threshold weight reparameterization for learnable sparsity. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5544–5555. PMLR, 2020. URL https://proceedings.mlr.press/v119/kusupati20a.html.

[24] Denis Kuznedelev, Eldar Kurtić, Elias Frantar, and Dan Alistarh. CAP: Correlation-aware pruning for highly-accurate sparse vision models. In *Advances in Neural Information Processing Systems*, volume 36, pages 28805–28831. Curran Associates, Inc., 2023. URL https://proceedings.neurips.cc/paper_files/paper/2023/hash/5bd9fbb3a5a985f80c16ddd0ec1dfc43-Abstract-Conference.html.

[25] Yanghao Li, Chao-Yuan Wu, Haoqi Fan, Karttikeya Mangalam, Bo Xiong, Jitendra Malik, and Christoph Feichtenhofer. MViTv2: Improved multiscale vision transformers for classification and detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 4804–4814, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Li_MViTv2_Improved_Multiscale_Vision_Transformers_for_Classification_and_Detection_CVPR_2022_paper.html.

[26] Mingbao Lin, Rongrong Ji, Yan Wang, Yichen Zhang, Baochang Zhang, Yonghong Tian, and Ling Shao. HRank: Filter pruning using high-rank feature map. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1529–1538, 2020. doi: 10.1109/CVPR42600.2020.00160. URL https://openaccess.thecvf.com/content_CVPR_2020/html/Lin_HRank_Filter_Pruning_Using_High-Rank_Feature_Map_CVPR_2020_paper.html.

[27] Liyang Liu, Shilong Zhang, Zhanghui Kuang, Aojun Zhou, Jing-Hao Xue, Xinjiang Wang, Yimin Chen, Wenming Yang, Qingmin Liao, and Wayne Zhang. Group fisher pruning for practical network compression. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 7021–7032. PMLR, 2021. URL https://proceedings.mlr.press/v139/liu21ab.html.

[28] Shiwei Liu, Tianlong Chen, Xiaohan Chen, Li Shen, Decebal Constantin Mocanu, Zhangyang Wang, and Mykola Pechenizkiy. The unreasonable effectiveness of random pruning: Return of the most naive baseline for sparse training. In *International Conference on Learning Representations*, 2022. doi: 10.48550/arXiv.2202.02643. URL https://openreview.net/forum?id=VBZJ_3tz-t.

[29] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 10012–10022, 2021. URL https://openaccess.thecvf.com/content/ICCV2021/html/Liu_Swin_Transformer_Hierarchical_Vision_Transformer_Using_Shifted_Windows_ICCV_2021_paper.html.

[30] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, Furu Wei, and Baining Guo. Swin transformer v2: Scaling up capacity and resolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12009–12019, 2022. doi: 10.1109/CVPR52688.2022.01170. URL https://openaccess.thecvf.com/content/CVPR2022/html/Liu_Swin_Transformer_V2_Scaling_Up_Capacity_and_Resolution_CVPR_2022_paper.html.

[31] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 11976–11986, 2022. URL https://openaccess.thecvf.com/content/CVPR2022/html/Liu_A_ConvNet_for_the_2020s_CVPR_2022_paper.html.

[32] Haiquan Lu, Yefan Zhou, Shiwei Liu, Zhangyang Wang, Michael W. Mahoney, and Yaoqing Yang. AlphaPruning: Using heavy-tailed self regularization theory for improved layer-wise pruning of large language models. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-0289.

[33] Ryan Lucas and Rahul Mazumder. Preserving deep representations in one-shot pruning: A hessian-free second-order optimization framework. In *International Conference on Learning Representations*, 2025. doi: 10.48550/arXiv.2411.18376. URL https://openreview.net/forum?id=eNQp79A5Oz.

[34] Tanvir Mahmud, Burhaneddin Yaman, Chun-Hao Liu, and Diana Marculescu. PaPr: Training-free one-step patch pruning with lightweight ConvNets for faster inference. In *Computer Vision – ECCV 2024*, volume 15081 of *Lecture Notes in Computer Science*, pages 110–128. Springer, 2024. doi: 10.1007/978-3-031-73337-6_7. URL https://doi.org/10.1007/978-3-031-73337-6_7.

[35] Charles H. Martin and Michael W. Mahoney. Heavy-tailed universality predicts trends in test accuracies for very large pre-trained deep neural networks. In *Proceedings of the 2020 SIAM International Conference on Data Mining*, pages 505–513, 2020. doi: 10.1137/1.9781611976236.57. URL https://epubs.siam.org/doi/10.1137/1.9781611976236.57.

[36] Charles H. Martin and Michael W. Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *Journal of Machine Learning Research*, 22(165):1–73, 2021. URL http://jmlr.org/papers/v22/20-410.html.

[37] Asit Mishra, Jorge Albericio Latorre, Jeff Pool, Darko Stosic, Dusan Stosic, Ganesh Venkatesh, Chong Yu, and Paulius Micikevicius. Accelerating sparse deep neural networks. *arXiv preprint arXiv:2104.08378*, 2021. doi: 10.48550/arXiv.2104.08378.

[38] Zizheng Pan, Bohan Zhuang, Jing Liu, Haoyu He, and Jianfei Cai. Scalable vision transformers with hierarchical pooling. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 367–376, 2021. doi: 10.1109/ICCV48922.2021.00043. URL https://openaccess.thecvf.com/content/ICCV2021/html/Pan_Scalable_Vision_Transformers_With_Hierarchical_Pooling_ICCV_2021_paper.html.

[39] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html.

[40] Alexandra Peste, Eugenia Iofinova, Adrian Vladu, and Dan Alistarh. AC/DC: Alternating compressed/decompressed training of deep neural networks. In *Advances in Neural Information Processing Systems*, volume 34, pages 8557–8570. Curran Associates, Inc., 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/48000647b315f6f00f913caa757a70b3-Abstract.html.

[41] Jeff Pool and Chong Yu. Channel permutations for N:M sparsity. In *Advances in Neural Information Processing Systems*, volume 34, pages 13316–13327, 2021. URL https://proceedings.neurips.cc/paper/2021/hash/6e8404c3b93a9527c8db241a1846599a-Abstract.html.

[42] Chaitanya Ryali, Yuan-Ting Hu, Daniel Bolya, Chen Wei, Haoqi Fan, Po-Yao Huang, Vaibhav Aggarwal, Arkabandhu Chowdhury, Omid Poursaeed, Judy Hoffman, Jitendra Malik, Yanghao Li, and Christoph Feichtenhofer. Hiera: A hierarchical vision transformer without the bells-and-whistles. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 29441–29454. PMLR, 2023. URL https://proceedings.mlr.press/v202/ryali23a.html.

[43] Maying Shen, Hongxu Yin, Pavlo Molchanov, Lei Mao, Jianna Liu, and Jose M. Alvarez. Structural pruning via latency-saliency knapsack. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 12894–12908. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/5434be94e82c54327bb9dcaf7fca52b6-Paper-Conference.pdf.

[44] Yitzchak Shmalo, Jonathan Jenkins, and Oleksii Krupchytskyi. Deep learning weight pruning with rmt-svd: Increasing accuracy and reducing overfitting. *arXiv preprint arXiv:2303.08986*, 2023. doi: 10.48550/arXiv.2303.08986. URL https://arxiv.org/abs/2303.08986.

[45] Zhuoran Song, Yihong Xu, Zhezhi He, Li Jiang, Naifeng Jing, and Xiaoyao Liang. Cp-vit: Cascade vision transformer pruning via progressive sparsity prediction. *arXiv preprint arXiv:2203.04570*, 2022. doi: 10.48550/arXiv.2203.04570. URL https://arxiv.org/abs/2203.04570.

[46] Max Staats, Matthias Thamm, and Bernd Rosenow. Boundary between noise and information applied to filtering neural network weight matrices. *Physical Review E*, 108(2):L022302, 2023. doi: 10.1103/PhysRevE.108.L022302.

[47] Andreas Steiner, Alexander Kolesnikov, Xiaohua Zhai, Ross Wightman, Jakob Uszkoreit, and Lucas Beyer. How to train your ViT? data, augmentation, and regularization in vision transformers. *Transactions on Machine Learning Research*, 2022. URL https://openreview.net/forum?id=4nPswr1KcP.

[48] Xinglong Sun, Barath Lakshmanan, Maying Shen, Shiyi Lan, Jingde Chen, and Jose M. Alvarez. Mdp: Multidimensional vision model pruning with latency constraint. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 20113–20123, 2025. doi: 10.1109/CVPR52734.2025.01873. URL https://openaccess.thecvf.com/content/CVPR2025/html/Sun_MDP_Multidimensional_Vision_Model_Pruning_with_Latency_Constraint_CVPR_2025_paper.html.

[49] Endri Taka, Ning-Chi Huang, Chi-Chih Chang, Kai-Chiang Wu, Aman Arora, and Diana Marculescu. Systolic sparse tensor slices: FPGA building blocks for sparse and dense AI acceleration. In *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, pages 159–171. ACM, 2025. doi: 10.1145/3706628.3708867. URL https://doi.org/10.1145/3706628.3708867.

[50] Yijun Tan, Kai Han, Kang Zhao, Xianzhi Yu, Zidong Du, Yunji Chen, Yunhe Wang, and Jun Yao. Accelerating sparse convolution with column vector-wise sparsity. In *Advances in Neural Information Processing Systems*, volume 35, pages 30307–30317. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/c383e44d9a878d1982d9abb838bd5d8a-Paper-Conference.pdf.

[51] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 10347–10357. PMLR, 18–24 Jul 2021. URL https://proceedings.mlr.press/v139/touvron21a.html.

[52] Hugo Touvron, Matthieu Cord, Alexandre Sablayrolles, Gabriel Synnaeve, and Hervé Jégou. Going deeper with image transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 32–42, October 2021. doi: 10.1109/ICCV48922.2021.00010. URL https://openaccess.thecvf.com/content/ICCV2021/html/Touvron_Going_Deeper_With_Image_Transformers_ICCV_2021_paper.html.

[53] Zhengzhong Tu, Hossein Talebi, Han Zhang, Feng Yang, Peyman Milanfar, Alan Bovik, and Yinxiao Li. MaxViT: Multi-axis vision transformer. In *Computer Vision – ECCV 2022*, volume 13684 of *Lecture Notes in Computer Science*, pages 459–479. Springer, 2022. doi: 10.1007/978-3-031-20053-3_27. URL https://doi.org/10.1007/978-3-031-20053-3_27.

[54] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. PVT v2: Improved baselines with pyramid vision transformer. *Computational Visual Media*, 8(3):415–424, 2022. doi: 10.1007/s41095-022-0274-8. URL https://doi.org/10.1007/s41095-022-0274-8.

[55] Ross Wightman. Pytorch image models. https://github.com/rwightman/pytorch-image-models, 2019.

[56] Jingjing Xie, Yuxin Zhang, Mingbao Lin, Zhihang Lin, Liujuan Cao, and Rongrong Ji. UniPTS: A unified framework for proficient post-training sparsity. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5746–5755,

June 2024. URL https://openaccess.thecvf.com/content/CVPR2024/html/Xie_UniPTS_A_Unified_Framework_for_Proficient_Post-Training_Sparsity_CVPR_2024_paper.html.

[57] Jingjing Xie, Yuxin Zhang, Mingbao Lin, Zhihang Lin, Liujuan Cao, and Rongrong Ji. UniPTS: A unified framework for proficient post-training sparsity. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5746–5755, June 2024. doi: 10.1109/CVPR52733.2024.00549. URL https://openaccess.thecvf.com/content/CVPR2024/html/Xie_UniPTS_A_Unified_Framework_for_Proficient_Post-Training_Sparsity_CVPR_2024_paper.html.

[58] Kaixin Xu, Zhe Wang, Chunyun Chen, Xue Geng, Jie Lin, Xulei Yang, Min Wu, Xiaoli Li, and Weisi Lin. LPViT: Low-power semi-structured pruning for vision transformers. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 269–287, 2024. doi: 10.1007/978-3-031-73209-6_16.

[59] Lu Yin, You Wu, Zhenyu Zhang, Cheng-Yu Hsieh, Yaqing Wang, Yiling Jia, Gen Li, Ajay Jaiswal, Mykola Pechenizkiy, Yi Liang, Michael Bendersky, Zhangyang Wang, and Shiwei Liu. Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity, 2024. URL https://arxiv.org/abs/2310.05175. ICML 2024.

[60] Chong Yu. Minimally invasive surgery for sparse neural networks in contrastive manner. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 3589–3598, 2021. doi: 10.1109/CVPR46437.2021.00359. URL https://openaccess.thecvf.com/content/CVPR2021/html/Yu_Minimally_Invasive_Surgery_for_Sparse_Neural_Networks_in_Contrastive_Manner_CVPR_2021_paper.html.

[61] Chong Yu, Tao Chen, Zhongxue Gan, and Jiayuan Fan. Boost vision transformer with GPU-friendly sparsity and quantization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22658–22668, June 2023. doi: 10.1109/CVPR52729.2023.02170. URL https://openaccess.thecvf.com/content/CVPR2023/html/Yu_Boost_Vision_Transformer_With_GPU-Friendly_Sparsity_and_Quantization_CVPR_2023_paper.html.

[62] Li Yuan, Qibin Hou, Zihang Jiang, Jiashi Feng, and Shuicheng Yan. VOLO: Vision outlooker for visual recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45 (5):6575–6586, 2023. doi: 10.1109/TPAMI.2022.3206108. URL https://ieeexplore.ieee.org/document/9888055.

[63] Yuxin Zhang, Mingbao Lin, ZhiHang Lin, Yiting Luo, Ke Li, Fei Chao, Yongjian Wu, and Rongrong Ji. Learning best combination for efficient N:M sparsity. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 941–953. Curran Associates, Inc., 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/file/06589ec9d86876508600a678f9c8f51d-Paper-Conference.pdf.

[64] Kang Zhao, Tao Yuan, Han Bao, Zhenfeng Su, Chang Gao, Zhaofeng Sun, Zichen Liang, Liping Jing, and Jianfei Chen. Beyond 2:4: Exploring V:N:M sparsity for efficient transformer inference on GPUs, 2024. URL https://arxiv.org/abs/2410.16135.

[65] Aojun Zhou, Yukun Ma, Junnan Zhu, Jianbo Liu, Zhijie Zhang, Kun Yuan, Wenxiu Sun, and Hongsheng Li. Learning N:M fine-grained structured sparse neural networks from scratch. In *International Conference on Learning Representations*, 2021. URL https://openreview.net/forum?id=K9bw7vqp_s.

[66] Michael Zhu and Suyog Gupta. To prune, or not to prune: exploring the efficacy of pruning for model compression, 2017. URL https://arxiv.org/abs/1710.01878.

[67] Mingjian Zhu, Yehui Tang, and Kai Han. Vision transformer pruning. *arXiv preprint arXiv:2104.08500*, 2021. doi: 10.48550/arXiv.2104.08500. URL https://arxiv.org/abs/2104.08500. Accepted by the KDD 2021 Workshop on Model Mining.